

Master's thesis

Master's Programme in Computer Science

Hiệu năng, an toàn và đánh đổi kỹ thuật của Rust và C++ trong hệ thống điều khiển RAN của Nokia

Ha Vien

August 24, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Hiệu năng, an toàn và đánh đổi kỹ thuật của Rust và C++ trong hệ thống điều khiển RAN của Nokia

Ohjaajat — Handledare — Supervisors

Prof. Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	August 24, 2026	110 pages, 9 appendix pages

Tiivistelmä — Referat — Abstract

Bối cảnh. Phần mềm mặt điều khiển của Mạng truy nhập vô tuyến (RAN) chủ yếu được viết bằng C++ vì hiệu năng, nhưng C++ mang theo rủi ro về an toàn bộ nhớ và tương tranh. Rust hứa hẹn hiệu năng tương đương với an toàn được bảo đảm tại thời điểm biên dịch, song mức độ phù hợp cho các mặt điều khiển viễn thông nhạy độ trễ vẫn chưa được chứng minh. Công trình được thực hiện tại Nokia trên một phân hệ mô hình hoá theo một thành phần gNB-CU-CP.

Mục tiêu. Luận văn so sánh C++ hiện đại và Rust trên một dịch vụ Quản lý Ngữ cảnh UE. Vì thông lượng trung bình tương đương đã được biết, luận văn nhắm tới bốn khía cạnh ít được khảo sát hơn: độ trễ đuôi và độ dao động khi quá tải, những lớp lỗi C++ nào Rust ngăn chặn khi biên dịch, chi phí của mô hình an toàn, và chi phí áp dụng tăng dần trong một mã nguồn C++ sẵn có.

Phương pháp. Trước hết, một đặc tả độc lập ngôn ngữ được cố định. Sau đó hai bản hiện thực tương đương về chức năng được xây dựng với cùng thuật toán, mô hình tương tranh, bộ cấp phát bộ nhớ và cấu hình biên dịch. Một khối lượng công việc tổng hợp được quét đến mức quá tải. Cả hai bản đều vượt qua cùng bộ kiểm thử tương đương, và phần đánh giá bổ sung một nghiên cứu tiêm lỗi và một nguyên mẫu FFI.

Kết quả. Rust an toàn từ chối cả năm lớp lỗi an toàn bộ nhớ và tranh chấp dữ liệu được tiêm vào tại thời điểm biên dịch. Ở C++ cùng năm lớp đó biên dịch sạch và chỉ lộ ra lúc chạy, nếu bộ dò lỗi bắt được. Năm lớp nữa thất bại ở cả hai ngôn ngữ. Trên hai lớp trong số đó công cụ C++ báo cáo những gì Rust không báo. Đường ghép sang C++ giữ cho lõi không cần `unsafe` và tốn tối đa 10.91% của một thao tác. Hiệu năng thay đổi tại điểm bão hoà. Dưới đó trung vị độ trễ khớp nhau, song một lời gọi đầu-cuối chỉ dành 0.250% cho công việc của ngôn ngữ. Phần còn lại là bàn giao giữa các luồng, nên phép quét phản ánh thiết kế hơn là ngôn ngữ. Đo riêng thao tác, Rust nhanh hơn 1.6 lần và tiến trình lớn hơn vài MiB. Trên bão hoà thứ tự đảo ngược: C++ giữ 1.8 lần thông lượng của Rust trong 20/20 lần lặp đan xen với cả hai bộ cấp phát. Phần đuôi xa dưới bão hoà không thể kết luận trên máy ảo hoá.

Kết luận. Chi phí chính khi áp dụng Rust là công sức kỹ thuật và đường cong học tập, không phải hiệu năng lúc chạy ở tải danh nghĩa. Lợi ích là thoát khỏi tranh chấp dữ liệu, chứ không phải mọi lỗi tương tranh. Quá tải thu hẹp tuyên bố đó: một dịch vụ phải suy giảm êm phụ thuộc vào các nguyên thủy đồng bộ hoá của thư viện chuẩn, không phải mô hình an toàn, và một thử nghiệm FFI [illegible]

Avainsanat — Nyckelord — Keywords

Rust, C++, đặc tả thực thi, HAN, ngữ nghĩa, an toàn bộ nhớ, tiêm lỗi, FFI

Säilytyspaikka — Förvaringsställe — Where deposited

Thư viện Đại học Helsinki

Muita tietoja — övriga uppgifter — Additional information

ACM Computing Classification System (CCS)

Software and its engineering → Software notations and tools → General programming languages

Computer systems organization → Real-time systems → Real-time system specification

Networks → Network types → Mobile networks

Software study track

Mục lục

List of Figures **v**

List of Tables **vii**

1 Giới thiệu **1**

1.1 Động lực và bối cảnh công nghiệp 2
1.2 Phát biểu vấn đề 5
1.3 Câu hỏi nghiên cứu và giả thuyết 5
1.4 Đóng góp, kết quả và hạn chế 6
1.5 Ý nghĩa và đối tượng hưởng lợi 7

2 Nền tảng: Các ngôn ngữ hệ thống cho phần mềm nhạy độ trễ **9**

2.1 Hệ thống mặt điều khiển RAN và gNB-CU-CP 9
2.2 Độ trễ, thông lượng và khả năng dự đoán trong phần mềm thời gian thực 10
2.3 C++ cho hệ thống nhạy độ trễ 12
2.4 Mô hình sở hữu và an toàn của Rust 13
2.5 Mô hình tương tranh cho phần mềm mặt điều khiển 16
2.6 Khả năng tương tác giữa C++ và Rust 17
2.7 Khoảng trống nghiên cứu 17

3 Bối cảnh bài toán: Quản lý Ngữ cảnh UE và tính dự đoán được **21**

3.1 Phân hệ Quản lý Ngữ cảnh UE 21
3.2 Các lớp lỗi an toàn bộ nhớ và đồng thời 22
3.3 Độ trễ đuôi và tính dự đoán được trong RAN 25

4 Phương pháp nghiên cứu **27**

4.1 Hướng tiếp cận nghiên cứu 27
4.2 Phương pháp cho bốn câu hỏi nghiên cứu 28

4.2.1 Đo hiệu năng dưới quét tải (RQ1, RQ3) . . . 29
4.2.2 Tiêm lỗi có kiểm soát (RQ2) . . . 30
4.2.3 Nghiên cứu tình huống di trú giao diện hàm ngoại (RQ4) . . . 31
4.3 Công bằng như chiến lược biến kiểm soát . . . 31
4.4 Xử lý thống kê các phép đo . . . 33
4.5 Tái lập và hiện vật nghiên cứu . . . 35

5 Thiết kế và Triển khai 37
5.1 Đặc tả dùng chung: chuẩn công bằng . . . 37
5.2 Mô hình khái niệm và thiết kế đồng thời . . . 37
5.3 Lựa chọn kiến trúc và ngăn xếp công nghệ . . . 38
5.4 Chi tiết triển khai . . . 39
5.4.1 Ngữ cảnh UE và máy trạng thái . . . 39
5.4.2 Phân mảnh, store và hàng đợi có giới hạn . . . 41
5.4.3 Yêu cầu, phản hồi và vòng đời dịch vụ . . . 42
5.4.4 Điểm bắt buộc theo ngôn ngữ và chi phí hai lõi . . . 43
5.5 API công khai và ngữ nghĩa thao tác . . . 46
5.6 Kiểm tra tương đồng . . . 46

6 Thiết lập thực nghiệm và Đánh giá 49
6.1 Cấu hình testbed . . . 49
6.2 Workload và phép đo đánh giá . . . 50
6.3 Dữ liệu đánh giá . . . 52
6.4 Kết quả đánh giá . . . 53
6.4.1 Hiệu năng (RQ1, RQ3) . . . 53
6.4.2 End-to-end số được ghép từ gì . . . 57
6.4.3 Độ trễ, throughput và bão hòa qua sweep . . . 59
6.4.4 Hành vi qua bão hòa (RQ1) . . . 61
6.4.5 Hàng đợi tranh chấp và các primitive hoàn thành (RQ1) . . . 63
6.4.6 Sử dụng tài nguyên (RQ3) . . . 64
6.4.7 An toàn qua tiêm lỗi (RQ2) . . . 65
6.4.8 Di chuyển (RQ4) . . . 71

7 Thảo luận 77
7.1 Tóm tắt phát hiện . . . 77
7.2 Thảo luận về các câu hỏi nghiên cứu . . . 79

7.3 Xem lại các giả thuyết 83
7.4 Hướng dẫn kỹ thuật cho áp dụng tăng dần 84
7.5 Công trình liên quan 86
7.6 Mối đe doạ tính hợp lệ và biện pháp giảm thiểu 91
7.7 Suy ngẫm đạo đức và chuyên môn 96

8 Kết luận và Công việc tương lai **97**

Statement on the Use of AI Tools **101**

Tài liệu tham khảo **103**

A Viết tắt và thuật ngữ chính

B Giao thức tìm kiếm tài liệu

C Thiết kế thống kê: thành viên họ và khả năng phân giải

D Cấu hình máy chủ và chiến dịch

E Ví dụ tiêm lỗi

List of Figures

2.1 Vị trí phân hệ nghiên cứu trong NG-RAN 5G 10
2.2 Vì sao đuôi, không phải trung bình, quyết định hạn chót RAN 11
2.3 Coordinated omission dưới tải vòng mở và vòng kín 12
2.4 Cùng use-after-free trong C++ và Rust an toàn 14
2.5 Hai mô hình tương tranh cho trạng thái mặt điều khiển 16
2.6 Điều các nghiên cứu lân cận kiểm soát và để thay đổi 19

3.1 Máy trạng thái vòng đời ngữ cảnh UE 21
3.2 Vách quá tải 26

5.1 Đường đi end-to-end của một thao tác 38
5.2 Kiến trúc không chia sẻ, một luồng cho mỗi shard 38
5.3 Mô hình dữ liệu ngữ cảnh UE dùng chung 40

6.1 Hai workload và vòng đời workload mixed đi qua 51
6.2 Họ test được vẽ dưới dạng hiệu ứng ghép 56
6.3 Thao tác end-to-end được phân rã, so với sweep có thể giải quyết . . . 59
6.4 Độ trễ đuôi so với tải đề nghị 60
6.5 Throughput đạt so với tải đề nghị 60
6.6 Độ trễ qua dải percentile, và giới hạn host 61
6.7 Phân kỳ qua bão hòa 62
6.8 Sử dụng tài nguyên và hai hiệu ứng trong peak RSS 65
6.9 Nơi bảo đảm dừng: ma trận kết quả 71
6.10 Đường nối di chuyển FFI C++-sang-Rust 72
6.11 Chi phí đường nối trên một thao tác, so với độ không chắc riêng . . . 73

7.1 Nơi bảo đảm biên dịch của Rust dừng 81

List of Tables

2.1 Cách mỗi ngôn ngữ thực thi các thuộc tính được đo 15

3.1 Danh mục tiêm và hiện thân trong UECM 25

4.1 Thiết kế nghiên cứu: cách trả lời từng câu hỏi 28
4.2 Chiến lược biến kiểm soát 32

5.1 Bảng chuyển trạng thái triển khai 41
5.2 Log điểm bắt buộc theo ngôn ngữ 44
5.3 Proxy kích thước và cyclomatic complexity cho hai lõi 45
5.4 Nguồn gốc khác biệt số dòng giữa hai lõi 45

6.1 Độ trễ mỗi thao tác ở tải đại diện 54
6.2 Test ghép hoán vị chính xác C++ so với Rust 55
6.3 Mỗi hiệu ứng có thể lớn bao nhiêu, cho so sánh test không giải quyết . 57
6.4 Chi phí end-to-end thực sự đi đâu 57
6.5 Mỗi variant vẫn duy trì gì qua bão hòa 62
6.6 Test ghép chính xác qua bão hòa 63
6.7 Thông lượng qua các primitive đồng bộ tranh chấp riêng 63
6.8 Sử dụng tài nguyên ở tải đại diện 65
6.9 Phân loại lớp lỗi: kết quả đo 70
6.10 Nguyên mẫu di chuyển FFI: kích thước đường nối, an toàn và chi phí . 73
6.11 Trục chi phí áp dụng: thời gian build, dấu chân tĩnh, bề mặt bên thứ ba 74

7.1 Tóm tắt phát hiện theo câu hỏi nghiên cứu 78
7.2 Nghiên cứu này cạnh năm luận văn nhà cung cấp 90
7.3 Mối đe doạ tính hợp lệ, biện pháp giảm thiểu và phần còn lại 93

A.1 Viết tắt và thuật ngữ chính i

D.1 Máy chủ đã dùng và giao thức testbed còn phải thực hiện i

D.2 Cấu hình dịch vụ, khối lượng công việc và chiến dịch ii

1 Giới thiệu

Mạng di động đang chuyển từ phần cứng chuyên dụng sang phần mềm trên máy chủ thông thường. Các chức năng từng chạy trên thiết bị tích hợp theo chiều dọc, chuyên biệt cố định, nay được tái hiện thực thành chương trình trên máy tính đa mục đích. Mạng truy nhập vô tuyến (RAN), lâu nay là phần phụ thuộc phần cứng nhất trong ngăn xếp, cũng nằm trong xu hướng đó.

Trong 5G, trạm gốc (gNB) được tách thành một đơn vị trung tâm và một hoặc nhiều đơn vị phân tán. Đơn vị trung tâm lại được tách thành các chức năng mặt điều khiển và mặt người dùng [1, 2]. Open-RAN đẩy cùng ý tưởng xa hơn: RAN trở thành phần mềm ảo hoá, đa nhà cung cấp, nối với nhau qua giao diện mở [3]. Mặt điều khiển thu được, gNB-CU-CP, là một chương trình lớn, có trạng thái, tương tranh cao. Nó theo dõi mọi thiết bị đang kết nối, chạy báo hiệu RRC [4], và điều phối di động cùng chuyển giao, tất cả trong giới hạn độ trễ nghiêm ngặt.

Tải tại một site bận rộn rất cao. Một site có thể chạy hàng nghìn thủ tục quản lý kết nối và di động mỗi giây, mỗi thủ tục có ngân sách vài mili giây [5]. Một mặt điều khiển triển khai có thể theo dõi hàng triệu thiết bị hoạt động cùng lúc. Khi RAN chuyển sang triển khai đám mây, tách rời, tính đúng đắn, độ tin cậy và thời gian của phần mềm này ngày càng quyết định chất lượng của toàn mạng. Một sự cố hoặc một đỉnh độ trễ đuôi ở đây không phải lỗi cục bộ. Mọi thiết bị site phục vụ đều bị ảnh hưởng [6, 7].

Phần mềm mặt điều khiển này, hiện nay, chủ yếu được viết bằng C++. Lựa chọn đó là hợp lý: C++ mang lại kiểm soát mức thấp, công cụ trưởng thành và hiệu năng mà hệ thống nhạy độ trễ đòi hỏi. Nhưng hiệu năng đó đi kèm chi phí đã được biết. C++ đặt gánh nặng quản lý bộ nhớ và tương tranh lên lập trình viên, và hàng thập kỷ kinh nghiệm cho thấy gánh nặng này thường xuyên bị xử lý sai, sinh ra lỗi use-after-free, double-free, tranh chấp dữ liệu và các lỗi liên quan, tốn kém để tìm, nguy hiểm khi vận hành, và chiếm phần lớn lỗ hổng bảo mật trong thực tiễn công nghiệp [8, 9]. Rust đã nổi lên như ngôn ngữ hệ thống đại chúng đầu tiên hứa hẹn thoát khỏi đánh đổi này: an toàn bộ nhớ và không tranh chấp dữ liệu được thực thi lúc biên dịch, với hiệu năng dự kiến cạnh tranh với C++ [10, 11]. Liệu lời hứa đó có đúng với mặt điều khiển cấp viễn thông, nhạy độ trễ, nơi khả năng dự đoán khi quá tải quan trọng ngang hiệu năng thuần, vẫn chưa được hiểu rõ. Luận văn này, thực hiện tại Nokia, trả lời câu hỏi trực tiếp: phát triển hai bản hiện thực tương đương về chức năng của một phân hệ RAN đại diện, một dịch vụ Quản lý Ngữ cảnh UE mô hình hoá theo một thành phần gNB-CU-CP thật của Nokia, bằng C++ hiện đại và Rust, rồi so sánh chúng dưới khối lượng báo hiệu tổng hợp sát thực tế.

Ba thuật ngữ lặp lại xuyên suốt. *gNB-CU-CP* là phần mềm mặt điều khiển mà công trình này xoay quanh. *Quản lý Ngữ cảnh UE (UECM)* là phân hệ cụ thể được tái hiện thực và đo lường. *Độ trễ đuôi*, các phân vị p99 và p99.9 thay vì trung bình, là

thuộc tính quyết định liệu hệ thống như vậy có đáp ứng hạn chót hay không [6, 7]. Phụ lục A cố định các thuật ngữ này cùng phần còn lại của từ vựng miền cho độc giả ngoài ngành viễn thông.

1.1 Động lực và bối cảnh công nghiệp

Ba xu hướng khiến phép so sánh này đúng thời điểm. Xu hướng thứ nhất là tiến hoá đám mây của chính RAN. Đưa các chức năng mặt điều khiển lên máy chủ đa mục đích làm tăng, chứ không giảm, đòi hỏi về chất lượng phần mềm: phần mềm sống lâu hơn, liên tục tiến hoá, chạy với mức tương tranh cao hơn, và phải đáp ứng hạn chót mà không còn tính xác định mà phần cứng chuyên dụng từng cung cấp. Xu hướng này được đẩy nhanh bởi tách rời Open-RAN, biến trạm gốc thành phần mềm ảo hoá, đa nhà cung cấp, nối qua giao diện mở [3], nên mặt điều khiển mà nhà cung cấp giao hàng ngày càng cạnh tranh bằng chất lượng kỹ thuật phần mềm (độ tin cậy, thời gian dự đoán được, bảo mật, và mức độ dễ tiến hoá an toàn một mã nguồn lớn) thay vì phần cứng chuyên dụng. Trong bối cảnh đó, một hồi quy độ trễ hoặc một lỗi hỏng bộ nhớ không còn là lỗi cục bộ mà là rủi ro tin cậy và bảo mật toàn mạng, và câu hỏi chọn ngôn ngữ không còn học thuật mà trở thành quyết định kỹ thuật cụ thể.

Xu hướng thứ hai là AI chuyển từ công cụ tinh chỉnh mạng sang khối lượng công việc mà mạng được xây để vận chuyển, và thế hệ kế tiếp đang được đặc tả theo hướng đó ngay từ đầu. IMT-2030, khung ITU cho 6G, liệt kê trí tuệ nhân tạo và truyền thông trong sáu kịch bản sử dụng và trí tuệ phổ biến trong bốn nguyên tắc thiết kế [12]. Kiến trúc hình thành quanh đó là AI-RAN, trong đó viễn thông và khối lượng AI được điều phối chung trên hạ tầng gia tốc dùng chung, nên các site chạy vô tuyến cũng lưu trữ nền tảng AI biên phân tán [13]. Liên minh AI-RAN nêu thẳng yêu cầu vận hành: một hạ tầng chung phải chạy đồng thời cả hai loại khối lượng, và thỏa thuận mức dịch vụ của chúng có thể khác nhau [14]. Điều đó thay đổi những gì lựa chọn ngôn ngữ phải mang lại cho mặt điều khiển. Phần mềm của nó không còn chiếm trọn máy chủ. Nó tranh CPU, bộ nhớ và cache với công việc suy luận có mẫu đến nó không kiểm soát, và đồng ở việc chiếm dụng làm giảm tải mà mặt điều khiển bão hoà. Bão hoà là nơi hai ngôn ngữ tách biệt rõ nhất trong các đo lường của Chương 6. Dấu chân bộ nhớ và hành vi sau điểm đó đang trở thành tiêu chí lựa chọn riêng, và không tiêu chí nào được quyết định bởi kiến trúc phía trên mã nguồn.

Xu hướng thứ ba là chuyển dịch toàn ngành sang ngôn ngữ an toàn bộ nhớ. Các tổ chức phần mềm lớn báo cáo phần lớn lỗ hổng bảo mật nghiêm trọng bắt nguồn từ lỗi an toàn bộ nhớ, và việc áp dụng ngôn ngữ an toàn bộ nhớ giảm chúng đo được [8, 9, 15, 16]. Luận điểm không còn chỉ dựa vào báo cáo nhà cung cấp: một nhóm gồm học thuật và công nghiệp đã lập luận rằng an toàn bộ nhớ giờ cần thuật ngữ và yêu cầu chuẩn hoá nếu muốn đặc tả và mua sắm [17]. Mối quan tâm đã lên tầm chính sách quốc gia: cơ quan an ninh mạng hiện khuyến khích nhà sản xuất phần mềm công bố "lộ trình an toàn bộ nhớ" và chuyển khỏi ngôn ngữ không an toàn bộ nhớ

như C và C++ [18, 19]. Trong thị trường mà phân hệ này được bán vào, vấn đề cũng trở thành quy định. Đạo luật Khả năng phục hồi mạng EU đặt nghĩa vụ thiết kế an toàn và xử lý lỗ hổng trong vòng đời lên nhà sản xuất sản phẩm có yếu tố số, với nghĩa vụ báo cáo lỗ hổng từ tháng 9 năm 2026 và phần lớn phần còn lại từ tháng 12 năm 2027 [20]. Văn bản không nêu ngôn ngữ lập trình, và không nên đọc nó như bắt buộc một ngôn ngữ. Điều nó thay đổi là các lớp lỗi lặp lại của ngôn ngữ không an toàn bộ nhớ trở thành rủi ro tuân thủ cũng như rủi ro kỹ thuật. Rust đặc biệt đã chuyển từ ngôn ngữ ngách vào nhân hệ điều hành, hạ tầng đám mây, và ngày càng vào viễn thông. Siêu phần ảo hoá sản xuất như AWS Firecracker, nền tảng của nền tảng serverless, đã được viết bằng Rust [21]. Song cơ sở bằng chứng về mức phù hợp của Rust chủ yếu dựa trên phần mềm đa mục đích và benchmark hiệu năng trường hợp trung bình. Câu hỏi cụ thể mà nhà cung cấp viễn thông phải trả lời, liệu Rust có đáp ứng yêu cầu độ trễ đuôi và khả năng dự đoán của mặt điều khiển RAN, liệu nó có thực sự ngăn các lớp lỗi ám ảnh mã C++ hiện có, và chi phí áp dụng tăng dần trong mã nguồn C++ lớn là bao nhiêu, chưa được trả lời bằng bằng chứng trực tiếp, có kiểm soát. Cung cấp bằng chứng như vậy cho một phân hệ đại diện là động lực của công trình này.

Bối cảnh công nghiệp. Luận văn này được thực hiện tại Nokia, một trong số ít nhà cung cấp xây dựng, duy trì và tiến hoá phần mềm RAN sản xuất từ đầu đến cuối. Thị trường đó tập trung rất ít tay: năm nhà cung cấp lớn nhất chiếm 96% doanh thu RAN toàn cầu trong ba quý đầu năm 2025, với chỉ số Herfindahl–Hirschman ở mức cao nhất trong mười năm [22, 23]. Phân hệ nghiên cứu, Quản lý Ngữ cảnh UE (UECM), được mô hình hoá theo một thành phần mặt điều khiển gNB-CU-CP thật chứ không phải ví dụ sách giáo khoa.

gNB-CU-CP không phải thành phần phụ. Trong kiến trúc tách rời 3GPP, nó là nửa mặt điều khiển của đơn vị trung tâm, lưu trữ RRC và quản lý vòng đời ngữ cảnh UE [1, 4], dưới ngân sách thủ tục cấp mili giây mà hệ thống 5G đặt lên báo hiệu điều khiển [5] và ở tốc độ sự kiện cùng số thiết bị kết nối mà mạng triển khai duy trì. Mục 2.1 đặt nó trong kiến trúc rộng hơn. Vì được viết bằng C++, nó thừa hưởng cả hiệu năng lẫn rủi ro an toàn bộ nhớ và tương tranh của ngôn ngữ đó. Đó chính là nơi ba xu hướng trên hội tụ, và phần mềm loại này được giao hàng, tiến hoá và vận hành trên mạng nhà mạng thật, nên quyết định chọn ngôn ngữ là mối quan tâm sản xuất chứ không phải giả định.

UECM là đối tượng nghiên cứu có chủ đích trong bối cảnh đó, vì các lý do Mục 3.1 nêu: đủ nhỏ để tái hiện thực trung thực và lý giải toàn bộ, song vẫn mang các thuộc tính khiến mã mặt điều khiển khó. Hiện thực hoá hoàn toàn theo một đặc tả công khai, đã đóng băng, chứ không theo mã độc quyền, để phép so sánh có thể tái lập bởi độc giả ngoài Nokia.

Một quyết định mang tầm một thế hệ. Lựa chọn ngôn ngữ trong phần mềm loại này không thường xuyên được xem xét lại, và nó sống lâu hơn thế hệ mà nó được

đưa ra. Thế hệ kế tiếp đang được đặc tả ngay bây giờ: 3GPP đã phân bổ Release 20 cho nghiên cứu 6G và Release 21 cho công việc chuẩn hoá, theo lịch ITU yêu cầu đề xuất công nghệ đầu năm 2029 và nộp đặc tả giữa 2030 [24]. Công việc đó bao gồm rõ ràng di chuyển từ và tương tác với 5G, nên mặt điều khiển viết cho thế hệ hiện tại được duy trì song song thế hệ kế tiếp thay vì nghỉ hưu khi thế hệ đó tới. Mục tiêu thế hệ sau cũng không nới lỏng động lực của phép so sánh này. IMT-2030 đặt bảo mật và khả năng phục hồi là một trong bốn nguyên tắc thiết kế, kỳ vọng hệ thống an toàn ngay từ thiết kế, định nghĩa khả năng phục hồi là khả năng tiếp tục vận hành đúng trong và sau sự kiện gây rối, và mang kịch bản độ trễ thấp siêu tin cậy về ở dạng củng cố [12]. Thoát khỏi cả lớp lỗi và hành vi dự đoán được khi bị nhiễu đang được ghi vào mục tiêu thế hệ kế tiếp thay vì giao cho người hiện thực.

Các mốc chính sách nêu trên nằm trong cùng cửa sổ. CISA và FBI xếp phát triển dòng sản phẩm mới bằng ngôn ngữ không an toàn bộ nhớ là thực hành xấu khi sản phẩm phục vụ hạ tầng quan trọng, phạm vi mà hạ tầng mạng di động nằm trọn trong đó, và họ miễn sản phẩm hiện có khỏi kỳ vọng lộ trình an toàn bộ nhớ chỉ khi thời hạn hỗ trợ công bố kết thúc trước tháng 1 năm 2030 [25]. Nhà cung cấp hoãn câu hỏi ngôn ngữ đến khi thế hệ kế tiếp được đặc tả sẽ không tránh được việc quyết định. Họ sẽ quyết định mặc định, bằng ngôn ngữ hiện hành, cho thêm một thập kỷ triển khai.

Điều luận văn này đóng góp cho quyết định đó có giới hạn, và giới hạn đáng nêu rõ. Các đo lường ở đây lấy trên phân hệ mô hình hoá theo thành phần thế hệ hiện tại và đánh giá theo yêu cầu đã công bố. Không có tuyên bố về yêu cầu chưa được viết. Điều vượt ranh giới thế hệ không phải một con số độ trễ mà chính phép so sánh cấp ngôn ngữ: lớp lỗi nào trình biên dịch có thể loại trừ, mô hình an toàn tốn bao nhiêu, và áp dụng tăng dần trong mã nguồn C++ lớn gồm những gì. Các câu hỏi đó có cùng câu trả lời bất kể đặc tả phía trên gọi thế hệ nào, và rẻ nhất khi trả lời trước khi bắt đầu mã nguồn kế tiếp chứ không phải sau.

Một kiểu lỗi cụ thể. Để làm rõ mức độ, hãy xét một cơn bão di động khi nhiều UE chuyển giao cùng lúc. Một con trỏ xử lý sai, ngữ cảnh UE được giải phóng khi thả trong khi thông điệp báo hiệu đang bay vẫn tham chiếu tới nó, sinh ra use-after-free. Trong C++ mã này biên dịch sạch và có thể vượt kiểm thử thường, chỉ để hỏng trạng thái UE không liên quan hoặc làm sập luồng worker đúng điều kiện tương tranh cao mà hệ thống tồn tại để xử lý. Cùng một tranh chấp tiềm ẩn cũng có thể xuất hiện thành đỉnh độ trễ đuôi khó đoán thay vì sập hẳn. Trong Rust an toàn cùng thiết kế không biên dịch: sở hữu và mượn từ chối tham chiếu treo lúc build. Kịch bản này nắm lý do luận văn đo cả *an toàn* (lỗi nào được ngăn, và khi nào) lẫn *khả năng dự đoán* (độ trễ đuôi khi quá tải) thay vì chỉ thông lượng trung bình.

1.2 Phát biểu vấn đề

Hai thuộc tính của miền này khiến lựa chọn ngôn ngữ có hệ quả. Thứ nhất, quản lý bộ nhớ và tương tranh thủ công trong C++ để phần mềm mắc các lỗi như use-after-free, double-free, tranh chấp dữ liệu và vô hiệu hoá iterator, im lặng lúc biên dịch và tốn kém để tìm sửa ở quy mô viễn thông. Thứ hai, điều quyết định liệu hạn chót RAN được đáp ứng không phải thông lượng trung bình mà hành vi trường hợp xấu nhất và đuôi dưới tải và quá tải [6], thuộc tính dễ hồi quy và khó đo đúng. Rust hứa loại bỏ lớp lỗi thứ nhất lúc biên dịch trong khi tuyên bố hiệu năng cạnh tranh ở lớp thứ hai, song tuyên bố đó chủ yếu dựa trên benchmark đa mục đích thay vì khối lượng viễn thông có trạng thái, tương tranh, nhạy độ trễ.

Gộp các mạch lại, vấn đề cụ thể mà luận văn này giải quyết như sau. Nhà cung cấp viễn thông chịu áp lực ngày càng lớn, cả kỹ thuật lẫn quy định, để áp dụng ngôn ngữ an toàn bộ nhớ, song với mặt điều khiển RAN họ thiếu bằng chứng *trực tiếp, có kiểm soát* để căn cứ quyết định đó. Chưa được thiết lập liệu bản hiện thực Rust của phân hệ mặt điều khiển có trạng thái, tương tranh có khớp C++ hiện đại về *độ trễ đuôi và khả năng dự đoán khi quá tải* (trái với thông lượng trung bình), liệu đảm bảo lúc biên dịch của Rust có thực sự loại bỏ các lớp lỗi cụ thể ảnh hưởng mã C++ hiện có, và chi phí áp dụng tăng dần *là bao nhiêu* trong mã nguồn C++ lớn. Bằng chứng sẵn có chủ yếu đa mục đích và trường hợp trung bình [26, 27], để ba câu hỏi (khả năng dự đoán, phạm vi lỗi bị ngăn, và chi phí di chuyển) mở với một phân hệ đại diện. Trả lời chúng trong điều kiện có kiểm soát, tái lập được là vấn đề mà luận văn này đặt ra để giải quyết.

1.3 Câu hỏi nghiên cứu và giả thuyết

Luận văn được tổ chức quanh bốn câu hỏi nghiên cứu, sắp theo thứ tự ưu tiên khả năng dự đoán và di chuyển trước hiệu năng trung bình thuần:

RQ1 Rust và C++ hiện đại so sánh thế nào về khả năng dự đoán độ trễ (trung vị, độ trễ đuôi p99 và p99.9, và giá trị cực đại) cho khối lượng mặt điều khiển RAN dưới tải tăng dần và quá tải?

RQ2 Lớp lỗi an toàn bộ nhớ và tương tranh nào xảy ra ở C++ bị loại bỏ lúc biên dịch trong bản hiện thực Rust, và các lỗi còn lại được phát hiện ở giai đoạn nào với chi phí bao nhiêu?

RQ3 Chi phí, nếu có, của mô hình an toàn Rust (kiểm tra biên, sao chép do sở hữu, chi phí trừu tượng hoá) lên thông lượng, sử dụng CPU và hành vi bộ nhớ là gì?

RQ4 Hệ quả thực tiễn (công sức, khả năng tương tác FFI và rủi ro) của việc áp dụng Rust tăng dần trong hệ sinh thái phần mềm RAN C++ sẵn có là gì?

Các câu hỏi được tiếp cận với ba kỳ vọng trước, nêu ở đây để phần đánh giá có thể xác nhận hoặc bác bỏ thay vì chỉ mô tả kết quả sau sự kiện. **H1:** với khối lượng này hai ngôn ngữ tương đương hiệu năng trong vùng vận hành liên quan dưới bão hoà, mọi khác biệt chỉ nằm ở đuôi xa. **H2:** Rust an toàn ngăn lúc biên dịch các lớp lỗi an toàn bộ nhớ và tương tranh mà C++ giao cho phát hiện nỗ lực tốt nhất, sau sự kiện. **H3:** chi phí chạy của tương tác C++–Rust là không đáng kể so với chi phí các thao tác được gọi, nên chi phí thật của áp dụng tăng dần là công sức kỹ thuật chứ không phải hiệu năng. H2 được phát biểu để có thể sai. Danh mục lỗi phía sau RQ2 trải cả lớp Rust an toàn dự kiến ngăn và lớp dự kiến không ngăn, để nghiên cứu định vị ranh giới đảm bảo thay vì chỉ xác nhận. Các giả thuyết được xem lại theo bằng chứng ở Mục 7.3.

1.4 Đóng góp, kết quả và hạn chế

Luận văn có sáu đóng góp. Hai là hiện vật: một khung benchmark tái lập được cho khối lượng mặt điều khiển viễn thông, và một đặc tả chung đã đóng băng với hai bản hiện thực đã kiểm thử tương đương của phân hệ mặt điều khiển RAN, một bằng C++ và một bằng Rust. Ba là kết quả. Thứ nhất là so sánh độ trễ và tài nguyên chia tại điểm gối bão hoà, nơi thứ tự giữa hai ngôn ngữ đảo ngược. Thứ hai là phân loại lớp lỗi từ tiêm lỗi có kiểm soát đánh dấu nơi đảm bảo lúc biên dịch dừng lại, theo cả hai hướng. Thứ ba là giới hạn trên những gì phép so sánh dạng này có thể quy cho ngôn ngữ, thu được bằng cách phân rã một thao tác đầu-cuối thành công việc ngôn ngữ thực hiện và bàn giao chéo luồng quanh nó. Cuối cùng là tập hướng dẫn kỹ thuật và di chuyển cho chọn ngôn ngữ trong tiến hoá RAN đám mây (Mục 7.4). Cả sự đảo ngược lẫn giới hạn quy kết đều không được dự đoán trước. Cả hai được tìm thấy khi đọc kết quả mà nghiên cứu dự định báo cáo theo cách khác, và Mục 7.3 ghi chi phí của cái đầu đối với H1.

Các phát hiện chia ba nhánh. Về *an toàn* (RQ2), mọi lớp lỗi an toàn bộ nhớ và tranh chấp dữ liệu được tiêm đều bị ngăn lúc biên dịch trong Rust an toàn. Cùng các lỗi đó đều biên dịch trong C++, và mỗi lỗi chỉ bị bắt, nếu có, lúc chạy và chỉ dưới bộ dò lỗi đúng, hoặc với một lớp thì không có bộ dò nào. Bảy lớp ranh giới được chọn có chủ đích, và không ngôn ngữ nào ngăn chúng. Với hai lớp trong số đó công cụ C++ đi trước, và với một lớp lỗi mà Rust an toàn loại trừ quay lại ngay khi chương trình viết `unsafe`. Về *di chuyển* (RQ4), đường ghép FFI nhỏ và giữ lõi không cần `unsafe`. Đo so với cùng thao tác gọi bản địa, chi phí tối đa 10.91% của một thao tác, nên chi phí áp dụng thật là thiết kế API chứ không phải chi phí chạy. Ba chi phí ngoài đường ghép được đo vì cùng lý do, và hai trong số đó chống lại quan niệm thông thường: phần lớn chi phí build thêm của Rust là biên dịch mã bên thứ ba chứ không phải công việc của chính trình biên dịch, và thứ tự kích thước nhị phân đảo ngược khi hai chuỗi công cụ được giữ cùng một chính sách liên kết. Cả hai kết quả này là cuối cùng.

Về *hiệu năng* (RQ1, RQ3) câu trả lời chia hai lần, và vẽ đúng cả hai phân chia là nơi phần lớn phân tích đi vào. Phân chia thứ nhất tách những gì phép quét đo khỏi

những gì nó có thể quy kết. Một thao tác đầu-cuối trong thiết kế này là 0.250% công việc ngôn ngữ và bàn giao chéo luồng chiếm phần còn lại, nên so sánh cặp khớp trên đường đó giới hạn thiết kế chứ không phải ngôn ngữ. Giới hạn đó đúng bất kể phép so sánh như vậy ra sao, và chạm mọi nghiên cứu báo cáo trung vị đầu-cuối mà không đo tỷ lệ phần. Đo tách khỏi bàn giao, thao tác nhanh hơn 1.6 lần trong Rust, điều mà phép quét không thấy.

Phân chia thứ hai là theo chế độ. Dưới bão hoà hai bản hiện thực tương đương đầu-cuối về hành vi trung tâm, với biên độ không triển khai nào nhận ra. Kiểm định cặp vẫn phân giải khác biệt trung vị $1.6\,\mu$s nghiêng về Rust dưới cả hai bộ cấp phát, với khoảng tin cậy chạm $2\,\mu$s, nên tương đương là thực tiễn chứ không phải thống kê. Đuôi xa cùng chế độ không thể kết luận trên máy chủ sẵn có. Qua bão hoà thứ tự đảo ngược. Bản C++ duy trì 1.8 lần thông lượng của bản Rust trong 20/20 lần lặp đan xen dưới cả hai bộ cấp phát, và dùng nhiều CPU hơn cho mỗi thao tác hoàn thành. Về tài nguyên tiến trình Rust giữ thêm vài MiB bộ nhớ cư trú, lại dưới cả hai bộ cấp phát, và nhỏ hơn một bậc độ lớn so với chi phí của chính lựa chọn bộ cấp phát.

Bốn hạn chế bao quanh tất cả điều này: khối lượng tổng hợp, một phân hệ đơn giản hoá, thiên lệch hiện thực một tác giả, và máy chủ đo ảo hoá. Chương 7 xử lý từng hạn chế chi tiết.

Phạm vi được vẽ có chủ đích để bảo vệ tính hợp lệ nội bộ của phép so sánh ngôn ngữ. Trong phạm vi là chức năng Quản lý Ngữ cảnh UE của Chương 3, thiết kế tương tranh không chia sẻ, phép quét tải đến quá tải, danh mục tiêm lỗi, và nguyên mẫu FFI. Ngoài phạm vi rõ ràng là toàn hệ gNB và các miền mặt điều khiển còn lại, xử lý PHY/L2 và mặt người dùng nặng dữ liệu, trace lưu lượng thật (thay vì khối lượng tổng hợp có căn cứ đặc tả), các lựa chọn ngôn ngữ có quản lý (loại trừ vì lý do độ trễ đuôi ở Chương 2), và triển khai sản xuất cùng mối quan tâm vận hành như nâng cấp, quan sát được và đa thuê bao. Các loại trừ này đổi tính hợp lệ ngoại bộ lấy điều kiện có kiểm soát mà phép so sánh ngôn ngữ công bằng cần, và mỗi loại trừ được xem lại như hạn chế ở Chương 7.

1.5 Ý nghĩa và đối tượng hưởng lợi

Kết quả hướng tới hành động trực tiếp. Với nhà cung cấp RAN và nhà vận hành mạng đám mây, luận văn cung cấp bằng chứng có kiểm soát cho câu hỏi chặn mọi quyết định ngôn ngữ. Câu hỏi đó không phải “Rust có nhanh trung bình không?” mà “Rust có giữ độ trễ đuôi và khả năng dự đoán khi quá tải, và áp dụng tăng dần tốn bao nhiêu?”, được trả lời cho phân hệ đại diện mã mặt điều khiển sản xuất. Với thảo luận tiêu chuẩn và chính sách quanh lộ trình an toàn bộ nhớ [18, 19], công trình neo một mandat chung vào một điểm dữ liệu viễn thông cụ thể. Với cộng đồng nghiên cứu, đặc tả đóng băng, các bản hiện thực đã kiểm thử tương đương, khung benchmark tái lập được và phân loại lớp lỗi tạo nền tái sử dụng cho nghiên cứu ngôn ngữ và hệ thống tiếp theo, và phân rã độ trễ cho nghiên cứu như vậy cách kiểm tra

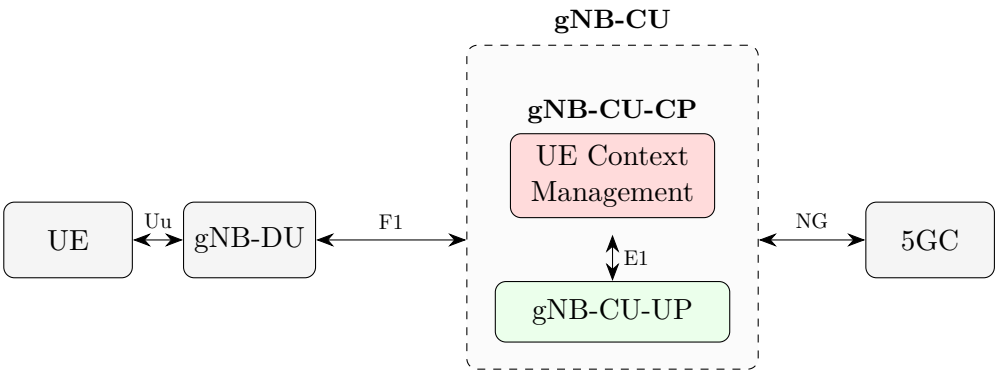
những gì đo lường của chính nó có thể quy kết. Với kỹ sư thực hành, danh mục lỗi và nguyên mẫu di chuyển đồng thời là ví dụ làm việc về những gì đảm bảo của Rust mua và không mua trong miền này.

2 Nền tảng: Các ngôn ngữ hệ thống cho phần mềm nhạy độ trễ

2.1 Hệ thống mặt điều khiển RAN và gNB-CU-CP

Kiến trúc NG-RAN tách gNB thành đơn vị trung tâm (gNB-CU) và các đơn vị phân tán, với mặt điều khiển (gNB-CU-CP) xử lý RRC và quản lý kết nối cho mỗi UE [1, 2, 4]. Hệ thống mặt điều khiển như vậy phải đáp ứng yêu cầu phi chức năng khắt khe: tốc độ sự kiện rất cao (di động UE, cơn bão báo hiệu), giới hạn độ trễ nghiêm ngặt và dự đoán được mà đặc tả dịch vụ và RAN 5G lượng hoá thành KPI [5, 28], phần mềm sống lâu với độ phức tạp liên tục tiến hoá, và khối lượng tương tranh cao. Trong các hệ thống này hành vi trường hợp xấu nhất và đuôi, thay vì trung bình, quyết định liệu hạn chót được đáp ứng, điều thúc đẩy trọng tâm khả năng dự đoán của luận văn này.

Về chức năng, gNB được tách rời theo hai trục độc lập. Trục thứ nhất là tách *trung tâm/phân tán*: đơn vị trung tâm (gNB-CU) lưu các tầng cao hơn của ngăn xếp giao thức, trong khi một hoặc nhiều đơn vị phân tán (gNB-DU) lưu các tầng thấp hơn, nhạy thời gian hơn, gần vô tuyến. Trục thứ hai là tách *mặt điều khiển/mặt người dùng* bên trong đơn vị trung tâm, tách chức năng mặt điều khiển (gNB-CU-CP) khỏi chức năng mặt người dùng (gNB-CU-UP). Các phần này giao tiếp qua giao diện chuẩn hoá: NG về lõi 5G, F1 giữa CU và DU, và E1 giữa CU-CP và CU-UP [1, 2, 29]. Sáng kiến Open-RAN chuẩn hoá thêm các giao diện này để mỗi phần có thể do nhà cung cấp khác cung cấp và chạy như phần mềm trên phần cứng thương mại [3]. Hình 2.1 đặt phân hệ nghiên cứu trong luận văn này, Quản lý Ngữ cảnh UE (UECM), bên trong gNB-CU-CP: nó sở hữu trạng thái mặt điều khiển theo từng UE và điều phối báo hiệu RRC cùng thủ tục di động, nên tính đúng đắn và hành vi thời gian của nó đại diện cho mặt điều khiển nói chung.

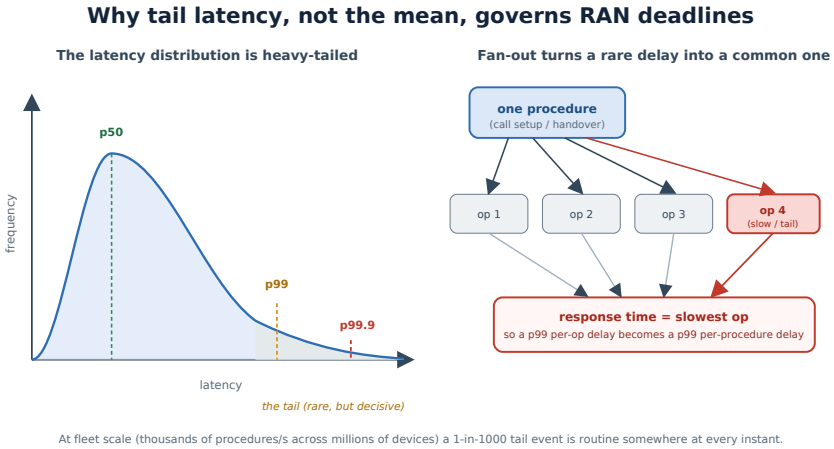


Hình 2.1: Vị trí phân hệ nghiên cứu trong NG-RAN 5G. gNB được tách thành đơn vị phân tán (gNB-DU) và đơn vị trung tâm (gNB-CU), và đơn vị trung tâm lại được tách thành chức năng mặt điều khiển (gNB-CU-CP) và mặt người dùng (gNB-CU-UP), nối qua các giao diện chuẩn F1, E1 và NG. Quản lý Ngữ cảnh UE (được tô) là trình quản lý trạng thái mặt điều khiển theo từng UE bên trong gNB-CU-CP mà luận văn này tái hiện thực và đo lường.

2.2 Độ trễ, thông lượng và khả năng dự đoán trong phần mềm thời gian thực

Hệ thống nhạy độ trễ được đánh giá ít hơn bằng lượng công việc trung bình mà bằng mức độ *dự đoán được* khi thực hiện. Ba chỉ số phải được phân biệt. *Thông lượng* là tốc độ bền vững của thao tác hoàn thành, *độ trễ* là thời gian một thao tác riêng lẻ mất, và *jitter* là độ biến thiên của độ trễ đó. Một hệ thống có thể có độ trễ trung bình xuất sắc và vẫn thường xuyên trễ hạn. Các con số có ý nghĩa do đó là độ trễ *đuôi* phân vị cao, phân vị thứ 99 (p99) và 99.9 (p99.9), thay vì trung bình. Ở quy mô mạng, độ trễ đuôi chi phối hành vi người dùng thấy được: khi một hành động của người dùng lan ra nhiều thao tác nội bộ, thao tác chậm nhất quyết định thời gian phản hồi, nên trễ hiếm trên mỗi thao tác trở thành trễ thường gặp trên mỗi yêu cầu [6]. Phần mềm trung tâm dữ liệu và mạng hiện đại ngày càng vận hành trong chế độ mà độ trễ liên quan đo bằng micro giây, dải mà chi phí hệ điều hành và phần mềm thông thường không còn không đáng kể [7]. Nghiên cứu đo chi tiết cho thấy, ngay cả ở mức sử dụng vừa phải, hiệu ứng phần cứng, hệ điều hành và ứng dụng như nhiễu nền, lập lịch, định tuyến ngắt và trạng thái tiết kiệm năng lượng phồng đuôi xa vượt xa dự đoán của mô hình hàng đợi [30]. Hình 2.2 đặt hai hiệu ứng cạnh nhau: khoảng cách giữa trung vị và các phân vị cao, và hiệu ứng lan truyền biến trễ hiếm trên mỗi thao tác thành trễ thường gặp trên mỗi thủ tục.

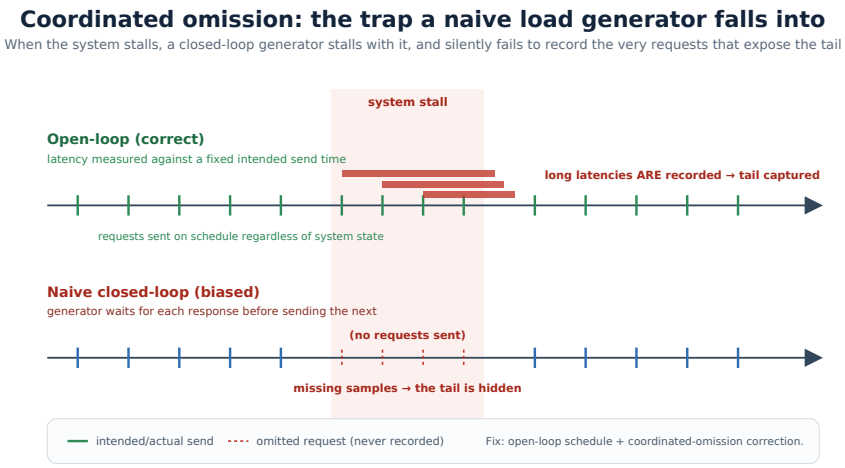
Hệ thống thời gian thực thường được phân cấp theo hệ quả của việc trễ hạn: trong thời gian thực *cứng* một lần trễ là thất bại, trong thời gian thực *firm* kết quả muộn vô dụng nhưng không thảm hoạ, và trong thời gian thực *mềm* một lần trễ chỉ làm giảm chất lượng. Mặt điều khiển RAN nên được xem là thời gian thực mềm đến firm: một phản hồi báo hiệu trễ thỉnh thoảng làm giảm dịch vụ chứ không gây nguy hiểm, nhưng ở quy mô đội tàu và dưới cơn bão báo hiệu các suy giảm đó tích lũy thành vấn đề tin cậy toàn mạng. Điều này càng nặng bởi hành vi hàng đợi: khi tải đưa vào tiến gần công suất phục vụ, trễ hàng đợi tăng mạnh và đuôi độ trễ bùng nổ



Hình 2.2: Vì sao đuôi, không phải trung bình, quyết định hạn chót RAN. Bên trái, hầu hết thao tác kết thúc nhanh và một số ít lâu hơn nhiều, nên p99 và p99.9 nằm xa bên phải trung vị. Bên phải, một thủ tục như thiết lập cuộc gọi chạy nhiều thao tác và chỉ kết thúc khi thao tác chậm nhất xong, nên trễ chạm một thao tác trong một trăm lần chạm tới phần lớn hơn nhiều của các thủ tục. Ở quy mô đội tàu, trễ một trong nghìn lần đang xảy ra ở đâu đó mọi khoảnh khắc.

gần bão hoà. Mô tả hệ thống do đó cần đẩy tải không chỉ ở mức danh nghĩa mà *vào quá tải*, đó là lý do phần đánh giá trong luận văn này quét tải lên và vượt điểm bão hoà.

Đo độ trễ đuôi đúng bản thân nó là cạm bẫy. Một bộ sinh tải vòng kín ngây thơ chờ mỗi phản hồi trước khi phát yêu cầu tiếp theo mắc *coordinated omission*: khi hệ thống đứng, bộ sinh cũng đứng theo và đơn giản không ghi các yêu cầu lẽ ra đã tới trong lúc đứng, che có hệ thống chính xác cái đuôi mà nó phải lộ ra. Hình 2.3 đối chiếu hai mô hình bộ sinh. Phân biệt bên dưới là mô hình hoá, không phải chi tiết hiện thực. Trong hệ kín một yêu cầu mới chỉ kích hoạt khi yêu cầu trước hoàn thành, trong khi hệ mở các lần đến độc lập với hoàn thành, và hai mô hình cho hành vi thời gian phản hồi khác hệ thống trên cùng quy trình phục vụ, với mô hình kín báo cáo con số thấp hơn [31]. Đo lường đúng do đó cần mô hình vòng mở với lịch dự định, hiệu chỉnh coordinated omission, histogram độ trễ độ phân giải cao giữ độ chi tiết qua nhiều bậc độ lớn, và báo cáo thống kê chặt với lần chạy lặp và khoảng tin cậy thay vì ước lượng điểm đơn [32]. Các kỹ thuật này được áp dụng ở Chương 6.



Hình 2.3: Coordinated omission. Bộ sinh vòng mở phía trên gửi theo lịch cố định ngay cả khi hệ thống đứng, và tính thời gian mỗi yêu cầu từ lúc đến hạn, nên yêu cầu bị kẹt được ghi với toàn bộ trễ và đuôi hiện ra. Bộ sinh vòng kín phía dưới chỉ gửi yêu cầu tiếp theo sau khi phản hồi trước tới. Khi hệ thống đứng nó cũng ngừng gửi, nên các yêu cầu chậm nhất không bao giờ được phát và không bao giờ được đo, và đuôi biến mất khỏi dữ liệu.

2.3 C++ cho hệ thống nhạy độ trễ

C++ là ngôn ngữ mặc định cho phần mềm mặt điều khiển RAN vì lý do chính đáng. Nó mang trừu tượng không chi phí, kiểm soát trực tiếp bố cục và cấp phát bộ nhớ, hủy xác định qua idiom Resource Acquisition Is Initialization (RAII), và trình biên dịch, thư viện cùng công cụ phân tích trưởng thành, tất cả không có bộ thu gom rác có thể chèn khoảng dừng khó đoán. C++ hiện đại (C++11 trở đi) thêm con trỏ thông minh (`std::unique_ptr`, `std::shared_ptr`) tự động hoá sở hữu, ngữ nghĩa move tránh sao chép thừa, và mô hình bộ nhớ chính thức với `std::atomic` cho tương tranh có khóa và không khóa. Các tính năng này cho phép viết mã hệ thống nhanh và, với kỷ luật, tương đối an toàn.

Khó khăn là trình biên dịch không thực thi an toàn này. Lập trình viên mới thực thi. RAII và con trỏ thông minh giảm lỗi vòng đời mà không loại bỏ hết. Con trỏ hoặc tham chiếu thô có thể sống lâu hơn đối tượng trỏ tới, biến đổi container có thể vô hiệu iterator, đối tượng có thể bị dùng sau khi đã move, và không gì trong hệ kiểu ngăn đối tượng mutable được chia sẻ và ghi bởi nhiều luồng cùng lúc. Kết quả là danh mục lỗi quen thuộc, use-after-free, double-free, tràn bộ đệm, tranh chấp dữ liệu, deadlock và vô hiệu iterator, dễ đưa vào và tốn kém để tìm ở quy mô viễn thông. Lỗi hỏng bộ nhớ loại này là vấn đề bảo mật dai dẳng hàng thập kỷ, và các giảm thiểu triển khai vẫn chỉ là phòng thủ một phần [33]. Mức độ phổ biến được ghi nhận rõ trong công nghiệp. Vấn đề an toàn bộ nhớ chiếm khoảng 70 % lỗ hổng bảo mật Microsoft báo cáo [8], và Google báo cáo giảm mạnh lỗ hổng loại này khi ngôn ngữ an toàn bộ nhớ được áp dụng trên Android, xuống dưới một phần năm tổng lỗ hổng nền tảng đó vào năm 2025 [9, 15]. Danh mục cụ thể này được phát triển chi tiết ở Chương 3 và là cơ sở cho nghiên cứu tiêm lỗi ở Chương 6.

Thực hành C++ giảm thiểu các lỗi này bằng công cụ thay vì đảm bảo ngôn ngữ, và công cụ vốn dĩ chỉ một phần. Bộ dò lỗi động như AddressSanitizer (ASan), ThreadSanitizer (TSan) và UndefinedBehaviorSanitizer phát hiện nhiều lỗi, nhưng chỉ trên đường mã mà kiểm thử cụ thể thực sự chạy với đầu vào đúng, và chúng đặt chi phí chạy và bộ nhớ khiến chúng không phù hợp sản xuất. TSan đặc biệt chỉ báo cáo tranh chấp dữ liệu xảy ra trong một lần chạy, nên tranh chấp chỉ lộ dưới xen kẽ hiếm có thể lọt. Bộ phát hiện deadlock của nó thừa hưởng giới hạn đó từ kỹ thuật nó hiện thực, suy luận tiềm năng deadlock từ đồ thị khóa của một trace đơn [34], và nó được phát hành như thành phần thử nghiệm chỉ bao họ mutex POSIX [35]. Phân tích tĩnh giúp nhưng mắc false negative và không lý giải toàn chương trình. Phát hiện do đó là nỗ lực tốt nhất và sau sự kiện, đúng tình huống mà ngôn ngữ có đảm bảo lúc biên dịch nhằm thay đổi.

Điều đó mô tả C++ như có thể triển khai hôm nay, và ngôn ngữ không đứng yên. Từ năm 2023 ủy ban tiêu chuẩn theo đuổi *profiles*: tập quy tắc đặt tên bao phủ an toàn kiểu, biên, vòng đời và số học mà triển khai tuân thủ phải thực thi ở mọi nơi được bật [36]. Hướng đó được chọn thay vì *Safe C++*, đề xuất tập con an toàn với kiểm tra mượn và thư viện chuẩn riêng, tức mô hình Rust diễn đạt trong C++ [37]. Phép so sánh ở đây vẫn giữ hiệu lực. Profiles chưa được chuẩn hoá, và tháng 1 năm 2026 chính tác giả vẫn kêu gọi hiện thực và coi thiết kế các profile ban đầu là việc còn phải làm [38]. Profile tương tranh, cái liên quan các lớp lỗi được liệt kê ở Chương 3, được đặt sau tập ban đầu và không nằm trong đó: “cần định nghĩa và hiện thực thêm profile, đặc biệt một profile tương tranh” [38]. Quy tắc chỉ thực thi khi bật cũng là thứ khác với thuộc tính hệ kiểu mang vô điều kiện, nhất là khi thực thi là kiểm tra chạy chứ không phải từ chối biên dịch. Các bản hiện thực so sánh ở đây do đó được xây theo C++23 như đã đặc tả và phát hành, và Mục 7.6 ghi điều đó ghim phép so sánh vào đâu.

2.4 Mô hình sở hữu và an toàn của Rust

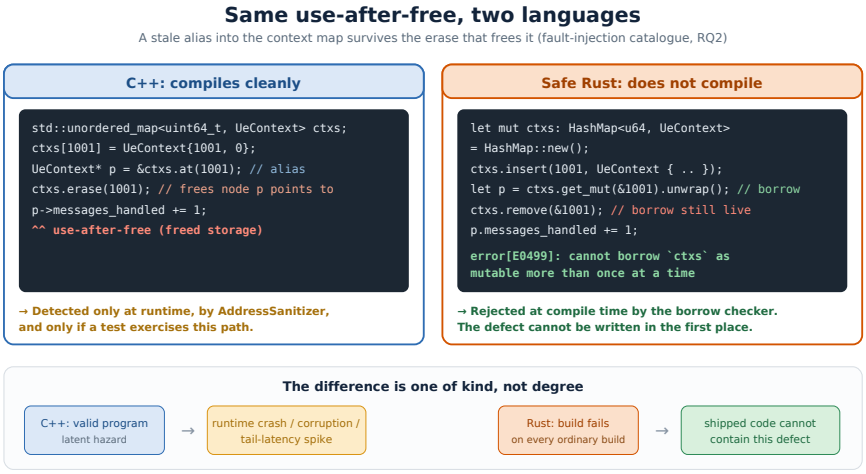
Rust thực thi an toàn bộ nhớ và không tranh chấp dữ liệu lúc biên dịch qua mô hình sở hữu và mượn [10], và đi kèm công cụ hiện đại trong Cargo cùng kiểm thử tích hợp. Hệ kiểu cấm tĩnh biến đổi trạng thái chia sẻ và giam các ngoại lệ không tránh khỏi vào mã `unsafe` giữ sau API an toàn [39].

Các đảm bảo này dựa trên hơn khẳng định. Tính đúng đắn của mảnh an toàn của ngôn ngữ, gồm vai trò `unsafe` như lối thoát được đánh dấu rõ, đã được thiết lập chính thức bởi dự án RustBelt [11], và kỷ luật alias mà ngay cả mã `unsafe` phải tuân theo đã được cho ngữ nghĩa vận hành chính xác [40]. Bằng chứng thực nghiệm ủng hộ mô hình trong thực tế: trong mã nguồn thật `unsafe` được dùng tiết kiệm và cục bộ [27, 41], và nghiên cứu mọi CVE an toàn bộ nhớ Rust tìm thấy mỗi CVE trừ một lỗi trình biên dịch đều liên quan mã `unsafe` [42]. Lưu ý là không an toàn lan qua phụ thuộc. Ít hơn một phần ba thư viện dùng `unsafe` trực tiếp, nhưng phần lớn hơn nhiều không thể kiểm tra tĩnh đầy đủ vì nó ẩn đâu đó trong chuỗi gọi [43], đó là lập luận thêm để giữ lõi an toàn quan trọng của phân hệ không cần `unsafe`, như luận

văn này làm.

Một thuộc tính quan trọng nhất trong bối cảnh nhạy độ trễ. An toàn này đến từ hệ kiểu và bộ kiểm tra mượn, không từ công cụ chạy, và được dự định là *không chi phí*. Nghiên cứu hệ thống lập luận Rust đạt an toàn không chi phí thu gom rác hay kiểm tra chạy, và còn cho phép kỹ thuật hiệu quả khó trong C [26]. Liệu tuyên bố không chi phí có đúng với khối lượng mặt điều khiển viễn thông chính là thuộc tính mà luận văn này đánh giá.

Cụ thể, bộ kiểm tra mượn từ chối lúc biên dịch chính use-after-free phác thảo ở phần giới thiệu: chương trình giải phóng ngữ cảnh trong khi tham chiếu tới nó vẫn còn sống thì không build. Hình 2.4 đặt lỗi đó ở dạng Quản lý Ngữ cảnh UE cạnh bản C++, biên dịch và chỉ bị bắt, nếu có, lúc chạy.



Hình 2.4: Cùng lỗi use-after-free trong C++ và Rust an toàn. Alias cũ vào bản đồ ngữ cảnh sống sót sau `erase`/`remove` giải phóng nó: trong C++ mã này biên dịch và chỉ bị bắt lúc chạy (và chỉ dưới AddressSanitizer, trên đường mà kiểm thử chạy), trong khi Rust an toàn bộ kiểm tra mượn từ chối lúc biên dịch (`E0499`), nên lỗi không thể viết ra. Khác biệt là về loại, không phải mức độ.

Cùng kỷ luật mở rộng sang tương tranh. Rust mã hoá an toàn luồng trong hệ kiểu qua marker trait `Send` và `Sync`: giá trị chỉ qua ranh giới luồng nếu kiểu là `Send`, và chỉ chia sẻ bằng tham chiếu qua luồng nếu là `Sync`. Chia sẻ giá trị cho phép biến đổi không đồng bộ do đó là lỗi biên dịch, không phải tranh chấp tiềm ẩn. Đây là thuộc tính Rust quảng bá là "fearless concurrency." Xử lý lỗi cũng tường minh: thao tác có thể thất bại trả về giá trị `Result` mà người gọi phải xử lý, thay vì ném ngoại lệ làm unwind qua luồng điều khiển ẩn. Cả chi phí lẫn luồng điều khiển của đường thất bại do đó vẫn nhìn thấy được, đó là lợi thế trong mã nhạy độ trễ nơi bất ngờ ở đuôi mới quan trọng.

Rust không phải ngôn ngữ an toàn bộ nhớ duy nhất, và lý do phép so sánh ở đây vẽ với C++ thay vì với runtime có quản lý thuộc cùng mô hình này. Java, C# và Go đạt an toàn bộ nhớ qua runtime và thu gom rác, đánh đổi xuất sắc cho nhiều phần mềm và kém cho mặt điều khiển nhạy độ trễ đuôi: khoảng dừng thu gom là nguồn độ trễ đuôi được công nhận trong hệ thống lớn [6], và runtime có quản lý từ bỏ kiểm

soát bố cục bộ nhớ và lập lịch mà miền này dựa vào. C, ở cực kia, không an toàn bộ nhớ như C++ trong khi ít trừu tượng hơn để xây hệ thống lớn. Rust là ngôn ngữ được hỗ trợ rộng đầu tiên kết hợp an toàn bộ nhớ và luồng lúc biên dịch với không có bộ thu gom rác và thông lượng cùng bộ nhớ cạnh C và C++ [26, 39, 44]. Đó là lý do ngôn ngữ có quản lý nằm ngoài phạm vi và lý do phép so sánh có ý nghĩa cho phần mềm mặt điều khiển RAN là C++ với Rust.

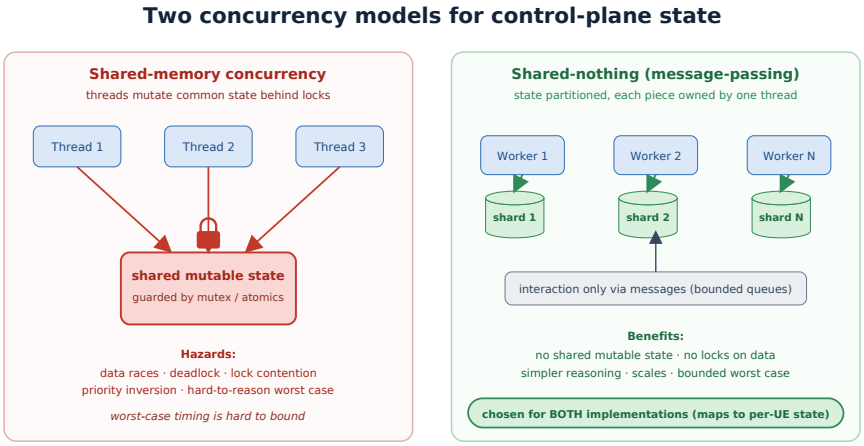
Bảng 2.1 đặt cơ chế hai ngôn ngữ cạnh nhau. Phân biệt nó xây quanh không phải mối nguy mỗi ngôn ngữ gọi tên gì, vì cả hai gọi tên gần giống nhau, mà *khi nào* và *bởi cái gì* mỗi thứ bị loại trừ, và hai hàng cuối đánh dấu nơi câu trả lời không còn là "bởi trình biên dịch" với cả hai.

Bảng 2.1: Cách mỗi ngôn ngữ thực thi các thuộc tính mà luận văn này đo. Các hàng trên quy tắc là nơi hai ngôn ngữ khác về loại chứ không chỉ mức độ: trình biên dịch từ chối chương trình loại trừ lỗi khỏi mọi lần thực thi, trong khi bộ dò lỗi chỉ loại trừ khỏi các lần thực thi mà kiểm thử có công cụ tình cờ chạy. Các hàng dưới quy tắc là ranh giới phát triển ở Chương 3 và đo ở Chương 6, nơi không ngôn ngữ nào có đảm bảo tĩnh.

Khía cạnh	C++ hiện đại	Rust
Vòng đời đối tượng	RAII và con trỏ thông minh, áp dụng theo quy ước. Con trỏ hoặc tham chiếu thô có thể sống lâu hơn đối tượng trỏ tới	Sở hữu: mỗi giá trị có một chủ sở hữu và bị giải phóng đúng một lần
Alias	Không hạn chế: bất kỳ số alias mutable nào cũng có thể cùng tồn tại	Mượn: hoặc một tham chiếu mutable hoặc nhiều tham chiếu chia sẻ, không bao giờ cả hai
Biên	Không kiểm tra trên `operator[]`, chỉ kiểm tra qua `at()`	Kiểm tra mặc định, bỏ qua nơi chỉ số chứng minh được trong phạm vi
An toàn luồng	Không diễn đạt trong hệ kiểu: chia sẻ đối tượng mutable qua luồng vẫn biên dịch	Marker trait `Send` và `Sync`. Chia sẻ không đồng bộ là lỗi kiểu
Báo hiệu lỗi	Ngoại lệ hoặc mã lỗi: đường thất bại có thể vô hình tại điểm gọi	Giá trị `Result` người gọi phải xử lý. Đường thất bại tường minh
Lối thoát	Toàn bộ ngôn ngữ	Khối `unsafe`, có giới hạn và đếm được cơ học
Giai đoạn phát hiện	Chạy, dưới bộ dò lỗi đúng, trên đường kiểm thử chạy	Biên dịch, trên mọi build thường, bất kể độ phủ kiểm thử
Thứ tự khóa	Không kiểm tra	Không kiểm tra
Tính nguyên tử của thao tác ghép	Không kiểm tra	Không kiểm tra

2.5 Mô hình tương tranh cho phần mềm mặt điều khiển

Cách cấu trúc tương tranh định hình cả tính đúng đắn lẫn độ trễ đuôi. Mô hình chi phối là tương tranh *bộ nhớ chia sẻ*, trong đó luồng biến đổi trạng thái chung được bảo vệ bởi mutex và atomic. Nó linh hoạt, nhưng là nguồn tranh chấp, deadlock, tranh chấp khóa và đảo ngược ưu tiên, và trường hợp xấu nhất khó giới hạn. Phương án thay thế là tương tranh *truyền thông* hoặc *không chia sẻ*, trong đó trạng thái được phân vùng để mỗi phần do một luồng sở hữu và mọi tương tác qua thông điệp, như mô hình actor. Nó đổi linh hoạt lấy khả năng mở rộng và lý luận đơn giản hơn nhiều, và ánh xạ tự nhiên lên phân vùng theo UE, vì trạng thái mỗi UE phần lớn độc lập. Lựa chọn cũng dịch chuyển đại lượng đo ở đây: cấu trúc một luồng mỗi lõi tự thay đổi độ trễ đuôi ứng dụng [45], nên mô hình được giữ cố định giữa hai bản hiện thực thay vì giao cho idiom mỗi ngôn ngữ. Hình 2.5 đối chiếu hai mô hình.



Hình 2.5: Hai mô hình tương tranh cho trạng thái mặt điều khiển. Tương tranh bộ nhớ chia sẻ (trái) cho luồng biến đổi trạng thái chung sau khóa và là nguồn tranh chấp dữ liệu, deadlock và tranh chấp khóa, với trường hợp xấu nhất khó giới hạn. Truyền thông không chia sẻ (phải) phân vùng trạng thái để mỗi shard do một worker sở hữu và mọi tương tác qua hàng đợi có giới hạn, loại khóa trên dữ liệu ngữ cảnh và ánh xạ lên phân vùng theo UE. Đây là mô hình hai bản hiện thực áp dụng.

Cấu trúc thứ ba, runtime bất đồng bộ trên bộ lập lịch work-stealing, là idiom trong cả hai ngôn ngữ và không được dùng trong cả hai. Hành vi thời gian của nó thuộc runtime, không thuộc ngôn ngữ chứa nó: nghiên cứu thời gian thực về Rust bất đồng bộ tìm một ứng dụng đáp ứng hoặc trễ hạn tùy runtime và ánh xạ ưu tiên luồng, và cấu hình đáp ứng hạn không phải cấu hình work-stealing thông thường [46]. Áp dụng runtime như vậy sẽ nhập biến thiên mà nghiên cứu này tồn tại để loại trừ, và Mục 7.5 quay lại phép so sánh mà nó không bị loại trừ.

Hàng đợi giữa nhà sản xuất và người tiêu thụ là lựa chọn thứ hai, độc lập. Hàng đợi không giới hạn che quá tải và để độ trễ cùng bộ nhớ tăng không giới hạn. Hàng đợi *có giới hạn*, dù chặn để backpressure hay loại bỏ, giới hạn bộ nhớ trường hợp xấu nhất và làm quá tải nhìn thấy được, điều độ trễ đuôi dự đoán được đòi hỏi. Cả hai

lựa chọn cố định kiến trúc mà hai bản hiện thực dùng ở Chương 5: thiết kế không chia sẻ, một luồng mỗi shard, trong đó trạng thái UE chia thành các lát độc lập gọi là *shard*, mỗi lát do đúng một luồng worker sở hữu, nên không hai luồng chạm cùng ngữ cảnh. Yêu cầu tới shard qua hàng đợi đa nhà sản xuất, một người tiêu thụ có giới hạn chặn khi đầy.

2.6 Khả năng tương tác giữa C++ và Rust

Áp dụng ngôn ngữ mới trong hệ thống lớn sẵn có buộc phải tăng dần: thành phần Rust phải cùng tồn tại và được gọi từ mã nguồn C++ đáng kể. Mẫu số chung cho việc đó là giao diện nhị phân ứng dụng C (ABI). Rust có thể lộ hàm với liên kết `extern "C"` và đưa ra *handle mờ* tới đối tượng do Rust sở hữu, mà C++ gọi qua header C. Công cụ tự động hoá một phần: `bindgen` (binding Rust từ header C), `cbindgen` (header C từ Rust), và crate `cxx` (cầu hai chiều an toàn hơn). Tương tác không miễn phí. Vượt ranh giới bước ra ngoài ngôn ngữ được kiểm tra, nên đường ghép phải chứa mã `unsafe`, giá trị phức tạp hơn đơn giản nhất cần marshalling, và giao diện ngoại phải được thiết kế có chủ đích. Kích thước đường ghép, `unsafe` nó tập trung, và chi phí vượt ranh giới mỗi lần gọi là đại lượng RQ4 đo ở Chương 6.

Một phân biệt giới hạn những gì các đo lường đó có thể thiết lập. Tập trung `unsafe` ở đường ghép nói trình biên dịch kiểm tra gì, không nói gì có thể xảy ra lúc chạy. Thành phần Rust liên kết vào host C++ chia một không gian địa chỉ với nó, nên lỗi phía C++ có thể hỏng dữ liệu do Rust sở hữu dù bao nhiêu phía Rust bộ kiểm tra mượn đã xác minh. Điều này đã được chứng minh thực nghiệm, và biện pháp đề xuất cô lập hai heap trong phần cứng hoặc runtime thay vì tin kỷ luật giao diện [47, 48]. Các lớp lỗi đặc thù ranh giới Rust/C được liệt kê tách khỏi lỗi của từng ngôn ngữ [49], và chính bộ sinh binding cũng mang lỗi [50]. Hệ quả được nêu một lần ở đây và được tôn trọng xuyên suốt: lõi không cần `unsafe` có nghĩa lỗi của chính lõi bị loại lúc biên dịch, và không có nghĩa lõi được bảo vệ khỏi host.

Điều gần đây thay đổi là tầm với của công cụ, không phải đảm bảo. Bộ kiểm tra mô hình không trạng thái cho Rust giờ khám phá không gian trạng thái chương trình cùng phụ thuộc ngoại, đưa tranh chấp dữ liệu phát sinh từ tương tác hai ngôn ngữ vào tầm một bộ kiểm tra dù chúng nằm ngoài những gì bộ kiểm tra mượn thấy [51]. Khả năng đó tới sau khi nguyên mẫu đo ở đây được xây, và Chương 8 ghi nó là bước tiếp theo cho đường ghép.

2.7 Khoảng trống nghiên cứu

Các mục trước dựng khoảng trống. Mặt điều khiển RAN là miền thời gian thực mềm đến firm, tương tranh cao, nơi hành vi đuôi khi quá tải, không phải thông lượng trung bình, quyết định chất lượng mạng giao. C++ cung cấp kiểm soát và hiệu năng miền đòi hỏi, nhưng giao cả lớp lỗi bộ nhớ và tương tranh cho phát hiện nỗ lực tốt nhất sau sự kiện. Rust hứa ngăn các lớp đó lúc biên dịch không tốn chi phí chạy, và

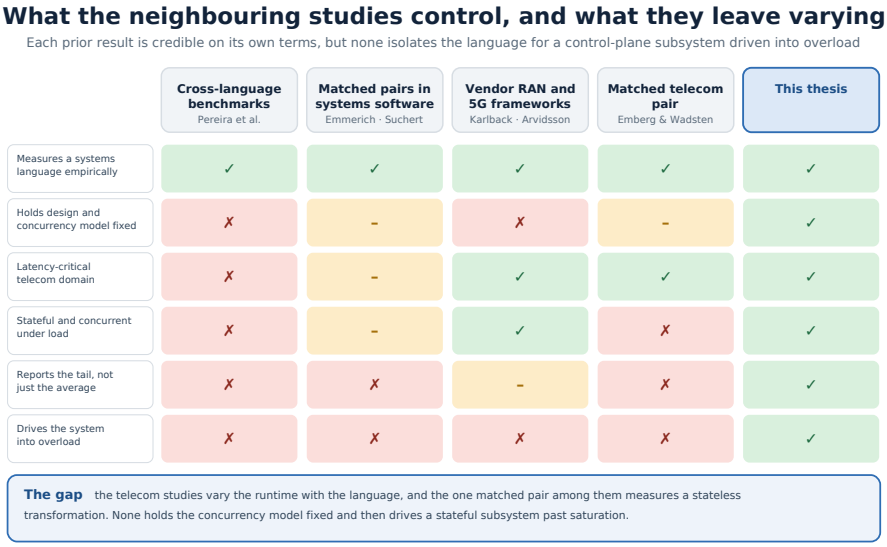
đưa ra lộ trình áp dụng tăng dần đáng tin qua ABI C.

Miền không phải chưa được nghiên cứu, và khối lượng công trước dễ bị đánh giá thấp vì gần như không có trong chỉ mục. Ít nhất năm luận văn thạc sĩ do nhà cung cấp lưu trữ, tất cả tại Ericsson, đã so sánh ngôn ngữ ứng viên cho phần mềm RAN và viễn thông. Một cân chi phí chạy với hiệu quả phát triển giữa C++, Rust và Go [52]. Một hỏi Rust có thể thay C trong quản lý nút RAN [53]. Một xây bộ xử lý gói một luồng mỗi lõi trên Rust bất đồng bộ và đo với ứng dụng DPDK nó sao chép [54]. Một đo trung vị và độ trễ phân vị thứ 99 cho thủ tục mặt điều khiển qua năm framework lập trình bất đồng bộ [55]. Cuối cùng xây bản C++ và Rust khớp của trình dịch ASN.1 và so sánh thống kê [56]. Cả năm đều liên quan trực tiếp, và hai báo cáo khác biệt bất lợi cho Rust.

Ba trong số đó thay đổi ngôn ngữ cùng runtime và mô hình tương tranh, nên khác biệt đo được không thể quy cho ngôn ngữ thay vì framework xây trên nó [52, 54, 55]. Cỡ hiệu ứng chạm hệ số năm, đó là dấu hiệu phần lớn đo lường thuộc runtime. Hai còn lại giữ thiết kế gần cố định hơn nhưng báo cáo thời gian thực thi và bộ nhỉ đỉnh thay vì hành vi đuôi dưới tải tăng [53, 56], và gần nhất, Emberg và Wadsten, đo biến đổi không trạng thái thay vì phân hệ có trạng thái, tương tranh. Hình 2.6 đặt các nghiên cứu lân cận so với các thuộc tính câu trả lời có kiểm soát cần, và mẫu nó cho thấy mỗi nghiên cứu giữ một số và không nghiên cứu nào giữ hết.

Sự tập trung bằng chứng đó cũng là một phần khoảng trống. Cả năm nghiên cứu đến từ một nhà cung cấp, và kết luận của chúng chưa bao giờ được thử với phần mềm, mục tiêu hay quy ước của tổ chức khác, nên điều biết về câu hỏi này là những gì framework của một công ty báo cáo về nó. Hồ sơ công bố phía kia trống hơn: không có nghiên cứu Rust trong các ấn phẩm được lập chỉ mục của nhà cung cấp chủ trì công trình này, và lập trường bạn đồng nghiệm gần nhất từ phía đó lập luận cho việc làm cho sanitisation lỗi bộ nhớ thực tế thay vì thay nó bằng đảm bảo lúc biên dịch [57], đó là chiến lược luận văn này đối chứng. Mục 7.5 so sánh năm nghiên cứu với nghiên cứu này trực tiếp.

Vì vậy điều còn thiếu trong tài liệu là bằng chứng *trực tiếp, có kiểm soát* cho phân hệ mặt điều khiển đại diện: liệu Rust có thực sự giữ độ trễ đuôi và khả năng dự đoán khi quá tải so với C++ hiện đại *khi mô hình tương tranh, thuật toán và cấu trúc dữ liệu được giữ cố định*, chính xác lớp lỗi nào nó loại bỏ và giai đoạn nào phần còn lại bị bắt, mô hình an toàn tốn bao nhiêu về thông lượng và tài nguyên, và áp dụng tăng dần tốn bao nhiêu trong thực tế. Lấp khoảng trống này là mục đích của bốn câu hỏi nghiên cứu. Chương tiếp theo neo chúng vào phân hệ cụ thể, lớp lỗi và thuộc tính thời gian liên quan, và Chương 4 nêu phương pháp luận để theo đuổi chúng.



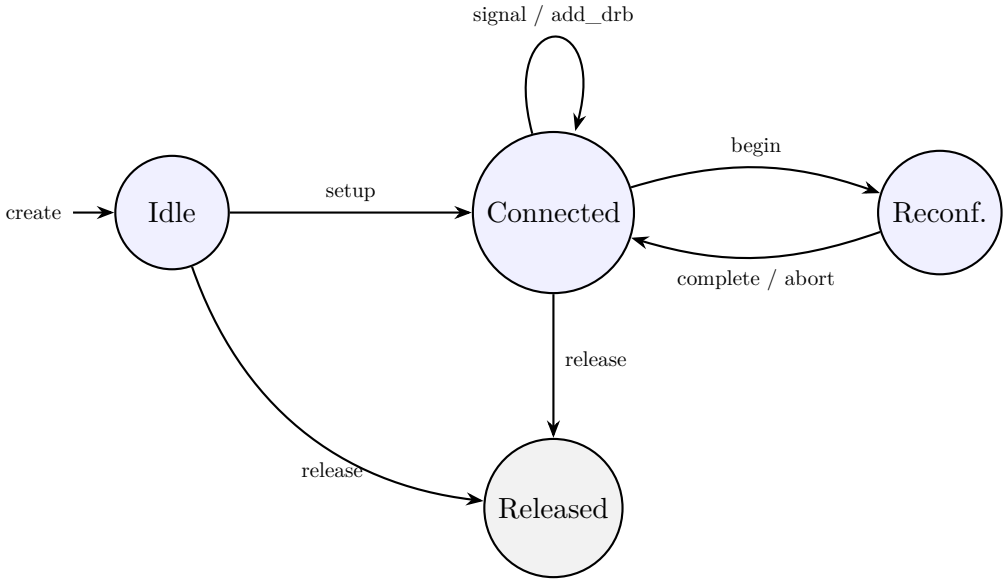
Hình 2.6: Điều các nghiên cứu lân cận kiểm soát và để thay đổi. Mỗi hàng là thuộc tính mà câu trả lời có kiểm soát cho câu hỏi nghiên cứu cần. Mỗi cột là một nghiên cứu. Các so sánh chéo ngôn ngữ đo đúng loại đối tượng nhưng sai chế độ, báo cáo trung bình trên khối lượng đa mục đích. Cặp khớp trong phần mềm hệ thống giữ thiết kế cố định, nhưng ngoài miền này và dưới bão hoà. Các nghiên cứu framework RAN và 5G của nhà cung cấp chạm đúng miền, và một trong số đó báo cáo đuôi, nhưng trong tất cả ngôn ngữ thay đổi cùng runtime và mô hình tương tranh. Cặp viễn thông khớp duy nhất, Emberg và Wadsten, kiểm soát thiết kế nhưng đo biến đổi không trạng thái. Giữ thiết kế và mô hình tương tranh cố định cho phân hệ có trạng thái, tương tranh rồi đẩy qua bão hoà là điều luận văn này thêm vào. Đóng góp là kiểm soát, không phải kỹ thuật đo mới.

3 Bối cảnh bài toán: Quản lý Ngữ cảnh UE và tính dự đoán được

3.1 Phân hệ Quản lý Ngữ cảnh UE

Phân hệ được nghiên cứu là một dịch vụ Quản lý Ngữ cảnh UE. Dịch vụ này được mô hình hoá theo một miền dịch vụ gNB-CU-CP thực tế của Nokia, sở hữu vòng đời ngữ cảnh UE, thực thi thủ tục RRC [4], và di động/bàn giao, tức các chức năng mà kiến trúc 3GPP giao cho phần mặt điều khiển của một gNB phân tách (Mục 2.1).

Hình 3.1 cho thấy máy trạng thái vòng đời ngữ cảnh UE mà cả hai bản hiện thực mã hoá giống hệt nhau. Máy trạng thái này chính là thứ mà khối lượng công việc hỗn hợp ở Mục 6.2 đưa mỗi UE đi qua.



Hình 3.1: Máy trạng thái vòng đời ngữ cảnh UE. `create` cấp phát một ngữ cảnh ở `Idle` và `setup` kết nối nó. `signal` và `add_drb` được xử lý tại chỗ khi ở `Connected`. Một tái cấu hình hoặc bàn giao đi qua `Reconfiguring` rồi trở lại khi `complete` hoặc `abort`, còn `release` tháo dỡ ngữ cảnh. Thao tác không hợp lệ với trạng thái hiện tại trả về `OpError` thay vì đổi trạng thái.

Vì sao chọn phân hệ này. UECM phù hợp làm đối tượng nghiên cứu vì vừa đại diện vừa đủ nhỏ để kiểm soát. Đây là chức năng lớp 3 RAN cốt lõi, và mang các thuộc tính khiến mã mặt điều khiển sản xuất khó xử lý: trạng thái có thể thay đổi theo từng UE, vòng đời thao tác chặt chẽ, mức độ đồng thời cao giữa các UE, cùng mục tiêu độ trễ và độ tin cậy mà hệ thống 5G đặt ra cho thủ tục mặt điều khiển [5, 28]. Kết luận về cách một ngôn ngữ xử lý các thuộc tính đó do đó chuyển được sang các dịch vụ mặt điều khiển có trạng thái và đồng thời khác. Phân hệ cũng đủ nhỏ để đặc tả chính xác, hiện thực trung thực bằng hai ngôn ngữ, và suy luận trọn vẹn,

tức điều kiện tiên quyết để so sánh có kiểm thử tương đương trở nên khả thi. Một phân hệ lớn hơn sẽ đổi lấy tính ngoại hiệu bằng tính nội hiệu mà một so sánh ngôn ngữ công bằng phụ thuộc vào. UECM nằm ở điểm cả hai vẫn trong tầm với.

Tập thao tác. Giao diện mà nghiên cứu khai thác cố ý nhỏ và theo hướng vòng đời. Nó có thể tạo và giải phóng một ngữ cảnh, thiết lập kết nối (`Idle`→`Connected`), xử lý một thông điệp báo hiệu trong phiên, thêm một data radio bearer, bắt đầu, hoàn tất hoặc huỷ một tái cấu hình hoặc bàn giao, và tra cứu trạng thái ngữ cảnh dưới tải đồng thời. Mỗi thao tác được định nghĩa cho mọi trạng thái của máy ở Hình 3.1: thao tác bất hợp pháp ở trạng thái hiện tại trả về lỗi có kiểu và giữ nguyên ngữ cảnh, thay vì ném ngoại lệ hoặc làm hỏng trạng thái. Tập thao tác kín và có kiểu này là điều cho phép cùng một bộ vectơ kiểm thử và cùng khối lượng công việc dẫn cả hai bản hiện thực theo các đường đi giống hệt nhau (Mục 5.6).

3.2 Các lớp lỗi an toàn bộ nhớ và đồng thời

Các lỗi ảnh hưởng tới mã mặt điều khiển C++ loại này chia thành hai họ. Họ *an toàn bộ nhớ* liên quan tới vòng đời và biên của đối tượng: *use-after-free* truy cập vùng nhớ đã được giải phóng, *double-free* giải phóng cùng một vùng hai lần, và *buffer overflow* đọc hoặc ghi vượt cuối vùng cấp phát. Họ *đồng thời* liên quan tới truy cập không đồng bộ hoặc sai thứ tự từ nhiều luồng: *data race* là một cặp truy cập không đồng bộ vào cùng vị trí mà ít nhất một bên là ghi, *deadlock* là chờ khoá vòng, còn *iterator invalidation* là dùng iterator hoặc tham chiếu vào container sau khi một thay đổi đã di chuyển hoặc giải phóng vùng nhớ bên dưới. Điểm chung là trong C++ chúng đều *im lặng*: trình biên dịch chấp nhận từng lớp, và trên các đường đi mà kiểm thử thông thường tình cờ chạm tới, chúng có thể trông như hoạt động đúng, chỉ lộ ra sau dưới dạng hỏng dữ liệu, sập chương trình, hoặc đột biến độ trễ không dự đoán được. Đây là cùng các lớp hỏng bộ nhớ mà hàng thập kỷ nghiên cứu bảo mật vẫn khó kìm hãm trong C và C++ [33].

Mỗi lớp có một hiện thân tự nhiên trong Quản lý Ngữ cảnh UE, đó là lý do phân hệ phù hợp để nghiên cứu chúng:

- **Use-after-free.** Một thông điệp báo hiệu giữ con trỏ thô hoặc tham chiếu tới ngữ cảnh UE mà một `release` đồng thời đã xoá khỏi bản đồ ngữ cảnh. Bộ xử lý đang bay khi đó giải tham chiếu vùng nhớ đã giải phóng.
- **Double-free.** Đường `release` chạy hai lần cho cùng một UE, ví dụ vì `release` được truyền lại không được làm bất biến, giải phóng tài nguyên sở hữu của ngữ cảnh lần thứ hai.
- **Iterator invalidation.** Mã lặp danh sách data radio bearer của ngữ cảnh trong khi thêm hoặc gỡ bearer, nên iterator trỏ vào vùng nhớ mà container đã tái cấp phát.

- **Data race.** Hai worker cập nhật cùng một trường ngữ cảnh, bộ đếm hoạt động hoặc cờ trạng thái, mà không đồng bộ hoá, nên các truy cập tranh chấp.
- **Unguarded shared mutation.** Một ngữ cảnh được chia sẻ bằng tham chiếu giữa các luồng và bị thay đổi mà không có khoá hay kỷ luật khác, một race mà nguyên nhân gốc là không có quy tắc sở hữu cho trạng thái dùng chung.
- **Use-after-move.** Thủ tục nhận di chuyển ngữ cảnh vừa dựng vào kho và rồi trả lời yêu cầu từ biến cục bộ vừa bị move khỏi, nên phản hồi mang danh tính rỗng. Lớp này được giữ vì công cụ C++ *không* nói gì về nó. Không có gì bị hỏng và không có hành vi không xác định, vì đối tượng đã move vẫn ở trạng thái hợp lệ nhưng không xác định, nên lỗi không có triệu chứng lúc chạy để sanitiser bắt.
- **Thread-unsafe handle shared across workers.** Tay cầm ngữ cảnh đếm tham chiếu mà bộ đếm không nguyên tử, thói quen đường nóng chọn khi khối điều khiển nguyên tử của con trỏ chia sẻ tốn hơn lợi ích, được truyền cho hai shard worker. Ràng buộc khiến tay cầm đúng được viết trong comment, vì C++ không có cách diễn đạt gắn kết luồng trong kiểu.

Bảy lớp này nằm gọn trong các bảo đảm mà Rust tuyên bố, nên chúng là thử nghiệm tự nhiên xem tuyên bố đó có đứng không. Tuy nhiên chỉ kiểm thử chúng sẽ khắc sẵn câu trả lời mong đợi vào lựa chọn ca kiểm thử, nên thêm bảy lớp nữa được đưa vào nghiên cứu vì lý do ngược lại, tức có cơ sở kỳ vọng Rust an toàn *không* ngăn được chúng:

- **Lock-order deadlock.** Một bàn giao khoá ngữ cảnh UE nguồn rồi đích, trong khi bàn giao đồng thời chiều ngược khoá đích rồi nguồn, và hai bên chờ nhau vô hạn. Không có gì vi phạm vòng đời đối tượng hay aliasing. Lỗi nằm ở thứ tự lấy khoá.
- **Lost update.** Hai worker tăng bộ đếm hoạt động của ngữ cảnh qua read-modify-write tách thành hai vùng tới hạn riêng. Mỗi truy cập riêng lẻ đều đồng bộ đúng, nên đây là *race condition* chứ không phải *data race*: vi phạm nguyên tử nằm ở tổ hợp thao tác, không phải ở bản thân truy cập bộ nhớ, và nó làm hỏng giá trị im lặng thay vì sập chương trình.
- **Leak through a reference cycle.** Một ngữ cảnh và đối tác bàn giao giữ tham chiếu sở hữu lẫn nhau, nên không bộ đếm nào về không và không bên nào bị huỷ. Đếm tham chiếu không thu gom chu trình trong cả hai ngôn ngữ, và rò rỉ là an toàn trong Rust theo thiết kế tường minh. Đây là phát biểu ranh giới rõ nhất trong danh mục, vì cả hai ngôn ngữ thất bại giống nhau.
- **Integer overflow in a release build.** Tính bước TEID của bearer vượt độ rộng kiểu. Trong C++ tràn số có dấu là *không xác định*, nên chương

trình không có nghĩa. Trong Rust nó *được định nghĩa* là quấn vòng, vì hồ sơ `opt-level = 2` mà nghiên cứu so sánh tắt `overflow-checks`, nên chương trình có nghĩa và nghĩa đó sai. Mẫu không giả định: CVE-2026-64467 trong `rust_binder` là lỗi bước cùng hình dạng.

- **Worker failure blast radius.** Một shard worker thất bại giữa chừng thủ tục. Mô hình đồng thời ở Mục 5.2 biến một shard thành miền lỗi theo cấu trúc, và không hệ kiểu nào của hai ngôn ngữ buộc nó hành xử như vậy: trong C++ ngoại lệ thoát khỏi callable của luồng chấm dứt cả tiến trình, còn trong Rust panic giam trong luồng nhưng độc trạng thái nó giữ, nên thiệt hại lan theo dữ liệu. Trực tiếp liên quan tới luận điểm tính sẵn sàng cấp nhà mạng, và không gì trong hai hệ kiểu giải quyết.

- **Double-free across the seam.** Máy chủ C++ giữ tay cầm mờ tới ngữ cảnh Rust và giải phóng hai lần, tức `release` truyền lại của lớp double-free ở trên, giờ diễn đạt theo cách ranh giới hàm ngoại bắt buộc. Rust an toàn từ chối giải phóng kép vì sở hữu theo dõi giá trị. Một đường nối không giữ được điều đó, vì đưa con trỏ thô cho caller C chính là thao tác chấm dứt kiến thức của trình biên dịch về nó. Lớp do đó xuất hiện hai phía ranh giới, là phát biểu sắc nhất về bản chất ranh giới.

- **Forgotten lock guard.** Thiết lập bearer lấy khoá ngữ cảnh và lỗi kiểm tra khiến hàm trả về mà không nhả khoá, nên worker kế tiếp chạm ngữ cảnh đó chờ vô hạn. Rust loại dạng thường gặp của lỗi này, vì guard nhả khi bị drop và không có unlock viết tay để bỏ qua. Nó không loại bỏ lỗi, vì guard là một giá trị và rò rỉ giá trị là an toàn. Nơi chu trình tham chiếu rò bộ nhớ, đây rò chính ngữ cảnh.

Cùng nhau mười bốn lớp này tạo danh mục được tiêm ở Chương 6, và sự tách hai nhóm là điều cho phép nghiên cứu báo cáo một ranh giới thay vì một phán quyết. Buffer overflow được mô tả ở trên cho đầy đủ nhưng không được tiêm, vì không bản hiện thực nào làm số học con trỏ thô trên kho ngữ cảnh, nên một ca tiêm sẽ là cấu trúc giả tạo chứ không phải lỗi hợp lý trong phân hệ này. Bảng 3.1 gom danh mục với kết quả kỳ vọng của từng ca *trước* khi nghiên cứu chạy, đúng dạng mà runner ghi nhận: một kỳ vọng viết trước có thể bị vi phạm theo cả hai chiều và được báo là sai lệch, còn kỳ vọng hình thành sau sự kiện thì không thể.

Bảng 3.1: Danh mục lỗi tiêm: mỗi lớp theo hình thức hợp lý trong Quản lý Ngữ cảnh UE, và kết quả ghi nhận trước nghiên cứu. Cột cuối là dự đoán chứ không phải kết quả. Bảng 6.9 báo cáo điều đã đo, và runner lỗi đánh dấu sai khác theo cả hai chiều thay vì hấp thụ vào kết luận. Buffer overflow được liệt kê cho đầy đủ và không tiêm: không bản hiện thực nào làm số học con trỏ trên kho ngữ cảnh.

Lớp lỗi	Hiện thân trong Quản lý Ngữ cảnh UE	Cách thất bại	Kỳ vọng trong Rust an toàn
Họ an toàn bộ nhớ: vòng đời và biên đối tượng			
Use-after-free	Bộ xử lý báo hiệu đang bay giải tham chiếu ngữ cảnh mà `release` đồng thời đã xoá khỏi bản đồ	Hỏng dữ liệu hoặc sập	Ngăn chặn
Double-free	`release` truyền lại không bất biến, nên đường giải phóng giải phóng tài nguyên ngữ cảnh lần hai	Hỏng dữ liệu hoặc sập	Ngăn chặn
Use-after-move	Thủ tục nhận trả lời từ ngữ cảnh vừa move vào kho	Danh tính rỗng im lặng	Ngăn chặn
Buffer overflow	Không có hiện thân hợp lý: kho ngữ cảnh chỉ qua API container	n/a	Không tiêm
Họ đồng thời: truy cập không đồng bộ hoặc sai thứ tự			
Iterator invalidation	Bearer được thêm trong khi đang lặp danh sách data radio bearer của ngữ cảnh	Hỏng dữ liệu hoặc sập	Ngăn chặn
Data race	Hai worker cập nhật cùng trường ngữ cảnh, bộ đếm hoặc cờ trạng thái, không đồng bộ	Giá trị xé hoặc mất	Ngăn chặn
Unguarded shared mutation	Ngữ cảnh chia sẻ giữa luồng và bị thay đổi không có quy tắc sở hữu	Giá trị xé hoặc mất	Ngăn chặn
Thread-unsafe handle shared	Tay cầm ngữ cảnh có bộ đếm không nguyên tử được truyền cho hai shard worker	Bộ đếm hỏng	Ngăn chặn
Nhóm ranh giới: đưa vào vì Rust an toàn được kỳ vọng không *ngăn chúng*			
Lock-order deadlock	Hai bàn giao đối chiều khoá nguồn-rồi-đích và đích-rồi-nguồn	Treo	Không ngăn
Lost update	Read-modify-write của bộ đếm tách qua hai vùng tới hạn, mỗi truy cập đồng bộ nhưng tổ hợp không nguyên tử	Giá trị sai im lặng	Không ngăn
Reference-cycle leak	Ngữ cảnh và đối tác bàn giao giữ tham chiếu sở hữu lẫn nhau, nên không bộ đếm nào về không	Tăng trưởng không giới hạn	Không ngăn
Integer overflow (release build)	Tính bước TEID của bearer vượt độ rộng kiểu	Giá trị sai im lặng	Không ngăn
Worker-failure blast radius	Một shard worker thất bại giữa thủ tục khi đang giữ trạng thái shard	Sự cố vượt một shard	Không ngăn
Double-free across the seam	Máy chủ C++ giải phóng tay cầm mờ tới ngữ cảnh Rust hai lần	Hỏng dữ liệu hoặc sập	Không ngăn
Forgotten lock guard	Thiết lập bearer trả về khi kiểm tra thất bại mà không nhả khoá ngữ cảnh	Treo	Không ngăn

3.3 Độ trễ đuôi và tính dự đoán được trong RAN

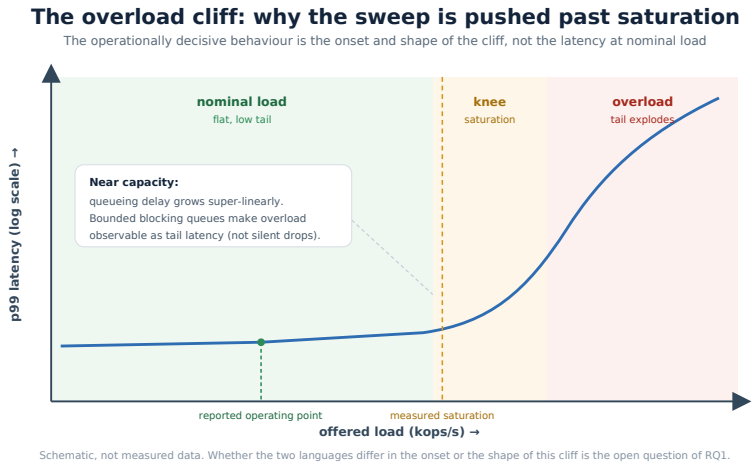
Mục 2.2 đã nêu vì sao đuôi chứ không phải trung vị chi phối hệ nhạy độ trễ, và đo đúng đòi hỏi gì. Điều mục đó để ở mức chung, mục này làm cụ thể, vì hai thuộc tính

của bối cảnh RAN biến ưu tiên phương pháp thành chỉ số chi phối.

Hai thuộc tính đó là fan-out và quy mô, minh hoạ ở Hình 2.2. Một hành động người dùng thấy, thiết lập cuộc gọi hoặc bàn giao, trở thành nhiều thao tác mặt điều khiển phụ thuộc mà thành phần chậm nhất chặn hoàn tất, và một trạm bận đẩy hàng nghìn thủ tục như vậy mỗi giây trên hàng triệu thiết bị kết nối. Gộp lại, p99.9 của một thao tác Quản lý Ngữ cảnh UE không phải nét lạ ở rìa phân phối. Đó là tốc độ thủ tục suy giảm mà mạng duy trì liên tục.

Quy mô mà các phân vị đó quan trọng không phải giả định. Một nghiên cứu đo trên RAN 5G phân tách đo thủ tục UE Context Setup từ CU-CP tới DU qua F1-C, và thiết lập ngữ cảnh bearer từ CU-CP tới CU-UP qua E1, dưới các sơ đồ bảo vệ giao diện khác nhau, và thấy lựa chọn đáng hàng trăm micro giây: trên F1-C, IPsec thêm cỡ 120 μs trung bình cho thiết lập ngữ cảnh UE, so với khoảng 430 μs cho DTLS [58]. Chi phí cỡ đó được coi là đáng kể đúng với họ thủ tục mà luận văn tái hiện thực, điều đặt các thao tác này vào chế độ mà hàng chục micro giây là đại lượng kỹ thuật chứ không phải nhiễu.

Nghiên cứu do đó đặc trưng hoá toàn bộ phân phối độ trễ, tập trung vào các phân vị cao (p99 và p99.9) và giá trị cực đại, cùng cách chúng suy giảm khi tải đưa vào tiến tới rồi vượt bão hoà (Chương 6). Dừng ở tải danh định sẽ bỏ lỡ chế độ quyết định chất lượng dịch vụ, vì độ trễ xếp hàng tăng siêu tuyến gần công suất. Đó cũng là chế độ mà khác biệt giữa hai bản hiện thực của một thiết kế, nếu có, còn chỗ để lộ ra.



Hình 3.2: Vách quá tải. Độ trễ đuôi thấp và phẳng khi tải đưa vào dưới công suất phục vụ, rồi leo dốc mạnh khi hệ thống bị đẩy qua bão hoà và yêu cầu xếp hàng sau các hàng đợi chặn có giới hạn. Hành vi quyết định vận hành là chỗ và hình dạng vách này bắt đầu, đó là lý do quét tải được đẩy qua điểm bão hoà thay vì dừng ở tải danh định. Hình là sơ đồ hiện tượng chứ không phải đồ thị dữ liệu nghiên cứu: liệu hai ngôn ngữ có khác ở chỗ vách bắt đầu hay độ dốc leo chính là điều RQ1 hỏi, và Mục 6.4.1 báo cáo máy phát triển không phân giải được.

4 Phương pháp nghiên cứu

Chương trước nêu nghiên cứu phải đạt gì. Chương này nêu cách đạt. Nó trình bày chiến lược nghiên cứu tổng thể, phương pháp chọn cho từng câu hỏi nghiên cứu và vì sao phù hợp, các kiểm soát giúp so sánh ngôn ngữ công bằng, xử lý thống kê các phép đo, cùng các cam kết tái lập và đạo đức mà công trình tuân theo.

4.1 Hướng tiếp cận nghiên cứu

Nghiên cứu mang tính thực nghiệm với yếu tố kiến tạo (khoa học thiết kế). Hiện vật là một cặp bản hiện thực tương đương chức năng của phân hệ, một bằng C++ và một bằng Rust, cùng khung đo hiệu năng. Cả hai được xây từ một đặc tả cố định duy nhất rồi đánh giá dưới thí nghiệm có kiểm soát. Bằng chứng được tam giác hoá qua ba phương pháp. Đo hiệu năng dưới quét tải trả lời RQ1 và RQ3, nghiên cứu tiêm lỗi có kiểm soát trả lời RQ2, và nghiên cứu tình huống di trú FFI, một phần đo một phần định tính, trả lời RQ4. Ba phương pháp độc lập. Một điểm yếu ở một trong số chúng, ví dụ độ nhạy môi trường của đo độ trễ đuôi, do đó không lan sang toàn bộ kết luận. Mỗi phép đo báo cáo phương sai qua nhiều lần chạy lặp, và Mục 4.3 nêu các kiểm soát giữ ngôn ngữ là biến duy nhất.

Thiết kế đó bám theo hình dạng các câu hỏi. Mỗi câu hỏi hỏi một kết quả (độ trễ đuôi, lỗi được ngăn, dùng tài nguyên, chi phí di trú) phụ thuộc vào một biến, ngôn ngữ hiện thực. Ngôn ngữ vì thế phải là khác biệt có hệ thống duy nhất giữa hai đối tượng thử. Điều này loại trừ dạng so sánh ngôn ngữ phổ biến nhất trong tài liệu, nơi hai chương trình viết độc lập được đo và khoảng cách quy cho ngôn ngữ. Khoảng cách đó còn mang thiết kế, thư viện và kỹ năng tác giả.

Xây cả hai phía từ một đặc tả loại bỏ nhiễu loạn đó. So sánh khi đó là giữa hai mã hoá của cùng thiết kế, và hiện vật vừa là đóng góp vừa là dụng cụ đo nó. Ở nơi câu hỏi trả lời được bằng số, xử lý là định lượng. Ở nơi không thể, như loại bảo đảm trong RQ2 và nỗ lực thiết kế API trong RQ4, xử lý là định tính nhưng vẫn có hệ thống.

Khoảng trống mà thiết kế này nhắm tới (Mục 2.7) chỉ tốt bằng tìm kiếm đã tìm nó, nên tìm kiếm đó được lưu hồ sơ. Phụ lục B nêu cơ sở dữ liệu, tổ hợp thuật ngữ, chuỗi trích dẫn và quy tắc lọc. Hai phát hiện của nó liên quan thiết kế chứ không chỉ thư mục tài liệu.

Các luận văn do nhà cung cấp gần nhất nghiên cứu này (Mục 7.5) không có trong cơ sở dữ liệu trích dẫn. Chuỗi trích dẫn do đó không chạm tới chúng, và chúng phải được tìm bằng cách tra trực tiếp kho lưu trữ của cơ sở. Cùng các tìm kiếm đó cũng chứng minh sự vắng mặt phía kia: không có luận văn Phần Lan về Rust trong bối cảnh truy nhập vô tuyến hoặc viễn thông, và không có ấn phẩm được lập chỉ mục nhắc Rust từ nhà cung cấp chủ trì công trình. Tài liệu chuyên ngành về câu hỏi ngôn

ngữ do đó là tiếng Thụy Điển và do Ericsson lưu trữ, và liệu có chuyển sang đây vẫn chưa được thử. Các luận văn đó là văn bản xám, nên được trích như bằng chứng về điều một nhà cung cấp đã thử và thấy, không như kết quả đã bình duyệt.

Tìm kiếm không trả về nghiên cứu nào, có lập chỉ mục hay xám, đo hành vi độ trễ đuôi của hai bản hiện thực Rust và C++ cùng một phân hệ mặt điều khiển có trạng thái, đồng thời, dưới quá tải.

Bảng 4.1 ánh xạ mỗi câu hỏi nghiên cứu tới phương pháp, hiện vật, dữ liệu và chỉ số trả lời nó. Cả bốn chuỗi cùng kết thúc ở một chỗ: Mục 7.2 phán đoán từng câu hỏi và Mục 7.3 xử lý giả thuyết của nó. RQ1 và RQ3 dùng chung H1, diễn đạt để bao cả tương đương hiệu năng theo nghĩa độ trễ và tài nguyên. Cách diễn đạt đó hóa ra là điểm yếu chính của giả thuyết. RQ2 không nhạy môi trường, và RQ4 cũng vậy trừ một trục: thời gian biên dịch là phép đo thời gian như mọi phép đo khác, nên được đọc như tỉ số giữa hai chuỗi công cụ đo luân phiên trên một máy. RQ1 và RQ3 phụ thuộc máy đo và được báo cáo theo những gì máy đó chịu được (Mục 6.1).

Bảng 4.1: Thiết kế nghiên cứu: cách trả lời từng câu hỏi nghiên cứu. Mỗi câu hỏi gắn với một phương pháp cụ thể, một hiện vật công bố, dữ liệu nó tạo ra, và chỉ số mà kết luận dựa vào. RQ1 và RQ3 mang hai phương pháp thay vì một: quét đo vòng khứ hồi mà một triển khai sẽ trải, còn phân rã thành phần đo phần mà ngôn ngữ chịu trách nhiệm. Mục 4.2.1 nêu vì sao một cái không thay được cái kia. RQ1 và RQ3 được kiểm với H1, RQ2 với H2, và RQ4 với H3.

RQ	Phương pháp	Hiện vật / dữ liệu	Chỉ số chính
RQ1	Quét tải tới quá tải, lặp xen kẽ	Khung đo, histogram độ trễ	trung vị, độ trễ đuôi p99/p99.9, và cực đại so với tải đưa vào
RQ3	Cùng quét, thu tài nguyên	Mẫu `getrusage` mỗi lần chạy	RSS đỉnh, mức dùng CPU, thông lượng
RQ1, RQ3	Phân rã thành phần: gọi lõi trực tiếp, rồi qua dịch vụ	Cặp microbenchmark khớp, lặp xen kẽ	chi phí mỗi thao tác tách thao tác và bàn giao giữa luồng
RQ2	Tiêm lỗi có kiểm soát	Danh mục lỗi, ASan/TSan/UBSan, `rustc`, nhật ký Loom/Shuttle và Miri	phân loại lớp lỗi (biên dịch / lúc chạy / không phát hiện)
RQ4	Nguyên mẫu di trú FFI	Máy chủ C++ + đường nối C-ABI Rust	kích thước đường nối, phạm vi `unsafe`, chi phí tương tác mỗi lần gọi

4.2 Phương pháp cho bốn câu hỏi nghiên cứu

Bảng 4.1 ghép mỗi câu hỏi với một phương pháp.

4.2.1 Đo hiệu năng dưới quét tải (RQ1, RQ3)

RQ1 và RQ3 được trả lời bằng cách lái cả hai bản hiện thực với cùng một khối lượng công việc tổng hợp xác định qua một quét tải đưa vào, từ tải nhẹ qua bão hoà tới quá tải. Hai quyết định thiết kế làm phép đo đáng tin. Bộ sinh tải theo lịch *vòng hở*: mỗi yêu cầu có thời điểm gửi dự định cố định, và độ trễ được đo so với thời điểm đó chứ không so với lúc nhận. Điều đó sửa coordinated omission như Chương 2 định nghĩa. Quét được đẩy qua bão hoà cố ý, vì vách quá tải chỉ xuất hiện khi tải đưa vào vượt công suất phục vụ (Mục 2.2). Độ trễ đi vào histogram dải động cao cấu hình giống hệt hai phía, và dùng tài nguyên (RSS cư trú đỉnh, mức dùng CPU, thông lượng) được lấy mẫu qua hệ điều hành.

Đo cả thành phần lẫn vòng khứ hồi. Một vòng khứ hồi gói hai loại chi phí khác nhau. Một là thao tác ngôn ngữ chạy trong một luồng: tra băm, bảo vệ máy trạng thái, cập nhật trường, tất cả do trình biên dịch sinh. Kia là bàn giao quanh nó giữa các luồng: xếp hàng, đánh thức futex, chuyển ngữ cảnh, hoàn tất, không thuộc ngôn ngữ. Chỉ phần đầu có thể khác giữa C++ và Rust, và không phép đo end-to-end nào tách chúng.

Phương pháp vì thế làm việc ở hai mức. Bộ xử lý thao tác được gọi trực tiếp, không có hàng đợi trên đường đi, rồi qua dịch vụ đa luồng đầy đủ. Hiệu hai mức là bàn giao. Cả hai mức là trung bình theo lô trên vòng lặp dài chứ không phải trung vị mỗi lần gọi, vì đọc đồng hồ tốn hàng chục nano giây, cùng bậc với bản thân thao tác, và dấu thời gian mỗi lần gọi sẽ đo dụng cụ. Hai ngôn ngữ xen kẽ qua các lần lặp và so sánh bằng cùng kiểm định ghép cặp chính xác như quét, nên hai mức khác nhau ở điều đo chứ không ở cách phân tích.

Báo cáo cả hai tách hai câu hỏi mà các câu hỏi nghiên cứu ban đầu không phân biệt rõ: triển khai hình dạng này sẽ trải gì, và ngôn ngữ đang làm gì. Mục 6.4.2 thấy hai mức cách nhau hơn hai bậc độ lớn. Sự phân biệt do đó quyết định liệu kết quả có diễn giải được không. Mức thứ hai được thêm sau vòng đầu chiến dịch, và Mục 7.6 ghi nhận sửa đổi đó.

Cô lập các nguyên thủ đồng bộ tranh chấp. So sánh quá tải ở Mục 6.4.4 quy khoảng cách cho hai mục forced-idiom nơi thư viện chuẩn lệch nhau. Chúng là hàng đợi chặn có giới hạn và tay cầm hoàn tất một lần. Quy kết đó được lập luận từ hình dạng phép đo dịch vụ đầy đủ, nên một microbenchmark khớp thứ ba lái hai nguyên thủ đó một mình, không có công việc `ShardCore` trên đường đi, dưới cùng lặp xen kẽ và kiểm định ghép cặp. Mục 6.4.5 nêu cấu hình và báo cáo liệu nguyên thủ cô lập có tái tạo thứ tự mà dịch vụ đầy đủ cho thấy.

4.2.2 Tiêm lỗi có kiểm soát (RQ2)

RQ2 hỏi ngôn ngữ ngăn lớp lỗi nào. Đo mã không lỗi không trả lời được: vắng mặt lỗi trong kiểm thử đậu không nói gì về việc ngôn ngữ có bắt được lỗi hay không. Phương pháp vì thế đối kháng, theo thực hành đã thành lập của tiêm lỗi phần mềm, nơi lỗi được đưa vào cố ý để xem hệ thống hành xử thế nào khi có lỗi [59].

Một danh mục các lỗi C++ đại diện được viết *cố ý*, rút từ các lớp ảnh hưởng phân hệ này (Chương 3). Mỗi mục là chương trình nhỏ tự chứa mang hương miền: con trỏ cũ vào bản đồ ngữ cảnh, giải phóng kép, thay đổi khi lặp, cập nhật dùng chung không đồng bộ. Mỗi mục được biên dịch và chạy dưới AddressSanitizer, ThreadSanitizer và UndefinedBehaviorSanitizer, ghi công cụ nào phát hiện, nếu có, và ở giai đoạn nào.

Cùng cấu trúc rồi được viết bằng Rust an toàn và biên dịch. Kết quả trình biên dịch được ghi, và nếu biên dịch được thì hành vi chương trình cũng vậy. Các lớp ranh giới không cho trình biên dịch gì để nói, nên phía Rust còn đi tới hai model checker, Loom và Shuttle, và với lớp nằm trong khối `unsafe` tới trình thông dịch Miri. Chúng trả lời nửa "ở giai đoạn nào và với chi phí gì" của RQ2 cho các lớp đó, và mỗi cái được ghép với đối chứng trong đó lỗi đã sửa, để tìm lỗi và báo cáo mọi thứ không trở thành cùng một quan sát.

Việc ThreadSanitizer báo đảo thứ tự khoá phụ thuộc lịch trình, nên một lần chạy sẽ ghi đúng kết quả máy tình cờ cho. Runner chạy ca đó mười hai lần và báo tỉ lệ, để phát hiện gián đoạn không bị nhầm là phát hiện đáng tin.

Vì sao phía C++ chỉ dùng sanitiser. Không có trình phân tích tĩnh nào chạy trên danh mục C++, và đó là quyết định. RQ2 không hỏi công cụ nào tìm nhiều lỗi nhất. Nó hỏi ở *giai đoạn* nào, và với lực gì, mỗi ngôn ngữ loại trừ lỗi, và ba kết quả của phân loại (từ chối lúc biên dịch, phát hiện lúc chạy, bỏ sót im lặng) là các hạng mục câu hỏi cần. Một lượt tùy chọn không thuộc hạng nào: cảnh báo mà bản dựng có thể cấu hình bỏ qua là đối tượng khác với biên dịch không thể thành công, và khác biệt đó là điều phương pháp đo.

Chi phí là phía C++ hẹp hơn điều tốt nhất một dự án C++ có thể làm, vì trình phân tích cấu hình tốt có thể chạm các ca an toàn bộ nhớ nội thủ tục nhỏ, dù không chạm các ca đồng thời. Nghiên cứu vì thế báo cáo công cụ đã chạy, thực hành dựa sanitiser ở Mục 2.3, chứ không phải trần của công cụ C++.

Lỗi tiêm được giữ cố định trong khi ngôn ngữ thay đổi, nên mọi khác biệt kết quả đến từ bảo đảm ngôn ngữ chứ không từ lỗi cụ thể. Phương pháp thiết lập *loại* bảo vệ mỗi ngôn ngữ cung cấp, không phải một con số hiệu năng.

Một quyết định thiết kế quyết định liệu phân loại đáng tin. Danh mục chỉ rút từ các lớp ngôn ngữ đã biết là ngăn được có thể xác nhận giả thuyết nhưng không bao giờ làm lung lay nó, và sẽ mô tả lựa chọn ca của tác giả nhiều như mô tả ngôn ngữ. Danh mục vì thế xây hai nhóm: các lớp trong bảo đảm đã nêu của Rust, và các lớp ngay ngoài nó, nơi sở hữu và kỷ luật `Send`/`Sync` không bảo vệ. Mỗi ca mang kỳ

vọng ghi nhận và runner báo vi phạm theo cả hai chiều, nên kết quả bất ngờ nổi lên như sai lệch thay vì bị hấp thụ vào tường thuật. Điều này đổi lấy một ranh giới chứ không phải điểm số: nghiên cứu có thể nêu lớp nào ngôn ngữ loại bỏ và lớp tương đương nào còn lại, đó là điều quyết định di trú cần.

4.2.3 Nghiên cứu tình huống di trú giao diện hàm ngoại (RQ4)

RQ4 liên quan chi phí thực tế khi áp dụng Rust trong hệ C++ sẵn có, vừa là câu hỏi kỹ thuật vừa là câu hỏi kỹ sư. Nó được trả lời bằng nguyên mẫu xây theo hướng di trú thực tế: máy chủ C++ gọi thành phần Rust qua giao diện nhị phân ứng dụng C. Ba đại lượng được đo trên đó. *Kích thước* đường nối đếm điểm vào, dòng mã và phụ thuộc. *Phạm vi* `unsafe` ghi nơi rời ngôn ngữ được kiểm tra và liệu lõi an toàn có bị ảnh hưởng. *Chi phí mỗi lần gọi* là giá vượt ranh giới, đo so với cùng thao tác gọi bản địa để so sánh cô lập phần đường nối thêm vào. Hai đại lượng đầu đọc từ mã, đại lượng thứ ba là cặp microbenchmark khớp. Ghép chúng tách chi phí thời gian chạy của tương tác khỏi chi phí thật, nằm ở thiết kế API và học hỏi của con người.

Tương tác là một trong ba thuật ngữ RQ4 nêu, và nguyên mẫu chỉ trả lời nó một mình. Ba đại lượng nữa bao hai thuật ngữ còn lại, chọn vì kỹ sư biên dịch chạm tới chúng trước mọi thứ đường nối có thể cho thấy. *Thời gian biên dịch* được đo sạch và tăng dần, cho thư viện lõi một mình và cho mọi mục tiêu mặc định, với cùng số job và lặp xen kẽ hai phía để tải một máy rơi đều lên hai chuỗi công cụ. *Dấu chân tĩnh* được đo trên runner conformance, tệp thực thi duy nhất cả hai ngôn ngữ dựng từ cùng đặc tả, và ở cả hai chính sách liên kết thay vì mặc định mỗi chuỗi công cụ, vì hai mặc định khác nhau ở việc có liên kết thư viện chuẩn hay không và một cặp số đơn sẽ báo lựa chọn đó như khác biệt ngôn ngữ. *Bề mặt bên thứ ba* đếm crate bắc cầu mỗi cấu hình giải quyết và các điểm `unsafe` bên trong, làm cho tuyên bố không phụ thuộc ở Mục 5.4.4 có thể kiểm. Đại lượng đầu nhạy máy chủ, hai kia thì không.

4.3 Công bằng như chiến lược biến kiểm soát

Mỗi câu hỏi nghiên cứu quy một khác biệt cho ngôn ngữ, nên tính hợp lệ phụ thuộc giữ mọi thứ khác bằng nhau. Công bằng vì thế là kiểm soát trung tâm của nghiên cứu, thực thi ở nhiều mức.

Đặc tả được cố định trước khi viết mã. Nó khóa API công khai, máy trạng thái, cấu trúc dữ liệu và chiến lược thuật toán của chúng, cùng mô hình sharding và đồng thời (Mục 5.1). *Nền thời gian chạy* được cân bằng bằng cách liên kết cùng bộ cấp phát hai phía và khớp mức tối ưu hoá bản dựng gần nhất mà hai chuỗi công cụ cho phép. *Khối lượng công việc* đến từ một driver cấu trúc giống nhau giữa các ngôn ngữ, với một seed và quần UE hoạt động cho trước, lái cả hai bản hiện thực qua cùng chuỗi thao tác. Mọi khác biệt độ trễ do đó phản ánh cách mỗi ngôn ngữ thực thi cùng công việc, không phải khác công việc.

Ở nơi hai ngôn ngữ thật sự không diễn đạt được cùng cấu trúc, độ lệch được ghi trong nhật ký forced-idiom. Điều còn không kiểm soát được, chủ yếu bất đối xứng kỹ năng một tác giả, được thừa nhận là mối đe doạ tính hợp lệ (Chương 7) và không được tuyên bố là đã loại bỏ.

Bảng 4.2 nêu chiến lược theo từng biến, và cột thứ ba quan trọng nhất. Thí nghiệm có kiểm soát đáng tin theo mức nó kế toán trung thực những gì không kiểm soát được. Ba hàng ghi phần dư chứ không phải sửa sạch: ngôn ngữ không luôn diễn đạt được cùng cấu trúc, chúng không ship cùng bố cục băm, và một người viết cả hai bản hiện thực. Nhật ký forced-idiom xử lý điều đầu, và hàng cuối đặt tên biến duy nhất không còn kiểm soát nào.

Bảng 4.2: Chiến lược biến kiểm soát: điều được giữ cố định để ngôn ngữ vẫn là khác biệt có hệ thống duy nhất, và điều mỗi kiểm soát không chạm tới. Cột thứ ba không phải danh sách sơ suất mà là tự kể giới hạn của nghiên cứu. Mỗi mục được nối lại như mối đe doạ tính hợp lệ ở Mục 7.6. Chỉ hàng cuối đặt tên biến không có kiểm soát nào.

Biến	Cách giữ cố định	Điều kiểm soát không chạm tới
Thiết kế	Một đặc tả, cố định trước khi viết mã, khóa API công khai, máy trạng thái, cấu trúc dữ liệu và chiến lược thuật toán, cùng cách chia trạng thái thành shard (§5.1)	Ở nơi hai ngôn ngữ không diễn đạt được cùng cấu trúc, độ lệch được ghi chứ không gỡ (Bảng 5.2)
Mô hình đồng thời	Một luồng mỗi shard qua hàng đợi MPSC chặn có giới hạn hai phía, không runtime bất đồng bộ ở phía nào	Không còn điểm mở
Băm	SplitMix64 cho chọn shard và FNV-1a trong shard, ghim giống nhau, để hasher mặc định của thư viện chuẩn không thành lợi thế tình cờ	Container vẫn là bản đồ băm riêng mỗi ngôn ngữ, và hai bên khác bố cục (§6.4.2)
Bộ cấp phát	Cùng `jemalloc` liên kết hai phía, với bản dựng bộ cấp phát hệ thống giữ làm đối chứng mỗi phía	Không còn điểm mở
Tối ưu hoá	`-O2` đối `opt-level = 2`	Tương đương danh nghĩa không phải sinh mã giống hệt
Khối lượng công việc	Một driver xác định. Với seed và quần UE hoạt động cho trước cả hai bản hiện thực thực thi cùng chuỗi thao tác, kiểm bằng đếm theo loại	Tổng hợp và biện minh bằng đặc tả chứ không phát lại từ lưu lượng sản xuất
Dụng cụ đo	Cùng thuật toán và cấu hình histogram HDR, cùng lịch vòng hở, cùng kỷ luật lặp và xen kẽ	Không còn điểm mở
Hành vi	Bộ kiểm thử conformance trung tính ngôn ngữ với bản ghi chuẩn được diff từng thao tác (§5.6)	Tương đương giới hạn hành vi, không phải hiệu năng
Tác giả	Không kiểm soát	Một người viết cả hai bản hiện thực và không thành thạo như nhau cả hai ngôn ngữ

4.4 Xử lý thống kê các phép đo

Các phép đo hiệu năng không xác định, nên số một lần chạy được tránh xuyên suốt. Mỗi cấu hình (ngôn ngữ × bộ cấp phát × mức tải) được đo qua nhiều lần lặp độc lập, và bốn biến thể được *xen kẽ* trong mỗi lần lặp để trôi chậm, nhiệt và hoạt động nền làm nhiễu đều nhau thay vì lệch một phía. Kết quả được tóm bằng trung vị với khoảng tin cậy bootstrap 95% không tham số [60] thay vì trung bình với độ lệch chuẩn, vì phân phối độ trễ đuôi nặng và lệch và khoảng không tham số trên trung vị không giả định chuẩn. Cách này bám hướng dẫn đã thành lập về đánh giá hiệu năng có thống kê chặt, cảnh báo báo cáo một số duy mời kết luận mà biến thiên giữa các lần chạy không ủng hộ [32]. Các phân vị đuôi (p99, p99.9, p99.99) đọc từ histogram HDR, có gom nhóm logarit giữ ba chữ số có nghĩa trên toàn dải để các phân vị cao không bị lượng tử hoá mất. Khoảng được báo đầy đủ, không phải nửa độ rộng đối xứng, vì khoảng bootstrap trên trung vị của mẫu đuôi nặng lệch rõ và nửa độ rộng đủ che biên trên sẽ đặt biên dưới dưới không.

Hai đại lượng này cần ghi chú riêng, vì mức sâu vào đuôi mà một chiến dịch nhìn thấy là thuộc tính của số thao tác đã ghi. Ở tải đại diện một lần chạy hoàn tất cỡ 170 nghìn thao tác đo, nên p99.99 của nó dựa trên khoảng mười bảy quan sát: p99.99 mỗi lần chạy bị nhiễu lấy mẫu chi phối ngay cả trên máy hoàn toàn yên. Đọc đuôi xa vì thế đòi lưu toàn bộ nội dung bucket HDR mỗi lần chạy cạnh các phân vị tóm tắt, để các lần lặp có thể được *gộp* thành một phân phối vài triệu thao tác trước. Khung làm vậy hiện nay, và thiết kế tách gộp và lấy trung bình, vì chúng là hai thống kê khác nhau. Một số gộp sẽ lấy thao tác làm đơn vị phân tích và phân giải đuôi xa, với chi phí không mang khoảng giữa các lần lặp, trong khi các số trung vị-của-lần-lặp báo cáo kèm khoảng giữ lần lặp làm đơn vị và vẫn là cơ sở duy nhất cho so sánh ghép cặp. Jitter cũng sẽ được đo trực tiếp thay vì suy từ phân phối độ trễ, như hiệu trị tuyệt đối giữa độ trễ liên tiếp trong một producer. Đó là đại lượng biến thiên độ trễ gói của RFC 3393 [61], và khớp cách luận văn dùng thuật ngữ: biến thiên từ thao tác này sang thao tác kế, không phải độ trải của cả phân phối. Cả hai không được báo cáo ở Chương 6, vì chiến dịch phân tích ở đó trước khi có dụng cụ đo (Mục 6.4.1).

Dù vậy p99.99 hay jitter cũng không vào họ kiểm định ghép cặp mô tả dưới đây. Điều đó bám cùng quy tắc với phần còn lại của thiết kế chứ không phải hệ quả của dữ liệu thiếu: cả hai tính từ đúng các thao tác mà nhiễu giao thoa chạm tới, tức lớp thống kê mà ghép cặp không cứu được, nên khoảng ghép cặp trên chúng không thể đọc như hiệu ứng ngôn ngữ. Chúng thuộc cột mô tả, và họ kiểm định giữ cỡ mà thiết kế cam kết.

Chỉ khoảng ước lượng thì là cơ sở yếu cho so sánh. Hai khoảng có thể chồng trong khi một biến thể dẫn ở mọi lần lặp, và chúng có thể tách trên khác biệt không tái lập. Nghiên cứu vì thế cũng kiểm định trực tiếp so sánh, và thiết kế xen kẽ là điều làm điều đó khả thi: vì bốn biến thể được đo liền nhau trong mỗi lần lặp, các lần chạy tạo thiết kế khớp mà nhiễu chung của một khối triệt tiêu trong hiệu ghép cặp.

Ngôn ngữ được so với ngôn ngữ với bộ cấp phát giữ cố định, dùng kiểm định hoán vị ghép cặp hai phía chính xác trên hiệu theo lần lặp. Giả thuyết không là mỗi cặp chỉ khác nhiễu giữa các lần chạy, dưới đó dấu mỗi hiệu ghép cặp có thể hoán đổi. Với số lần lặp dùng ở đây mọi một trong 2^n gán dấu được liệt kê thay vì lấy mẫu, nên p-value thu được là chính xác và, như bootstrap, không giả định gì về hình dạng phân phối. Vì cùng so sánh được làm cho nhiều chỉ số và cả hai bộ cấp phát, họ kiểm định được hiệu chỉnh bằng thủ tục Holm bước xuống [62], kiểm soát tỉ lệ lỗi họ mà không giả định độc lập giữa các kiểm định.

Hai quyết định về cách đọc họ đó được đưa ra ở đây chứ không sau kết quả, vì một trong hai có thể bị chọn cho hợp dữ liệu nếu không.

Quyết định thứ nhất là so sánh nào vào họ, và câu trả lời phụ thuộc điểm vận hành chứ không phụ thuộc chỉ số. Dưới bão hoà mọi biến thể hoàn tất công việc nhanh bằng tốc độ được đưa vào, nên thông lượng đạt được ở đó bị ghim vào tải đưa vào theo cấu trúc và không thể biểu đạt hiệu ứng ngôn ngữ dù ngôn ngữ làm gì. Nó được báo cáo kèm khoảng thay vì được kiểm. Trên bão hoà lập luận đảo ngược, vì điều mỗi biến thể vẫn duy trì không bị ghim vào gì và trở thành đại lượng duy nhất có thể tách chúng, nên một điểm vận hành thứ hai được cố định trong vùng quá tải và thông lượng duy trì cùng mức dùng CPU được kiểm ở đó (Mục 6.4.4). Hai cam kết đi kèm. So sánh quá tải gia nhập cùng họ Holm thay vì tạo họ thứ hai, để chia hai điểm vận hành không bị đọc như tách chọn sau khi đã biết p-value. Và không phân vị độ trễ nào được kiểm trên bão hoà, vì producer không theo kịp lịch dự định cố định sẽ tụt lại tích luỹ, khiến các phân vị ở đó là cận dưới chứ không phải ước lượng. Phụ lục C nêu lập luận sau cả hai.

Quyết định thứ hai là kết luận gì khi khoảng và p-value mâu thuẫn. Ở số lần lặp vừa phải chúng thường sẽ: khoảng ghép cặp có thể nằm hoàn toàn một phía không trong khi p-value đã hiệu chỉnh vẫn xa mọi ngưỡng thông dụng. Tin dụng cụ nào không phải lựa chọn tự do, và không phụ thuộc chỉ số có hấp dẫn hay không. Nó phụ thuộc liệu ghép cặp có gỡ nhiễu mà phép đo đó chịu hay không. Ghép cặp triệt tiêu mọi thứ một khối xen kẽ chia sẻ. Nó không triệt tiêu nhiễu rơi vào một lần chạy của cặp mà không rơi vào lần kia. Tóm tắt mức lần chạy như RSS đỉnh không bị lay bởi vài thao tác bị lên lịch lại, và thống kê bậc trung tâm như trung vị của hàng chục nghìn mẫu mỗi thao tác cũng vậy. Với những đại lượng đó, hiệu ghép cặp giữ dấu qua các lần lặp là khác biệt trong phần mềm. Phân vị cao là trường hợp ngược: p99 và p99.9 tính từ đúng các thao tác bị quấy, nên khoảng loại trừ không có thể vừa khớp một lần chạy của cặp không may mắn vừa khớp hiệu ứng thật. Bảng 6.2 vì thế được chia khối theo phân biệt đó, và khoảng ở khối thứ hai không được đọc như phát hiện dù cách không bao nhiêu so với không. Áp quy tắc cho khối thuận lợi và lặng bỏ khối kia là đúng lỗi mà cam kết trước này tồn tại để ngăn.

Độ phân giải mà thiết kế đạt được bị giới hạn bởi số lần lặp, và được nêu rõ chứ không để ngầm. Với 20 lần lặp và 14 so sánh, sàn hai phía chính xác của kiểm định là 0.0000019, và 0.000027 sau hiệu chỉnh, nên null báo ở đây phản ánh dữ liệu chứ không phải số học. Trích cả hai là cách để độc giả phân biệt kết quả null và kết quả

không phân giải được. Chọn số lần lặp để sàn đã hiệu chỉnh nằm dưới 5% thông dụng vì thế là phần giao thức đo, cùng với ghim đồng hồ và cô lập lõi (Mục 6.1). Phụ lục C suy ra hai sàn.

4.5 Tái lập và hiện vật nghiên cứu

Nghiên cứu được thiết kế để bên thứ ba tái tạo kết quả mà không cần mã độc quyền của Nokia. Ba điều làm điều đó khả thi. Các bản hiện thực được lái hoàn toàn bởi đặc tả công khai đã cố định. Khung đo, danh mục lỗi và runner của nó, cùng nguyên mẫu FFI được công bố cùng luận văn. Và mọi hình và bảng trong phần đánh giá do một script duy nhất tạo trực tiếp từ nhật ký đo thô, nên chạy lại chiến dịch trên máy khác tái tạo hiện vật nguyên văn.

Môi trường thực thi được ghi tự động cạnh mỗi tập kết quả: kiểu CPU, kernel, phiên bản trình biên dịch và chuỗi công cụ, cùng cài đặt tần số và lập lịch của testbed. Điều kiện của một phép đo vì thế đi cùng dữ liệu. Điều đó quan trọng cho một phân biệt cụ thể. Tách kết quả nhạy môi trường (quét hiệu năng) khỏi kết quả không nhạy (kiểm thử tương đương, tiêm lỗi, và cấu trúc FFI) là điều cho phép luận văn nêu, theo từng kết quả, độc giả đang nhìn loại nào. Không gì trên đường đi đòi mã nguồn độc quyền, đó là điều làm tái lập bên thứ ba khả thi chứ không chỉ được phép.

5 Thiết kế và Triển khai

5.1 Đặc tả dùng chung: chuẩn công bằng

Trước khi viết bất kỳ mã nào, một đặc tả đã được đóng băng, và không phần nào trong đó gọi tên một ngôn ngữ. Đặc tả cố định API công khai, máy trạng thái UE, cấu trúc dữ liệu và thuật toán đứng sau chúng (bản đồ băm phân mảnh theo mã định danh UE, ví dụ), mức chi tiết của khóa và phân mảnh, và mô hình đồng thời. Cả hai triển khai tuân theo đặc tả. Điều đó giữ cho so sánh là so sánh hai ngôn ngữ, chứ không phải hai thiết kế hay hai mức kỹ năng. Đặc tả là một sản phẩm độc lập, và là biện pháp kiểm soát chính chống thiên lệch triển khai.

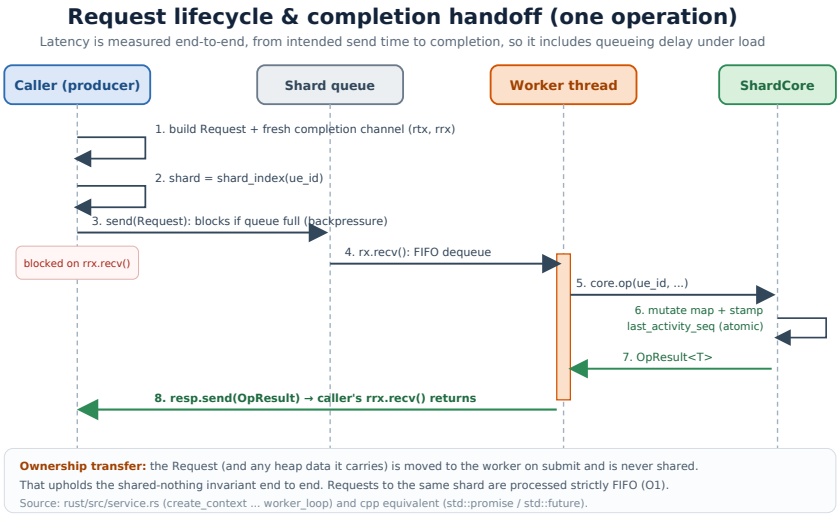
5.2 Mô hình khái niệm và thiết kế đồng thời

Hai triển khai được xây dựng theo đặc tả dùng chung, và chúng làm cùng một việc: một bản C++ hiện đại (C++23, STL, RAII, luồng chuẩn) và một bản Rust (bộ công cụ ổn định, mô hình bộ nhớ dựa trên ownership, đồng thời an toàn). Cả hai dùng một mô hình đồng thời, mô tả là *không chia sẻ, một luồng cho mỗi shard*, cách mà RAN L3 thực tế chia trạng thái UE. Mô hình async và mô hình dựa trên luồng không bao giờ được trộn, và đó là có chủ ý.

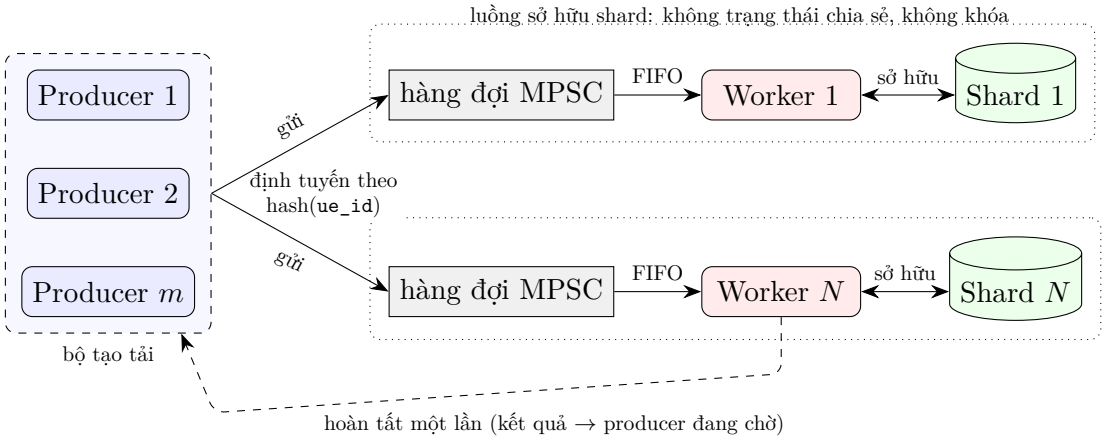
Không gian mã định danh UE được chia thành N_{shards} shard, và mỗi shard thuộc về đúng một luồng worker. Tám là mặc định. Chiến dịch đo ghim bảy thay vì, để mỗi worker và mỗi producer sở hữu một trong các lõi được ghim, như Mục 6.1 ghi lại. Chỉ worker sở hữu đọc hoặc thay đổi ngữ cảnh của shard mình. Worker do đó không chia sẻ trạng thái có thể thay đổi, và dữ liệu ngữ cảnh không cần khóa. Caller (các luồng bộ tạo tải) không chạm trực tiếp vào trạng thái ngữ cảnh. Chúng gửi thông điệp yêu cầu vào hàng đợi MPSC có giới hạn của một shard, rồi chờ trên tay cầm hoàn tất dùng một lần mà worker ghi kết quả vào. Hàng đợi là *có giới hạn và chặn*. Khi hàng đợi của một shard đầy, gửi yêu cầu bị chặn cho đến khi có chỗ trống. Đó là điều tạo ra độ trễ xếp hàng thực tế khi quá tải, và cũng là điều làm độ trễ đuôi quan sát được, vì chính sách bỏ yêu cầu thay vì chặn sẽ che giấu quá tải. Yêu cầu tới cùng một shard chạy nghiêm ngặt theo thứ tự vào trước ra trước.

Cấu trúc này cho so sánh đồng thời an toàn có thứ để đo. Cơ chế được thử nghiệm là hàng đợi, bàn giao hoàn tất, và chuyển ownership của mỗi yêu cầu, chứ không phải xung đột khóa. Một phần trạng thái có thể thay đổi được chia sẻ được phép: một bộ đếm nguyên tử ghi số thứ tự hoạt động theo ngữ cảnh. Nó chỉ là ghi sổ, và không nhánh luồng điều khiển phụ thuộc giá trị của nó. Độ trễ được đo trên toàn đường đi, từ thời điểm gửi dự kiến đến hoàn tất, nên nó bao gồm độ trễ xếp hàng khi có tải và bao phủ cùng một vòng qua luồng trên cả hai bên. Mục 6.4.2 sau đây tách số liệu thành phần ngôn ngữ thực thi và phần hệ điều hành đảm nhiệm. Hình 5.2 cho

thấy luồng dữ liệu, và Hình 5.1 truy vết một thao tác qua đó theo trình tự.



Hình 5.1: Đường đi end-to-end của một thao tác và bàn giao hoàn tất. Caller xây dựng một yêu cầu với tay cầm hoàn tất dùng một lần, định tuyến theo `shard_index(ue_id)` tới hàng đợi có giới hạn của shard sở hữu (chặn nếu đầy), và chờ. Worker duy nhất của shard dequeue theo thứ tự FIFO, thực thi thao tác trên store chỉ mình sở hữu, và ghi kết quả trở lại tay cầm hoàn tất. Ownership của yêu cầu được chuyển cho worker khi gửi và không bao giờ chia sẻ. Độ trễ được đo trên toàn đường đi.



Hình 5.2: Kiến trúc không chia sẻ, một luồng cho mỗi shard. Producer gửi thông điệp yêu cầu tới hàng đợi MPSC có giới hạn, chặn của một shard (chọn bằng hash ghim của `ue_id`) và chờ trên tay cầm hoàn tất dùng một lần. Mỗi shard chỉ được đọc và ghi bởi đúng một luồng worker, nên không có khóa trên trạng thái ngữ cảnh. Chặn khi hàng đợi đầy là điều làm quá tải quan sát được như độ trễ đuôi.

5.3 Lựa chọn kiến trúc và ngăn xếp công nghệ

Bên C++ dùng CMake với Ninja, thư viện chuẩn, và `std::thread`. Bên Rust dùng Cargo và bộ công cụ ổn định. Cả hai liên kết cùng một bộ cấp phát bộ nhớ, điều đó giữ so sánh về hai ngôn ngữ chứ không phải hai bộ cấp phát: C++ liên kết `jemalloc` của hệ thống, và Rust cài một bộ cấp phát toàn cục `jemalloc` (từ `tikv-jemallocator`).

Mỗi bên cũng giữ một bản build với bộ cấp phát hệ thống làm đối chứng. Độ trễ đi qua cùng thuật toán histogram HDR (HDR) [63] trên cả hai bên, cấu hình giống nhau (phạm vi $[1\,\text{ns}, 3.6\,\text{s}]$, ba chữ số có nghĩa). Rust dùng crate `hdrhistogram` chuẩn. C++ dùng bản port header-only của cùng thuật toán gom nhóm, được kiểm tra với phân phối đã biết. Tối ưu build được khớp gần nhất mà hai bộ công cụ cho phép, nên profile release của Rust được ghim ở `opt-level = 2` so với build C++ `-O2`.

5.4 Chi tiết triển khai

Cả hai triển khai tuân theo đặc tả đóng băng từng trường, và cả hai đủ nhỏ để đọc từ đầu đến cuối. Lõi C++ là 674 dòng trên tám header và hai translation unit. Lõi Rust là 762 dòng trên tám module. Bỏ dòng trống và comment còn 517 và 611. Cả hai số mô tả build mà chiến dịch thực sự đo. Một flag build bật hai thăm dò gán như Mục 5.4.4, và các khối điều kiện chọn chúng được bỏ khỏi số liệu, vì đếm chúng sẽ mô tả không phải build được so sánh cũng không phải thăm dò.

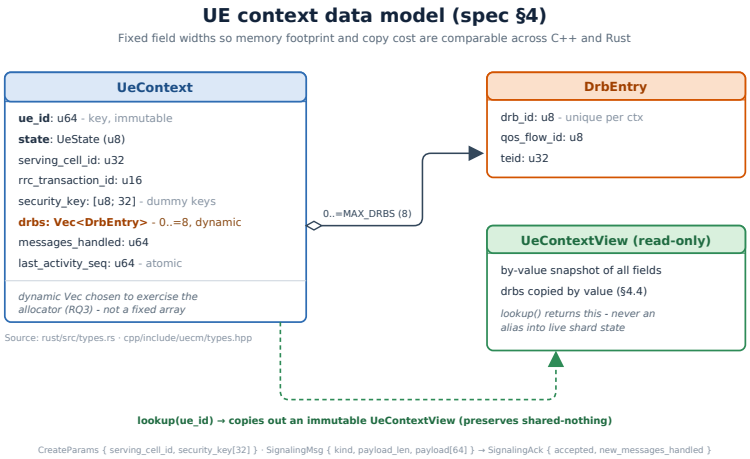
Bản Rust do đó lớn hơn, 13% theo dòng thô và 18% theo dòng mã. Điều đó nên nằm trong văn bản chứ không phải chú thích cuối trang, vì hai lý do. Số dòng là một trong các proxy chất lượng phần mềm mà nghiên cứu này đặt ra để thu thập. Và khác biệt không phải dư thừa ngẫu nhiên: nó là kết quả ròng của nhiều điểm bắt buộc kéo theo chiều ngược nhau, mà Mục 5.4.4 phân tích từng file. Phần còn lại của mục này đi từ mô hình dữ liệu ra cơ chế đồng thời, và báo cáo chi phí mỗi phân kỳ buộc theo ngôn ngữ khi nó xuất hiện.

5.4.1 Ngữ cảnh UE và máy trạng thái

Mỗi ngữ cảnh UE lưu mã định danh, trạng thái vòng đời (enumeration bốn giá trị `Idle`, `Connected`, `Reconfiguring`, `Released`), cell phục vụ, mã giao dịch RRC, một ngữ cảnh bảo mật giả 32 byte, danh sách động tối đa tám data radio bearer, và hai bộ đếm (số thông điệp xử lý và số thứ tự hoạt động). Danh sách data radio bearer cố ý là container có thể mở rộng (`std::vector` / `Vec`) chứ không phải mảng cố định, để nghiên cứu bộ cấp phát (RQ3) quan sát cấp phát thực tế theo ngữ cảnh. Đọc trả về bản sao bất biến (`UeContextView`) sao chép ra từ worker, không phải tham chiếu vào trạng thái shard đang sống. Điều đó giữ cam kết không chia sẻ và giữ hai triển khai giống nghĩa hoàn toàn. Hình 5.3 cho thấy bản ghi và đường đọc đầy đủ.

Khai báo trường giống hệt không tạo ra object giống hệt. Đo trên máy đánh giá, `UeContext` chiếm 96 byte trong C++ và chỉ 88 byte trong Rust. Chính sách bố cục giải thích toàn bộ. C++ đảm bảo member dữ liệu không tĩnh nằm theo thứ tự khai báo, nên enumeration một byte `state` giữa mã định danh tám byte và cell id bốn byte buộc 3 byte padding, và khóa bảo mật 32 byte, kết thúc ở offset 50, buộc thêm 6 trước vector bearer căn tám byte. Rust không có chú thích `#[repr(C)]` ở đây, nên có thể sắp xếp lại trường, và nó làm vậy. Nó gom các scalar nhỏ và cả hai lỗ hổng biến mất.

Đặc tả không cố xóa khác biệt đó, và lý do cần nói rõ. Ghim bố cục Rust bằng `#[repr(C)]` sẽ phạt Rust chỉ vì tuân thủ quy tắc padding của C++. Sắp xếp lại khai báo C++ thủ công sẽ là tối ưu mà đặc tả không yêu cầu bên nào. Khoảng cách do đó được ghi là hiệu ứng cấp ngôn ngữ và sau đó được giới hạn. Với 5 000 UE hoạt động mà sweep dùng, khoảng 8 byte tương đương khoảng 40 kB. RSS được đo bằng megabyte, nên điều này không ảnh hưởng so sánh RQ3.



Hình 5.3: Mô hình dữ liệu ngữ cảnh UE dùng chung cho cả hai triển khai. Chiều rộng trường được cố định để dấu chân bộ nhớ và chi phí sao chép có thể so sánh giữa ngôn ngữ. Danh sách data-radio-bearer là `Vec`/`std::vector` động có chủ ý, để kích hoạt bộ cấp phát cho RQ3. `lookup` trả về `UeContextView` bất biến sao chép theo giá trị chứ không phải alias vào trạng thái shard đang sống, giữ cam kết không chia sẻ.

Bản ghi được điều khiển bởi vòng đời của Hình 3.1. Mỗi thao tác là một chuyển trạng thái có bảo vệ, và Bảng 5.1 cho bảng triển khai đầy đủ. Hai tính chất giữ ở mọi nơi, và chúng là điều làm hai triển khai có thể so sánh từng thao tác. Thứ nhất, mỗi thao tác kiểm tra trước khi thay đổi bất cứ gì. Chuỗi bảo vệ xác thực mã định danh, tìm ngữ cảnh, kiểm tra trạng thái, và trả `OpError` khớp trước khi chạm trường nào. Thao tác bị từ chối do đó để ngữ cảnh bit-identical với trước đó. Thứ hai, không thao tác nào có thể thành công một phần, vì mỗi thao tác chỉ thực hiện tối đa một đột biến sau bảo vệ cuối.

Hai hành vi cần nói rõ, vì bảng che chúng. Trạng thái `Released` thực sự không được lưu. `release_context` xóa entry khỏi map shard hoàn toàn, nên `Released` chỉ là nhãn đi qua trong đặc tả chứ không phải giá trị triển khai có thể bị bắt khi nghỉ. Lần release thứ hai do đó trả `NotFound` chứ không phải `InvalidState`. Trường hợp khác là `abort_reconfigure`. Nó khôi phục *trạng thái* về `Connected`, nhưng không khôi phục serving cell mà `begin_reconfigure` đã ghi đè bằng mục tiêu handover. Đó là đơn giản hóa có chủ ý, vì mô hình không lưu cell trước handover để rollback. Vẫn là cam kết ngữ nghĩa chứ không phải sơ suất, và cả hai triển khai làm giống nhau, điều bộ parity của Mục 5.6 kiểm tra.

Bảng 5.1: Bảng chuyển trạng thái triển khai, giống hệt ở cả hai ngôn ngữ. Mọi thao tác còn trả `InvalidArgument` cho `ue_id` bằng không, và thao tác bị từ chối để ngữ cảnh không thay đổi. `Released` không bao giờ được lưu: release xóa entry.

Thao tác	Trạng thái yêu cầu	Trạng thái kết quả	Lỗi
`create_context`	ngữ cảnh không có	`Idle`	`AlreadyExists`, `CapacityExceeded`
`setup_connection`	`Idle`	`Connected`	`NotFound`, `InvalidState`
`handle_signalling`	`Connected`	`Connected`	`NotFound`, `InvalidState`
`add_drb`	`Connected`	`Connected`	`NotFound`, `InvalidState`, `CapacityExceeded`, `AlreadyExists`
`begin_reconfigure`	`Connected`	`Reconfiguring`	`NotFound`, `InvalidState`
`complete_reconfigure`	`Reconfiguring`	`Connected`	`NotFound`, `InvalidState`
`abort_reconfigure`	`Reconfiguring`	`Connected`	`NotFound`, `InvalidState`
`release_context`	mọi trạng thái sống	entry bị xóa	`NotFound`
`lookup`	mọi trạng thái sống	không thay đổi	`NotFound`

5.4.2 Phân mảnh, store và hàng đợi có giới hạn

Cả hai bên ghim hashing, để hasher của thư viện chuẩn không trở thành lợi thế ngôn ngữ ngẫu nhiên. Gán shard chạy `ue_id` qua finalizer SplitMix64, ba vòng xor-shift và nhân với hằng `0xBF58476D1CE4E5B9` và `0x94D049BB133111EB`, rồi giảm modulo số shard. Finalizer quan trọng vì workload vẽ mã định danh tuần tự, và modulo thuần sẽ ánh xạ chúng lên shard theo mẫu lặp cố định. Xáo trộn mã định danh trước làm tải đều mỗi shard ở cả hai ngôn ngữ theo cấu trúc, không phải may mắn.

Container mỗi shard là hash map chuẩn của mỗi ngôn ngữ (`std::unordered_map` / `std::collections::HashMap`), nhưng hasher mặc định bị bỏ qua. Chúng khác quá nhiều để giữ nguyên. `std::hash<uint64_t>` của C++ thường là hàm identity, trong khi `SipHash-1-3` mặc định của Rust mạnh về mật mã và tương ứng chậm hơn. Giữ cả hai sẽ đo lựa chọn thư viện chuẩn, không phải ngôn ngữ. Cả hai map do đó được key qua hash FNV-1a 64 bit ghim trên tám byte little-endian của mã định danh, và một test kiểm tra `BuildHasher` của Rust tạo digest bit-identical với `fnv1a_64` tham chiếu.

Cả hai map reserve capacity ban đầu 4 096 entry khi khởi động để đo ổn định không bị nhiễu bởi rehash. Cấu hình còn lại cũng chia sẻ: mặc định tám shard, giới hạn hàng đợi 1 024 yêu cầu mỗi shard, tối đa tám bearer và payload signalling 64 byte mỗi ngữ cảnh, và giới hạn 200 000 ngữ cảnh mỗi shard. Chỉ số shard được thay từ mặc định, và chỉ bởi chiến dịch, ghim để khớp lõi được cấp.

Hàng đợi chặn có giới hạn là nơi hai thư viện chuẩn phân kỳ rõ nhất. Rust cung cấp nguyên thủy đặc tả yêu cầu. `sync_channel(capacity)` là kênh MPSC có giới hạn mà `send` chặn khi hàng đợi đầy và `recv` chặn khi rỗng, đúng backpressure cần thiết trong một lời gọi. Thư viện chuẩn C++, tới C++23, không có hàng đợi chặn có giới hạn. Cùng ngữ nghĩa do đó được ghép thủ công từ `mutex`, hai `condition_variable` và `std::queue`, trong class 45 dòng.

Điểm so sánh không phải C++ viết kém. Nó ngắn, dùng dạng predicate của `wait` để xử lý wakeup giả đúng, và thả khóa trước khi notify. Điểm là nó tồn tại. Mỗi dòng là nơi lỗi đồng thời có thể lọt vào. Và lập luận đúng đắn cho nó phụ thuộc tác giả, không phải compiler hay thư viện: hai condition variable không bị nhầm, predicate khớp notification, không mất wakeup. Đây là cùng bất đối xứng mà nghiên cứu tiêm lỗi Chương 6 đo, xuất hiện ở đây trong mã thường không có lỗi cố ý thay vì trong bug được đặt vào.

5.4.3 Yêu cầu, phản hồi và vòng đời dịch vụ

Kiểu thông điệp qua hàng đợi là phân kỳ buộc thứ hai. Rust mô hình yêu cầu là `enum` với một variant cho mỗi thao tác. Mỗi variant mang đúng tham số của nó cộng tham số `Sender` theo đúng kiểu kết quả của thao tác đó. C++ không có enumeration mang payload theo idiom chuẩn dùng ở đây, nên yêu cầu là struct có tag. Nó giữ discriminant `OpKind` cùng *mọi* trường tham số mà bất kỳ thao tác nào có thể cần, ghép với một `std::promise`. Kiểu giá trị của promise do đó phải là `std::variant` trên cả bốn kiểu kết quả có thể.

Điều này có ba hệ quả đo được. Thứ nhất là kích thước. `Request` là 160 byte trong C++ so với 96 trong Rust, vì struct C++ luôn mang tham số create, thông điệp signalling 68 byte, entry bearer, và tham số reconfiguration, dù thao tác đang chạy có dùng hay không. Enum Rust được size theo variant lớn nhất riêng. Với mặc định tám shard giữ tối đa 1 024 yêu cầu mỗi shard, occupancy đầy nghĩa là khoảng 1.3 MB lưu trữ yêu cầu trong hàng đợi so với 0.8 MB. Hệ quả thứ hai là chi phí sao chép trả trên mỗi gửi, vì trường không dùng được khởi tạo mặc định và move cùng trường dùng.

Thứ ba quan trọng nhất cho luận điểm luận văn, và là khoảng trống an toàn kiểu. Caller C++ phải khôi phục kết quả có kiểu từ variant bằng `std::get<OpResult<T>>`, ném `std::bad_variant_access` nếu worker ghi alternative caller không mong đợi. Exception đó không thể xảy ra trong mã hiện tại, vì mỗi `OpKind` được xử lý bởi đúng một nhánh và nhánh đó lưu alternative khớp. Nhưng invariant đứng sau nó được duy trì bằng đọc hai file, không phải compiler. Rust thiết lập cùng invariant theo cấu trúc. Sender nhúng trong mỗi variant đã có kiểu, nên không có union phản hồi, không cần unwrap, và không dispatch runtime trên đường trả.

Khởi động dịch vụ cấp phát shard, spawn một luồng worker mỗi shard, và giao worker sở hữu duy nhất store của mình. Đường yêu cầu sau đó là đường Mục 5.2 mô tả. Shutdown là nửa tinh tế hơn. Nó phải dừng worker bị chặn trên hàng đợi rỗng mà không bỏ yêu cầu còn đang chạy. Cả hai triển khai giải bằng sentinel `Stop` in-band. Một yêu cầu `Stop` được đẩy vào hàng đợi mỗi shard, và vì hàng đợi nghiêm ngặt FIFO, mọi yêu cầu đã xếp trước nó chạy hoàn tất trước khi worker thấy sentinel và trả về. Không cần flag đóng riêng, timeout, hay API drain, và không tay cầm hoàn tất bị bỏ chưa fulfill. Luồng worker sau đó được join, nên shutdown chỉ trả về khi mọi shard thực sự dừng.

Cả hai ngôn ngữ gắn điều này với hủy object, và cơ chế là một trong vài chỗ hai bên gần giống hệt: destructor C++ gọi `shutdown()`, và triển khai `Drop` của Rust làm tương tự, nên dịch vụ ra khỏi phạm vi không thể để luồng detached. Cả hai chống gọi đôi bằng flag `running`, làm shutdown idempotent.

Khoảng giữa hai lời gọi là điều kiện tiên quyết cho mọi thao tác, và được viết một lần. Thao tác gửi trước `start()` hoặc sau `shutdown()` là lỗi caller chứ không phải điều kiện runtime, nên được báo ngoài band và không mở rộng tập `OpError` đóng của Mục 5.6. C++ ném `std::logic_error` và Rust panic. Nói rõ không phải hình thức. Đặc tả cấm lời gọi từ đầu, không triển khai nào kiểm tra, và trong khoảng trống đó hai ngôn ngữ không thất bại giống nhau. Sự kiện được báo cùng kết quả an toàn Mục 6.4.7, vì lỗi trong mã viết để đúng mang bằng chứng mà lỗi tiêm không thể.

Một mảnh kỷ luật ownership nữa giữ invariant không chia sẻ từ đầu đến cuối. Yêu cầu được move vào hàng đợi chứ không sao chép, và mang tay cầm hoàn tất, nên producer và worker không bao giờ cùng giữ tham chiếu dùng được tới cùng yêu cầu. C++ buộc bằng làm `Request` chỉ move, tính chất tự động từ `std::promise` chỉ move bên trong. Trong Rust nó theo move semantics thường, và compiler từ chối mọi cố dùng yêu cầu sau khi gửi. Trạng thái có thể thay đổi chia sẻ duy nhất trong cả hai triển khai là bộ đếm hoạt động nguyên tử. C++ giữ bằng tham chiếu, Rust là `Arc<AtomicU64>` clone, và cả hai tăng bằng ordering relaxed vì không nhánh theo giá trị.

5.4.4 Điểm bắt buộc theo ngôn ngữ và chi phí hai lõi

Khi hai ngôn ngữ phải phân kỳ để đáp ứng cùng yêu cầu, phân kỳ được ghi trong log điểm bắt buộc của đặc tả để được cân nhắc chứ không lẫn vào kết quả. Bảng 5.2 gom log đầy đủ, và các mục trước đã định lượng hai mục có hệ quả đo được.

Bảng 5.2: Log điểm bắt buộc: mọi điểm hai triển khai phân kỳ để đáp ứng cùng yêu cầu đặc tả, với hệ quả đo được nếu có.

Vấn đề	C++	Rust
Kiểu kết quả	`std::expected<T, OpError>`	`Result<T, OpError>`
Yêu cầu	struct có tag: discriminant cộng mọi trường tham số (160 B)	`enum`, một variant mỗi thao tác, chỉ mang tham số riêng (96 B)
Phản hồi	`std::variant` của bốn kiểu kết quả, caller unwrap bằng `std::get`	không cần: mỗi variant mang sender của kiểu kết quả riêng
Hàng đợi có giới hạn	`mutex` viết tay cộng hai `condition_variable` (51 dòng)	`sync_channel(capacity)`
Hoàn tất	`std::promise` / `std::future`	`mpsc::channel()` mới mỗi yêu cầu
Map mỗi shard	`std::unordered_map` với functor `FnvHash`	`HashMap` với `FnvBuildHasher`
Bộ đếm hoạt động	`std::atomic<uint64_t>` giữ bằng tham chiếu	`Arc<AtomicU64>` clone vào mỗi worker
Tháo dỡ	destructor gọi `shutdown()`	`Drop` gọi `shutdown()`
Mức tối ưu	`-O2`	`opt-level = 2`

Log cũng giải thích khác biệt số dòng ở đầu mục này, và nói nhiều hơn số ròng, vì đóng góp không cùng chiều. Bảng 5.4 chia khoảng 88 dòng theo vấn đề. Rust *ngắn hơn* nhiều ở đúng một chỗ, hàng đợi có giới hạn, nơi không cần mã so với 51 dòng C++. Nó dài hơn gần như mọi nơi khác, và ba mục chiếm phần lớn phần dư.

Lớn nhất là module dịch vụ, vì lý do trên. Vì phản hồi C++ là một `variant`, mọi thao tác công khai mười đi qua một helper `submit` chia sẻ, và mỗi wrapper có kiểu co còn hai hoặc ba dòng. Sender có kiểu theo variant của Rust không cho phép helper tương tự không dùng generic, nên mỗi thao tác lặp cùng chuỗi: tạo kênh hoàn tất, dựng variant, gửi, nhận. Mục tiếp cho thấy trade-off rõ nhất, và là hashing. C++ đáp ứng yêu cầu hasher của `std::unordered_map` bằng functor năm dòng bọc `fnv1a_64`. Rust phải cung cấp toàn bộ apparatus trait, mỗi dòng boilerplate, và đổi lại là hasher dispatch tĩnh, kiểm tra kiểu thay vì callable theo hình thường. Mục thứ ba là cây module và danh sách re-export công khai, việc C++ có miễn từ cấu trúc header.

Không phần nào là lỗi của bên nào, và không xuất hiện trong số liệu hiệu năng. Đó là loại chi phí kỹ thuật mà log điểm bắt buộc tồn tại để phơi bày. Rust tiêu dòng nguồn cho machinery trait và lặp tại call site, và đổi lại là cam kết compiler có thể kiểm tra. Sau đó nó thu hồi phần lớn chi tiêu ở chỗ thư viện chuẩn cung cấp nguyên thủy mà C++ không có.

Đếm dòng chỉ trả lời nửa câu hỏi reviewer của codebase sẽ hỏi, vì nó co một dòng boilerplate và một dòng nhánh là cùng loại. Bảng 5.3 do đó thêm độ phức tạp cyclomatic, đo bằng `lizard` [64] trên cả hai lõi. Một công cụ áp dụng cho cả hai ngôn ngữ làm số liệu có thể so sánh, vì nó dùng cùng định nghĩa điểm quyết định mỗi bên. Công cụ riêng mỗi ngôn ngữ sẽ đếm hơi khác. Hai đo lường sau đó đi ngược

chiều. Rust dài 18% hơn C++ theo dòng mã, nhưng tổng cyclomatic complexity thấp 15%, 81 so với 95, trải trên hơi nhiều hàm (40 so với 37). Cả hai lõi đều không đòi hỏi theo ngưỡng thường, và cả hai có median complexity mỗi hàm bằng một, nên số hàm ở đỉnh mang tổng hợp.

Đỉnh là nơi khác biệt nằm, và truy ngược phân kỳ đã mô tả, không phải thứ rải mỏng. Hàm phức tạp nhất trong lõi C++ là `worker_loop` ở 13, hơn gấp đôi đối tác Rust 6. Nó switch trên discriminant `OpKind` rồi lưu alternative khớp vào `variant` phản hồi. Thiết kế struct có tag Mục 5.4.3 do đó gom toàn bộ dispatch và lập luận đúng đắn đi kèm vào một hàm nhiều nhánh. Sender có kiểu theo variant của Rust loại bỏ dispatch đường trả hoàn toàn, nên worker Rust không có gì tương đương unwrap. Trade-off sắc hơn số dòng gợi ý. Dòng thêm Rust đổi lại bằng nhánh ở chỗ nhánh khó review nhất. Và invariant `std::bad_variant_access` mà Mục 5.4.3 mô tả là duy trì bằng đọc hai file nằm trong đúng hàm mà đo lường này đánh dấu.

Bảng 5.3: Proxy kích thước và cyclomatic complexity cho hai lõi, đo bằng `lizard` [64] áp dụng giống nhau cho cả hai ngôn ngữ. Driver benchmark, test, runner conformance, seam FFI, và khối điều kiện thăm dò của Mục 5.4.4 bị loại. Chỉ mã so sánh mà hợp đồng parity ràng buộc được đếm. Complexity là một cộng số điểm quyết định trong hàm. Số dòng là NLOC theo `lizard`, cũng loại dòng chỉ có brace và preprocessor. Đó là cùng đo lường trích ở đầu Mục 5.4.

Đo lường	C++	Rust
Files	10	8
Functions	37	40
Lines of code (blank and comment lines excluded)	517	611
Total cyclomatic complexity	95	81
Mean complexity per function	2.6	2.0
Median complexity per function	1	1
Most complex function	13 (`worker_loop`)	6 (`add_drb`)

Bảng 5.4: Nguồn gốc khoảng 88 dòng giữa hai lõi, theo vấn đề (dòng thô, bao comment. Số C++ gộp header và translation unit. Khối điều kiện thăm dò loại như Mục 5.4). Rust chỉ ngắn hơn ở chỗ thư viện chuẩn cung cấp nguyên thủy C++ thiếu.

Vấn đề	C++	Rust	Δ
Service (queues, workers, API)	234	278	+44
Hashing (functor vs. trait pair)	36	70	+34
Module tree and re-exports	0	26	+26
Data model	73	85	+12
Message types	48	60	+12
Configuration	18	29	+11
Shard core (state machine)	182	188	+6
Error model	32	26	−6
Bounded queue	51	0	−51
Tổng	674	762	+88

Một tính chất của lõi Rust không phải chi phí, và nó kết mục này vì là thứ các chi phí trên mua được. Thư viện Rust chứa *không* mã `unsafe`, và được kiểm tra cơ học trên crate. Safe Rust do đó đảm bảo không data race cho lõi này ở thời điểm biên

dịch, qua trait `Send` và `Sync`. Store shard được một worker sở hữu và không chia sẻ, kiểu yêu cầu được move qua ranh giới luồng, và phần trạng thái thực sự chia sẻ là bộ đếm hoạt động nguyên tử. Không phần nào cần sanitiser runtime để thiết lập, và không phần nào cần coverage test để giữ. Triển khai C++ không làm quản lý bộ nhớ thủ công ngoài container STL và không khóa tường minh ngoài class hàng đợi, và được kiểm tra không data race dưới ThreadSanitizer.

Hai câu không phải cùng loại bằng chứng, và khác biệt đó là luận điểm an toàn trung tâm của luận văn ở dạng nhỏ. Tính chất Rust là định lý về chương trình, compiler giải quyết trên mọi build thường và bao mọi thực thi. Tính chất C++ là quan sát về các thực thi binary có instrument tình cờ chạy, và phải được thiết lập lại sau mọi thay đổi, trên build hầu hết dự án không chạy mặc định. Định lượng hai loại cam kết cách xa nhau là mục đích nghiên cứu tiêm lỗi Chương 6.

5.5 API công khai và ngữ nghĩa thao tác

Cả hai triển khai expose cùng API nhỏ, theo vòng đời, theo đặc tả dùng chung, nên bề mặt caller thấy giống hệt ngoài quy ước tên của mỗi ngôn ngữ. Dịch vụ được tạo với số shard cố định và giới hạn hàng đợi mỗi shard. Caller sau đó gọi thao tác ngữ cảnh, mỗi thao tác theo `ue_id` mục tiêu: create, setup, handle-signalling, add-bearer, reconfigure begin/complete/abort, release, và tra trạng thái. Mỗi thao tác là *total* và *typed*. Nó trả giá trị thành công hoặc `OpError` có kiểu từ enumeration đóng năm giá trị (`NotFound`, `AlreadyExists`, `InvalidState`, `CapacityExceeded` và `InvalidArgument`). Thao tác bất hợp với trạng thái hiện tại của ngữ cảnh (Hình 3.1) trả lỗi khớp và để ngữ cảnh không đổi. Nó không đột biến ngữ cảnh, và không báo thất bại ngoài band.

Lựa chọn quan trọng cho công bằng cũng như đúng đắn. Không triển khai nào dùng exception hay mã lỗi làm kênh luồng điều khiển ẩn, nên chi phí và luồng điều khiển của đường thất bại hiển nhiên và giống hệt hai bên. Điều đó cũng nghĩa là cùng vector conformance (Mục 5.6) có thể khẳng định đồng ý từng thao tác cả giá trị trả và trạng thái kết quả.

5.6 Kiểm tra tương đồng

Toàn bộ so sánh dựa trên hai bản chức năng giống hệt, nên parity được kiểm tra chứ không giả định. Bộ conformance trung lập ngôn ngữ viết vector test là chuỗi thao tác với kết quả mong đợi và tóm tắt trạng thái sau. Runner mỗi ngôn ngữ thực thi vector và xuất transcript một định dạng cố định, và hai transcript được diff để đồng ý từng thao tác. Bộ có seed bao phủ toàn mô hình lỗi, đường reconfiguration begin/complete/abort, data-radio-bearer đầy và tràn, và tính quyết định gán shard. Cả hai triển khai hiện pass và đồng ý. Workload benchmark thêm tín hiệu parity thứ hai, độc lập với thứ nhất. Với seed và quần UE hoạt động cho trước, số đếm theo loại thao tác của workload mixed được kiểm tra giống hệt giữa ngôn ngữ, xác nhận

cả hai triển khai được điều khiển qua cùng đường mã.

Hai điều giới hạn điều đó thiết lập, và cả hai được trả lời chứ không nhượng bộ. Cross-diff hai triển khai không phân biệt “cả hai đúng” với “cả hai sai cùng cách”, và đó là chế độ thất bại mà đặc tả dùng chung và một tác giả làm dễ nhất. Kết quả mong đợi trong vector viết tay cũng đến từ đặc tả qua người viết triển khai. Triển khai thứ ba của cùng đặc tả viết làm oracle độc lập. Nó viết bằng Python, không chia sẻ mã với ngôn ngữ nào, và phải tái tạo mọi 223 kỳ vọng suy ra tay trước khi được tạo kỳ vọng riêng. Sau đó nó tạo chuỗi thao tác ngẫu nhiên dưới nhiều profile: vòng đời thực tế, chuỗi adversarial nặng về chuyển trạng thái bất hợp, chuỗi ở giới hạn capacity, và chuỗi stress gán shard. Mô hình tính mọi kết quả mong đợi. Chiến dịch khoảng 793 000 thao tác tạo không có phân kỳ. Hai triển khai đồng ý với nhau, và với cách đọc độc lập đặc tả, trên chuỗi không ai chọn. Generator có seed và quyết định, nên mọi phân kỳ tương lai là hồi quy tái lập vĩnh viễn chứ không phải giai thoại.

Ba kiểm tra nữa giới hạn bộ theo hướng khác. Ma trận chuyển trạng thái của đặc tả được duyệt đầy đủ chứ không mẫu: mỗi thao tác áp lên ngữ cảnh ở mọi trạng thái nó có thể chiếm. Mỗi trường hợp khẳng định lỗi chính xác, trạng thái kết quả khi thành công, và không thay đổi trường khi thất bại, kể cả thứ tự ưu tiên lỗi đặc tả để không thứ tự. Triển khai C++ sau đó chạy dưới AddressSanitizer, ThreadSanitizer và UndefinedBehaviorSanitizer trên toàn bộ, và không báo gì. Điều đó quan trọng cho công bằng. Safe Rust mang không data race và an toàn bộ nhớ theo cấu trúc, nên không chạy sanitiser thì nhánh C++ tương đương sẽ dựa trên chưa thất bại thay vì đã được kiểm tra. Miri, chạy trên bộ máy trạng thái Rust, cũng không báo hành vi không định nghĩa. Thứ ba, phạm vi bộ được đo chứ không khẳng định, vì kết quả âm từ sanitiser chỉ rộng bằng mã đã chạy. Coverage dòng của thư viện C++ là 94.6% và thư viện Rust 98.2%, và processor thao tác lõi và facade dịch vụ của cả hai triển khai được bao phủ đầy đủ. Hai số không so sánh nghiêm ngặt, vì `gcov` và `llvm-cov` không phân giải dòng thực thi giống nhau. Trên vùng chi tiết hơn `llvm-cov` đếm, số Rust là 99.3%. Mỗi số giới hạn luận điểm về triển khai của nó.

6 Thiết lập thực nghiệm và Đánh giá

6.1 Cấu hình testbed

Hai môi trường thực thi phải được phân biệt, vì luận văn cố ý báo cáo kết quả với hai mức sức nặng bằng chứng khác nhau, và phân biệt hoàn toàn phụ thuộc nơi chúng được tạo ra.

Môi trường đã dùng. Mọi kết quả hiệu năng báo cáo trong chương này được đo trên một máy chủ dual-socket chuyên dụng: hai bộ xử lý Xeon E5-2680 v4, 16 vCPU trên một núm NUMA trình bày một luồng mỗi lõi, chạy Rocky Linux 8.10 với `g++ 15.1.1` và `rustc 1.97`. Benchmark được ghim tới lõi 2–15, để mỗi worker và mỗi producer sở hữu một lõi và không tranh với hệ điều hành. Bảng D.1 liệt kê máy chủ so với protocol mô tả dưới đây. Mỗi lần chạy ghi bản ghi môi trường riêng (mô hình CPU và topology, kernel, phiên bản compiler và toolchain, và cài đặt tần số, turbo, và multithreading) cùng dữ liệu đo, nên điều kiện của một phép đo đi cùng nó.

Một tính chất của máy này phải nói thẳng, vì nó giới hạn mọi luận điểm đuôi dưới đây: đó là máy ảo. Bản ghi môi trường xác định guest KVM, và ba hệ quả theo sau. Thứ nhất, governor tần số CPU và trạng thái turbo do host điều khiển. Chúng không chỉ chưa đặt mà không quan sát được từ trong guest, nên ổn định tần số được giả định chứ không bị ép. Thứ hai, bản ghi cho thấy CPU steal khác không, khoảng mười millisecond thời gian vCPU mỗi giây. Nhỏ theo tổng, nhưng cùng bậc độ trễ đuôi được đo, và không gì trong guest có thể loại bỏ. Thứ ba, topology không như vẻ ngoài. Guest được hiện hai socket tám lõi single-threaded, không phải hình dạng E5-2680 v4, phần mười bốn lõi có simultaneous multithreading. Topology do đó là tổng hợp. Mười sáu vCPU có nằm trên lõi vật lý riêng hay chia thread sibling là quyết định host, và không đọc được từ trong guest. Sibling sẽ tranh tài nguyên thực thi của một lõi. Lõi cũng không bị gỡ khỏi scheduler bằng `isolcpus`, nên luồng benchmark vẫn có thể mất lõi cho việc khác.

Đây là công cụ tốt hơn nhiều so với laptop phát triển dùng chung mà lượt trước của chiến dịch dùng. Lõi đồng nhất và server-class, không có gì quan trọng khác chạy trên máy, và spread run-to-run ở p99 giảm hơn một bậc magnitude giữa hai lần. Vẫn không phải máy bare-metal mà protocol dưới đây chỉ định.

Protocol mà chiến dịch hiệu năng vẫn thiếu. Số đuôi cấp testbed cần máy bare-metal chuyên dụng. Protocol cho một máy như vậy được chỉ định, viết script, và công bố, nên có thể chạy không cần thiết kế thêm (`bench/TESTBED.md`), và yêu cầu

nằm ở cột phải của Bảng D.1. Việc chạy sẽ bổ sung hẹp hơn trước. Ba trong bốn thứ nó sẽ mua đã có không cần nó: knee bão hòa, hình dạng vách quá tải (Mục 6.4.4), và tái lập so sánh tài nguyên qua đối chứng bộ cấp phát (Mục 6.4.6). Còn lại là đuôi xa *dưới* bão hòa. Ở đó khoảng tin cậy trên p99 của một variant đơn vẫn nhiều lần spread giữa cả bốn variant, và nguyên nhân có thể là steal và preemption scheduler, không phải gì hai ngôn ngữ làm. Kết quả đó do đó được báo là chưa giải quyết chứ không là null, và máy phải sửa, không phải phân tích.

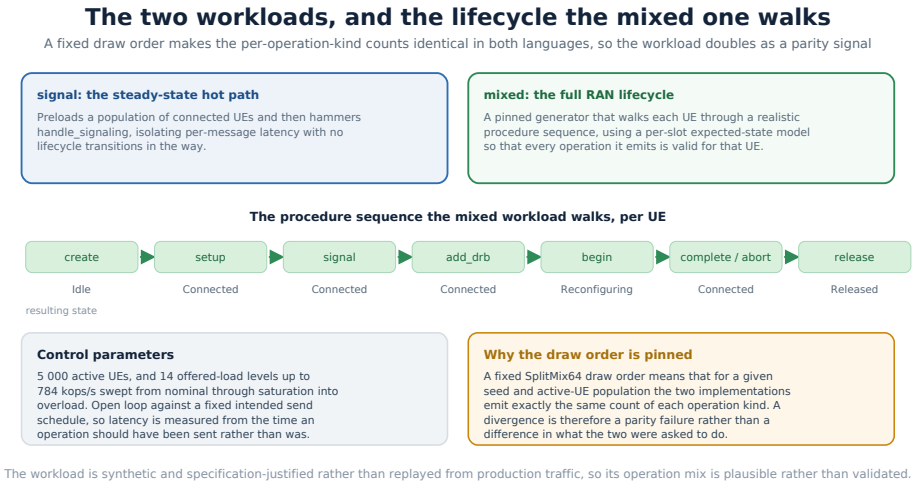
Kỷ luật đo lường chung cho cả hai. Dù host nào, mỗi điểm sweep được lặp 20 lần và báo là median với khoảng tin cậy bootstrap 95%, không bao giờ một lần chạy. Bốn variant ngôn ngữ×bộ cấp phát được xen kẽ trong mỗi lần lặp, nên trôi nhiệt hoặc nền không cân bằng chúng như nhau thay vì thiên một bên. Xen kẽ đó là điều làm host tranh chấp dùng được cho kết luận *so sánh* dù phá hỏng số tuyệt đối, vì nhiễu khi đó được chia sẻ chứ không gán. Throughput được đếm chỉ trong giai đoạn đo, bỏ warm-up khỏi cả mẫu số và tử số, nên throughput đạt được so trực tiếp với tải đề nghị.

Phân chia Bảng D.1 vẽ không phải giữa kết quả quan trọng và không quan trọng. Nó là giữa kết quả là tính chất của compiler và nguồn, mà host này thiết lập chắc như mọi host, và kết quả là tính chất của máy, mà nó không thể thiết lập.

6.2 Workload và phép đo đánh giá

Workload viễn thông tổng hợp nhưng có căn đặc tả được tạo bởi driver quyết định mà cấu trúc cả hai triển khai chia sẻ, nên hai bản được kích hoạt qua cùng đường mã. Hai workload được cung cấp. Workload *signal* preload quần UE đã kết nối rồi đập đường nóng `handle_signalling`, cô lập độ trễ mỗi thông điệp ổn định. Workload *mixed* là generator vòng đời RAN ghim. Nó đi mỗi UE qua chuỗi thủ tục thực tế, tạo ngữ cảnh, thiết lập kết nối, signalling, thiết lập bearer, reconfiguration/handover (begin/complete/abort), và release, dùng thứ tự vẽ SplitMix64 cố định và mô hình trạng thái mong đợi theo slot để mọi thao tác phát ra hợp lệ. Thứ tự vẽ cố định nghĩa số đếm theo loại thao tác giống hệt trong C++ và Rust cho quần UE hoạt động và seed cho trước, đóng vai tín hiệu parity mô tả ở Mục 5.6. Tham số điều khiển là tốc độ sự kiện mục tiêu và số UE hoạt động, sweep từ tải danh nghĩa qua bão hòa và quá tải. Lịch gửi cố định và histogram được hiệu chỉnh, nên stall không gây bias coordinated-omission [31, 63]. Ba chiến dịch đo được chạy: sweep tải cho hiệu năng, nghiên cứu tiêm lỗi, và prototype migration FFI. Hình 6.1 cho thấy hai workload và chuỗi thủ tục workload mixed đi qua.

"Có căn đặc tả" khẳng định gì, và không khẳng định gì. Thuật ngữ được dùng chính xác, vì phân biệt giới hạn workload hỗ trợ gì. *Chuỗi* thao tác được biện minh. Đó là vòng đời kết nối RRC mà đặc tả 3GPP định nghĩa cho UE, từ thiết lập qua signalling trong phiên và thiết lập bearer đến reconfiguration và release [1,



Hình 6.1: Hai workload. Workload *signal* cô lập đường nóng ổn định. Workload *mixed* đi mỗi UE qua toàn chuỗi thủ tục RAN, với mô hình trạng thái mong đợi theo slot đảm bảo mọi thao tác phát ra hợp pháp với trạng thái ngữ cảnh thực sự ở. Vì thứ tự vẽ SplitMix64 được ghim, seed và quần UE hoạt động cho trước tạo số đếm theo loại thao tác giống hệt ở cả hai ngôn ngữ, nên phân kỳ là thất bại parity chứ không phải khác biệt trong việc hai triển khai được yêu cầu làm.

4]. Mô hình trạng thái mong đợi theo slot sau đó đảm bảo mọi thao tác driver phát ra hợp pháp với trạng thái ngữ cảnh thực sự ở, nên workload không đẩy dịch vụ xuống đường đặc tả cấm. *Tỷ lệ* không được biện minh cùng cách. Tần suất cell thực handover so với thiết lập bearer phụ thuộc triển khai, mobility, và thời điểm trong ngày, và không có số công khai cố định. Mix dùng ở đây do đó hợp lý chứ không được xác thực. Mục 7.6 báo đó là mối đe dọa tính ngoại sinh, và replay trace đứng đầu danh sách công việc tương lai vì lý do này.

Với câu hỏi *so sánh*, phân biệt ít tốn hơn so với câu tuyệt đối, và nói vì sao là phần lập luận dữ liệu phù hợp nhiệm vụ. Cả hai triển khai thực thi cùng chuỗi cùng thứ tự cho seed và quần UE hoạt động cho trước, và số đếm theo loại thao tác xác minh. Mix không đại diện do đó dịch chuyển cả hai ngôn ngữ bằng nhau và không thể tạo khác biệt giữa chúng. Mix thao tác chưa xác thực giới hạn số đi xa bao nhiêu. Nó không làm suy yếu công bằng của so sánh chúng được dùng tạo.

Bảng D.2 gom cấu hình dịch vụ, workload và chiến dịch ở một chỗ. Các giá trị đó nếu không gom sẽ rải qua đặc tả, Chương 5, và chương này. Tiện khi đọc từng phần, và khó cho ai muốn tái lập chiến dịch hoặc đánh giá tham số có chọn để phù hợp kết quả. Mỗi mục trong khối chiến dịch hoặc cố định bởi đặc tả hoặc tạo từ log đo, nên không phần nào lệch khỏi dữ liệu báo cáo dưới đây. Hai giá trị suy ra được dùng xuyên suốt phần sau. Điểm vận hành đại diện không chọn bằng mắt. Nó là tải sweep bận nhất mà cả bốn variant build vẫn giữ p99 dưới 5 ms, đặt nó ở 56 kops/s, so với bão hòa đo 335 kops/s.

6.3 Dữ liệu đánh giá

Ba chiến dịch tạo ba tập dữ liệu, và chúng khác loại chứ không chỉ nội dung. Phần sau mô tả mỗi tập khi được ghi. Người đọc đánh giá phân tích có đúng cần biết gì được ghi và ở mức chi tiết nào, không chỉ gì được báo. Mọi đại lượng đo chương này trích dẫn đến trang là macro được tạo. `bench/refresh_thesis.sh` tái tạo tất cả, cùng mọi hình và thân bảng, từ log thô, nên rerun di chuyển văn bản và không thể để số tiêu đề bị bỏ lại. Proxy chất lượng phần mềm không thu ở đây mà báo nơi đọc rõ nhất: cạnh log điểm bắt buộc nó giải thích (Bảng 5.3), và cạnh seam FFI, nơi duy nhất một trong hai triển khai chứa `unsafe` (Bảng 6.10).

Sweep hiệu năng. Mỗi trong 1120 lần chạy của thiết kế factorial Bảng D.2 đóng góp một bản ghi, viết dạng JSON newline-delimited. Chiến dịch do đó là log append-only chứ không phải bảng tóm tắt. Không gì được gộp khi ghi, và mọi statistic chương này được tính lại từ bản ghi thô. Bản ghi mang cấu hình lần chạy (ngôn ngữ, bộ cấp phát, workload, số shard và producer, tốc độ mục tiêu), kết quả (thao tác hoàn tất, lỗi, thời lượng wall-clock, throughput đạt), phân phối độ trễ đọc từ histogram HDR (mean, p50, p90, p99, p99.9, maximum, nanosecond), counter tài nguyên qua `getrusage` (peak RSS, giây CPU user và system, mức sử dụng CPU), và số đếm theo loại thao tác. Giữ số đếm loại trong mọi bản ghi là điều làm phân kỳ giữa hai ngôn ngữ hiện là thất bại parity (Mục 5.6) thay vì bị nhầm là khác biệt hiệu năng.

Phân rã thành phần và nguyên thủy tranh chấp. Cấp đo thứ hai Mục 4.2.1 cho tập dữ liệu nhỏ hơn nhiều, hình dạng khác. Với mỗi trong 15 lần lặp xen kẽ nó ghi chi phí batch-mean ở cả hai ngôn ngữ, một lần gọi thao tác trực tiếp và một lần cùng thao tác qua dịch vụ có luồng. Tập thứ ba, cùng hình ghép qua 20 lần lặp, ghi throughput bền vững qua hàng đợi có giới hạn và nguyên thủy hoàn tất riêng (Mục 6.4.5). So sánh FFI Mục 6.4.8 có cùng cấu trúc qua 15 lần lặp. Cả ba được ghép theo cấu trúc, điều test ghép chính xác Mục 4.4 yêu cầu.

Catalogue tiêm lỗi. Tập dữ liệu này là phân loại chứ không số. Mỗi trong 14 trường hợp lỗi cho bốn log công cụ: output AddressSanitizer, ThreadSanitizer và UndefinedBehaviorSanitizer cho chương trình C++, và output `rustc` cho bản Rust an toàn tương ứng. Mỗi trường hợp cũng cho một dòng taxonomy máy đọc được, ghi ba kết quả sanitiser, giai đoạn phát hiện, kết quả compiler với mã chẩn đoán kích hoạt, và kỳ vọng đăng ký trước nghiên cứu. Hai trường hợp boundary gốc cho bảng thứ hai từ model checker, và `unsafe` một bảng thứ ba từ interpreter. Mang kỳ vọng trong dữ liệu thay vì trong văn bản là điều cho runner báo mismatch theo cả hai chiều. Bảng và hình Mục 6.4.7 được tạo từ file đó, không chép từ nó.

Điều đi cùng dữ liệu. Mỗi chiến dịch viết bản ghi môi trường cạnh kết quả. Nó ghi host và kernel, distribution, phiên bản compiler và toolchain, mô hình CPU và

topology, và governor tần số, turbo, và cài đặt simultaneous-multithreading, *kể cả* trường host cho trước không expose. Chi tiết cuối quan trọng hơn nghe. Bản ghi xác định host là guest KVM, và governor và turbo được lưu unavailable thay vì bỏ đi. Hạn chế Mục 6.1 tài liệu do đó là tính chất của tập dữ liệu chứ không chỉ văn bản mô tả, và người đọc có thể kiểm tra không cần tin.

Tập dữ liệu hiệu năng đầy đủ và tự nhất quán cho phân tích và được dùng dưới đây để trả lời RQ1 và RQ3 tạm thời. Chạy lại cùng script trên chiến dịch ghi trên máy bare-metal thay các số bằng số cấp testbed không thay đổi phân tích.

6.4 Kết quả đánh giá

Kết quả an toàn và migration không phụ thuộc máy đo. Kết quả hiệu năng và tài nguyên phụ thuộc, và Mục 6.1 nêu máy đó mang và không mang gì. Mọi hình và bảng dưới đây được đo trên nó trừ khi nói khác.

Kết quả đã giải quyết và chưa giải quyết được báo cạnh nhau. Nói ngay đầu cái nào là gì ngăn kết quả null bị đọc như không có. Sáu phát hiện đã giải quyết: taxonomy lớp lỗi và ranh giới (Mục 6.4.7), chi phí seam FFI (Mục 6.4.8), khác biệt bộ nhớ giữa hai build (Mục 6.4.6), chi phí thao tác thuần (Mục 6.4.2), tương đương bốn variant ở median dưới bão hòa, và phân kỳ qua nó (Mục 6.4.4). Cái cuối là hiệu ứng ngôn ngữ lớn nhất và tái lập nhất trong nghiên cứu.

Đuôi xa *dưới* bão hòa là kết quả chưa giải quyết duy nhất, và host là lý do. Spread run-to-run ở p99 vượt mọi khác biệt giữa variant, và không số lần lặp trên máy ảo với CPU steal khác không đóng khoảng đó (Mục 6.4.1). Chỉ protocol bare-metal Mục 6.1 sẽ trả lời. Phân kỳ quá tải được giải quyết như phép đo, dù chưa đầy như cơ chế. Mục 6.4.5 kiểm tra hàng đợi có giới hạn và nguyên thủy hoàn tất riêng có giải thích không. Chúng không tái tạo thứ tự full-service ở mức ý nghĩa, để lại việc ở đâu đó trên toàn đường handoff ngoài nguyên thủy cô lập.

6.4.1 Hiệu năng (RQ1, RQ3)

Giả thuyết được thử là H1: hai ngôn ngữ tương đương hiệu năng dưới bão hòa, mọi khác biệt chỉ ở đuôi xa. Mục này báo nơi, nếu có, mỗi bên thắng và bao nhiêu, qua sweep tới và qua bão hòa.

Mọi thứ trong mục được tạo từ log thô bởi `bench/plot_figures.py`: hình, bảng, và số trích trong văn bản. Mục do đó có thể tái tạo nguyên văn nếu chiến dịch rerun trên testbed bare-metal. Số tuyệt đối thuộc host Mục 6.1. Mỗi điểm gộp 20 lần lặp xen kẽ. Giá trị bảng và dải shaded là median với khoảng tin cậy bootstrap 95% nonparametric.

Bảng 6.1 tóm tắt phân phối độ trễ mỗi thao tác ở tải đề nghị bền vững 56 kops/s. Điểm vận hành không chọn bằng mắt: nó là tải sweep cao nhất mà mọi một trong bốn variant build vẫn giữ p99 dưới 5 ms, tức điểm bận nhất mà cả bốn cùng khỏe, làm nó nơi công bằng nhất để so sánh.

Bảng phải đọc ở hai mức tin cậy khác nhau, và tách chúng là cách đọc trung thực của chiến dịch. Ở *median* bốn variant nằm trong 2 μs của nhau, hay 2.1%, và mọi cặp khoảng tin cậy giao nhau. Bảng đơn lẻ do đó không phân tách gì. Đó là câu về khoảng không ghép chứ không về ngôn ngữ. Test ghép dưới đây giải quyết khác biệt nhỏ và nhất quán ở median, và khoảng giữa hai cách đọc chính xác là lý do Mục 4.4 test so sánh thay vì nhìn khoảng.

Trên median tình huống khác loại. Điểm không phải ngôn ngữ khác mà host này không thể nói có khác không. Giá trị p99 khác 293 μs giữa variant, trong khi khoảng tin cậy 95% rộng nhất trên p99 của một variant đơn trải 1480 μs, nhiều lần khác biệt giữa variant. Khi uncertainty về một variant vượt khoảng giữa cả bốn, không thứ tự nào đọc được. Báo một sẽ là báo scheduler. Cùng điều mạnh hơn ở p99.9 và maximum.

Đây là dạng cụ thể của hạn chế Mục 6.1 nêu, và chi phí được giới hạn. Nó không chạm throughput và hành vi bão hòa, kết quả tài nguyên dưới đây, hay gì trong chương RQ2 và RQ4. Nó nghĩa là một nửa RQ1 chỉ nhận câu trả lời phủ định từ chiến dịch này, tức hai ngôn ngữ hành xử thế nào ở percentile cao khi dịch vụ vẫn đáp ứng tải đề nghị. Đuôi đó chưa đo được. Nửa kia không bị chạm, và Mục 6.4.4 giải quyết nó dứt khoát trên cùng host này. Đóng khoảng cần protocol bare-metal, không thêm lần lặp ở đây, vì spread là nhiễu hệ thống chứ không phải nhiễu lấy mẫu. Mục 6.4.2 cũng cho thấy đại lượng ngôn ngữ có thể ảnh hưởng nằm hơn ba bậc magnitude dưới đại lượng phép đo này giải quyết.

Một điều chỉnh về metric thuộc đây chứ không chỉ thảo luận tính hợp lệ. RQ1 nêu median, p99, p99.9, và maximum vì bốn đó là chiến dịch này cung cấp. Harness cũng ghi jitter giữa thao tác và p99.99, và giữ histogram đuôi xa sẽ phải gộp từ, nhưng chiến dịch báo ở đây trước instrumentation đó. Khả năng dự đoán do đó được chứng minh ở p99 và p99.9 và không xa hơn. Đó là khoảng trong bằng chứng chứ không trong thiết kế, và nó đóng với chiến dịch bare-metal mà luận điểm đuôi đã chờ.

Bảng 6.1: Độ trễ mỗi thao tác ở tải đề nghị bền vững 56 kops/s (workload vòng đời RAN mixed, 7 shard). Giá trị microsecond: median qua 20 lần lặp trên khoảng tin cậy bootstrap 95% nonparametric. Khoảng in đầy đủ thay vì nửa rộng đối xứng vì nó rất bất đối xứng ở các percentile này, và nửa rộng đủ che bound trên sẽ đặt bound dưới dưới không.

Variant	p50	p99	p99.9	max
C++ (system alloc)	79.9	4667	6222	7817
	[79.6, 80.7]	[4272, 4895]	[5501, 7242]	[6572, 8886]
C++ (jemalloc)	80.3	4506	5949	7472
	[79.4, 80.7]	[4289, 5216]	[5540, 7160]	[6470, 8826]
Rust (system alloc)	79.0	4530	6013	7932
	[78.7, 79.7]	[4197, 5677]	[5612, 7905]	[6527, 9327]
Rust (jemalloc)	78.6	4375	5624	6951
	[78.1, 79.7]	[4173, 4661]	[5476, 6425]	[6363, 8143]

Test so sánh thay vì nhìn mắt. Khoảng giao nhau là công cụ yếu theo cả hai chiều, nên so sánh cũng được test trực tiếp, theo thủ tục cố định Mục 4.4 và báo

Bảng 6.2: Test ghép hoán vị chính xác C++ so với Rust ở 56 kops/s, với bộ cấp phát giữ cố định. Hiệu ứng là median của 20 hiệu ghép (Rust − C++), hiển thị trên khoảng tin cậy bootstrap 95%. Statistic test là mean của chúng. p chính xác qua mọi 2^{20} gán dấu, và cột cuối áp dụng hiệu chỉnh Holm trên cả họ 14 test. Sàn có thể đạt là 0.0000019 chưa hiệu chỉnh và 0.000027 sau hiệu chỉnh. Các dòng được khối theo tập tiêu chí Mục 4.4: ghép cứu statistic ở khối trên, nên khoảng nhất quán ở đó là bằng chứng, trong khi percentile cao dưới được tính từ đúng thao tác nhiễu rơi vào, nên khoảng loại trừ không được đọc là phát hiện. Throughput đạt không được test, vì lý do cùng mục, và xuất hiện trong Bảng 6.8. Đo trên host Xeon ảo Mục 6.1.

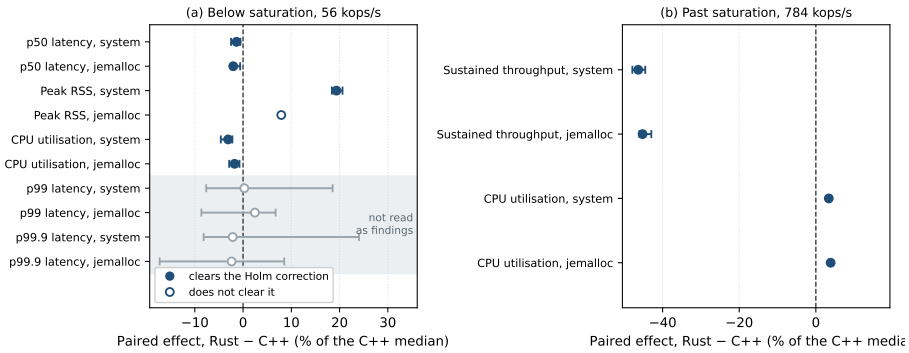
Metric	Allocator	Effect (Rust − C++)	p	Holm
Robust to interference on single operations				
p50 latency (µs)	system	−1.1 [−2.0, −0.4]	0.002	0.012
p50 latency (µs)	jemalloc	−1.6 [−2.0, −0.5]	0.000086	0.00069
Peak RSS (MiB)	system	+1.9 [+1.8, +2.0]	0.0000019	0.000027
Peak RSS (MiB)	jemalloc	+3.4 [+3.3, +3.5]	0.727	1.000
CPU utilisation (%)	system	−3.4 [−5.0, −2.4]	0.0000076	0.000069
CPU utilisation (%)	jemalloc	−1.9 [−3.1, −0.8]	0.005	0.029
Determined by it: high percentiles				
p99 latency (µs)	system	+12 [−356, +866]	0.433	1.000
p99 latency (µs)	jemalloc	+111 [−389, +304]	0.462	1.000
p99.9 latency (µs)	system	−133 [−508, +1495]	0.460	1.000
p99.9 latency (µs)	jemalloc	−141 [−1028, +508]	0.407	1.000

trong Bảng 6.2. Điều cho phép ghép là tính chất của chiến dịch này chứ không của statistic. Bốn variant được xen kẽ trong mỗi lần lặp, nên lần chạy i của mỗi được đo trong cùng block dưới cùng điều kiện môi trường. Mọi nhiễu block mang là chung cho cặp, và hủy trong hiệu của chúng. Đó là thứ hai kỷ luật xen kẽ mua, ngoài lập luận công bằng nó được chọn vì.

Hình 6.2 vẽ cùng họ trên một trục, và ba điều trong đó dễ thấy hơn trong bảng mười bốn dòng: hiệu ứng nào sống sót hiệu chỉnh, hiệu ứng có giữ dấu khi bộ cấp phát đổi không, và khoảng đuôi rộng hơn phần còn lại bao nhiêu.

Năm điều theo sau. Thứ nhất, 9 trong 14 test của họ sống sót hiệu chỉnh Holm. Ở điểm vận hành này chúng là hai dòng median-latency, peak RSS dưới bộ cấp phát hệ thống, và cả hai dòng CPU. Bốn còn lại sống sót là so sánh quá tải Mục 6.4.4. Peak RSS dưới jemalloc là dòng khối trên duy nhất không vượt hiệu chỉnh, dù khoảng xa không, và Mục 6.4.6 nói vì sao.

Thứ hai, quy tắc đọc cố định Mục 4.4 phải áp dụng trước khi diễn giải bất kỳ. 10 khoảng ghép loại trừ không. Trong khối đuôi những loại trừ là: none. Quy tắc được nêu trước, và không ở điểm này, chính xác vì kết quả đó không được đảm bảo. Lượt



Hình 6.2: Mọi test trong Bảng 6.2 và 6.6 dưới dạng hiệu ứng ghép với khoảng bootstrap 95%, chuẩn hóa theo median C++ để microsecond, mebibyte, điểm phần trăm và kops/s chia sẻ trục. Marker đầy là 9 trong 14 test vượt hiệu chỉnh Holm. Hai bộ cấp phát của metric được vẽ cạnh nhau, nên hiệu ứng đổi dấu giữa chúng sẽ hiện là cặp qua đường không. Các dòng shaded là percentile cao, tính từ đúng thao tác nhiễu rơi vào, mà Mục 4.4 cố định trước là dòng có khoảng không đọc là phát hiện. Metric giữ đơn vị riêng trong bảng.

trước bảy lần lặp cùng chiến dịch đã tạo khoảng đuôi loại trừ không, và thêm lần lặp sau đó gỡ chúng. Đó là hành vi quy tắc dự đoán.

Thứ ba, đối chứng bộ cấp phát cung cấp kiểm tra thứ hai, độc lập với thứ nhất. Mọi so sánh chạy hai lần dưới điều kiện không nên đổi hiệu ứng ngôn ngữ, nên hiệu ứng thực sự là ngôn ngữ nên giữ dấu dù liên kết bộ cấp phát nào. Ở đây p50 latency, p99 latency, p99.9 latency, Peak RSS and CPU utilisation tất cả giữ, và 0 đảo. Đối chứng xác nhận thay vì phủ quyết không phải bằng chứng nó không cần. Trong chiến dịch trước, trước khi hai build `jemalloc` bị giữ một cấu hình, hiệu ứng CPU vượt hiệu chỉnh dưới một bộ cấp phát và đảo dấu dưới bộ kia, và được báo chưa giải quyết theo chính quy tắc này. Đối chứng kiếm chỗ bằng điều nó sẽ bắt, không phải điều nó tình cờ bắt.

Thứ tư, 5 dòng hiệu chỉnh không vượt không nói cùng một điều, và Bảng 6.3 là thứ tách chúng. Test không giải quyết khác biệt vẫn giới hạn nó: có margin chặt nhất trong đó hai variant có thể gọi tương đương, và đó là tính chất dữ liệu chứ không phán quyết. Đọc vậy khối trên và khối đuôi tách rõ. Ở median bound là 2.0 μs, nên khác biệt test giải quyết cũng nhỏ theo tuyệt đối và hai cách đọc đồng ý. Trong khối đuôi bound chạy tới 1298 μs, là khả năng host này không nhìn thấy được nêu dưới dạng số thay vì null. Phân biệt quan trọng vì hai thứ thường được báo giống. Null với bound chặt là kết quả. Null với bound rộng hơn mọi khác biệt triển khai quan tâm là chiến dịch từ chối trả lời, và nói cái nào là điều tối thiểu bảng này làm cho nửa RQ1 host không thể giải quyết.

Bảng 6.3: Mỗi hiệu ứng có thể lớn bao nhiêu. Với mọi so sánh ở 56 kops/s, hiệu ứng quan sát so với margin chặt nhất trong đó hai variant có thể gọi tương đương, và phán quyết Bảng 6.2 đạt. Hai câu hỏi cố ý giữ ở hai bảng: một hỏi khác biệt có ở đó không, bảng này hỏi nó có thể lớn bao nhiêu nếu có, và một dòng có thể quan trọng ở một mà không ở bảng kia. Margin độ trễ là microsecond, peak RSS mebibyte, sử dụng CPU điểm phần trăm.

Metric	Allocator	|Effect|	Equivalent within	Verdict
p50 latency (µs)	system	1.1	2.0	significant
p50 latency (µs)	jemalloc	1.6	1.8	significant
p99 latency (µs)	system	12	662	not resolved
p99 latency (µs)	jemalloc	111	252	not resolved
p99.9 latency (µs)	system	133	1298	not resolved
p99.9 latency (µs)	jemalloc	141	692	not resolved
Peak RSS (MiB)	system	1.9	2.0	significant
Peak RSS (MiB)	jemalloc	3.4	3.5	not resolved
CPU utilisation (%)	system	3.4	4.8	significant
CPU utilisation (%)	jemalloc	1.9	2.9	significant

Thứ năm, median giờ là khác biệt đã giải quyết chứ không phải null: 1.6 μs theo hướng có lợi Rust, nhất quán dưới cả hai bộ cấp phát. Khoảng trên nó chạm 2 μs so với median khoảng 80 μs, và cám dỗ là báo đó là bound trên hiệu ứng ngôn ngữ. Cách đọc đó sai, và mục sau giải thích vì sao.

6.4.2 End-to-end số được ghép từ gì

Mọi độ trễ sweep báo là vòng quay end-to-end, và Mục 4.2.1 nêu vì sao đó gói hai chi phí khác loại. Một là thao tác, thực thi trong một luồng và do compiler tạo. Cái kia là handoff gửi-và-hoàn tất qua luồng, tính cho hệ điều hành. Chỉ cái đầu có thể khác giữa ngôn ngữ, và sweep không tách hai.

Điều quan trọng cho bound vừa trích, vì bound chỉ nói bằng đại lượng nó là phần của. Cả hai chi phí do đó được đo trực tiếp, bằng phương pháp hai cấp mô tả ở đó, ở cả hai ngôn ngữ và ở số shard và quần ngữ cảnh của sweep. `ShardCore` gọi không có hàng đợi trên đường cho thao tác. `Service` cho thao tác cộng handoff. Hiệu của chúng là handoff. Bảng 6.4 cho kết quả, và nó định hình lại toàn so sánh hiệu năng.

Bảng 6.4: Chi phí end-to-end thao tác thực sự đi đâu, đo ở 7 shard và 5 000 UE hoạt động với một caller không tranh chấp. Mỗi dòng là median qua 15 lần lặp xen kẽ trên khoảng tin cậy bootstrap 95%, theo đơn vị magnitude gọi: hai thành phần khác hơn hai bậc magnitude, điểm của bảng. p là test ghép hoán vị hai phía chính xác C++ so với Rust trên hiệu theo lần lặp. Handoff có bằng hiệu hai dòng đo, nên mang uncertainty của cả hai.

Thành phần	C++	Rust	p
Operation alone, no queue in the path (ns)	34.5 [32.4, 35.2]	21.3 [21.0, 21.5]	0.000061
Cross-thread submit-and-complete handoff (µs)	13.58 [13.38, 13.83]	13.59 [12.69, 14.52]	0.439
End-to-end round-trip, uncontended (µs)	13.62 [13.41, 13.87]	13.61 [12.71, 14.54]	0.421
Operation's share of the round-trip	0.250 %	0.160 %	

Thao tác là 34.5 ns trong C++ và 21.3 ns trong Rust. Handoff là 13.58 μs và 13.59 μs. Việc ngôn ngữ thực sự làm chiếm tối đa 0.250% của sweep đo. Phần bù 99.750% của mọi độ trễ Bảng 6.1 là hệ điều hành di chuyển thông điệp giữa hai luồng.

Hình dạng đó không đặc thù hệ này hay cặp ngôn ngữ này. Zhou et al. xem lại phân rã kinh điển chi phí xử lý giao dịch trên database shared-nothing, xây như dịch vụ này với một partition mỗi worker thread. Sau khi việc mỗi thao tác được tối ưu, họ báo phần còn lại chủ yếu là giao tiếp qua ranh giới cô lập chứ không phải thao tác [65]. Bảng trên là cùng mẫu trong dịch vụ control-plane, làm kết quả dưới đây tính chất chung của thiết kế chứ không artefact của phép đo.

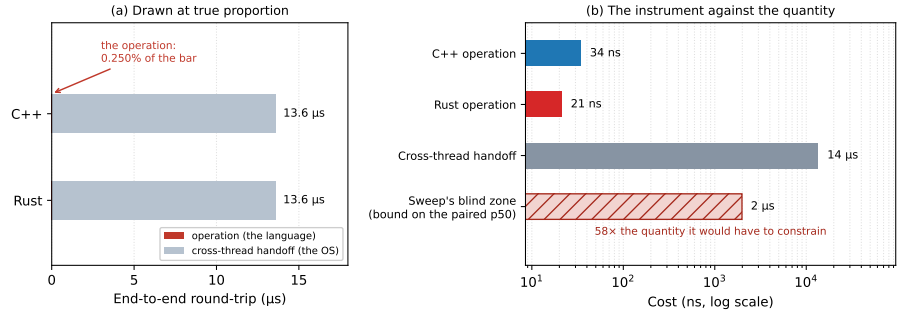
Hai kết luận theo sau, và chúng kéo ngược chiều.

Thứ nhất, sweep không thể bound hiệu ứng ngôn ngữ, và bound trên không được đọc như nó bound. Khoảng 2 μs trên hiệu median ghép là 58 lần chi phí riêng của thao tác. Hiệu ứng ngôn ngữ có thể vài trăm phần trăm việc đang làm và vẫn nằm gọn trong nó. Sweep thiết lập yếu hơn và vẫn hữu ích: không ngôn ngữ làm rối vòng quay do scheduler chi phối, điều triển khai hình dạng này thực sự trải nghiệm. Nó không thể thiết lập gì về mã compiler phát ra. Giới hạn đó đến từ cách phép đo được xây chứ không từ host, nên không như kết quả đuôi sẽ không sửa bằng chuyển bare metal. Nó được sửa bằng đo thành phần riêng, điều mục này làm.

Thứ hai, một khi thành phần *được* đo riêng, ngôn ngữ không tương đương. Rust chạy thao tác thuần nhanh hơn khoảng 1.6×, khác biệt xuất hiện ở đại đa số lần lặp, và test ghép đặt nó hai bậc magnitude dưới ngưỡng thông thường. Phân rã này là thí nghiệm riêng so với sweep và không phần của họ Holm. Nó chạy 15 lần lặp thay vì 20, nên sàn có thể đạt riêng là 0.000061 chứ không phải 0.000027 trích Mục 4.4, và không hiệu chỉnh áp dụng cho nó. Cùng điều cho so sánh interoperability Mục 6.4.8. Đây là nơi duy nhất trong cả chiến dịch hiệu năng mà khác biệt ngôn ngữ trong *thời gian* được giải quyết, và sweep mù với nó hơn hai bậc magnitude.

Hình 6.3 vẽ cả hai kết luận trên một trang, vì mỗi cái dễ thấy hơn đọc từ bảng số trải bốn decade. Panel thứ hai là lập luận mục này: công cụ nằm hoàn toàn trên đại lượng nó phải ràng buộc.

Gán nhận cần cẩn thận, vì cách đọc tự nhiên là sai. Không gì ở đây liên quan borrow checker, kiểm tra bound, hay ownership. Nguồn có thể nhất đã ghi trong log điểm bắt buộc Bảng 5.2. Container mỗi shard là hash map chuẩn của mỗi ngôn ngữ, và chúng là cấu trúc dữ liệu khác. `std::unordered_map` của C++ phải node-based để giữ cam kết ổn định iterator và reference, nên lookup đuổi pointer từ mảng bucket ra node cấp phát riêng. `HashMap` của Rust là bảng open-addressed giữ entry inline và thường giải quyết lookup trong một cache line. Hàm hash được ghim FNV-1a cả hai bên (Mục 5.4) chính xác để hashing không thể là giải thích, để lại bố cục container. Đó là khác biệt thật và có thể báo giữa hai ngôn ngữ như thực tế dùng, vì developer chọn hash map chuẩn nhận cái ngôn ngữ họ ship. Không phải bằng chứng về mô hình an toàn Rust và không nên trích như vậy. Xác nhận cơ chế cần counter cache-miss Mục 7.6 liệt kê là khoảng instrumentation rõ nhất.



Hình 6.3: Chi phí end-to-end thao tác đi đâu, và vì sao sweep không bound hiệu ứng ngôn ngữ. **(a)** Vòng quay theo tỷ lệ thật: thao tác ngôn ngữ thực hiện là 0.250% thanh, phần còn lại handoff qua luồng. **(b)** Cùng đại lượng trên trục log so với hiệu ghép nhỏ nhất sweep có thể giải quyết, hiển thị là vùng mù. Cả hai phép đo thao tác nằm trong nó bằng hệ số 58, và (b) cũng là nơi khác biệt thời gian duy nhất chiến dịch giải quyết nằm.

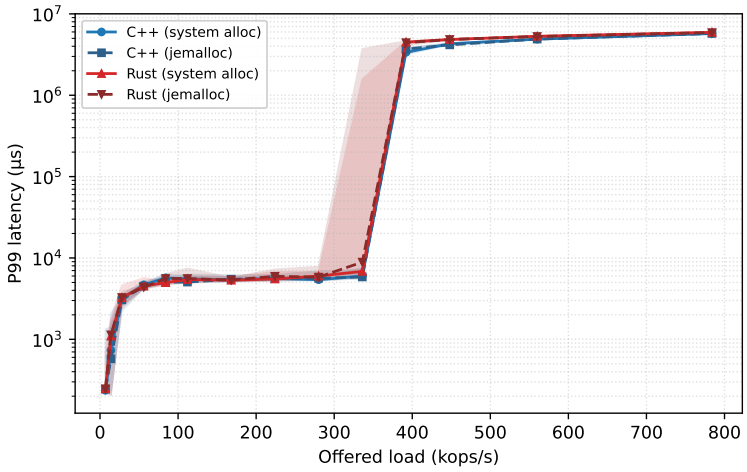
Ba giới hạn giữ phát hiện đúng kích thước. Đó là microbenchmark một luồng, không tranh chấp, nên nói thao tác tốn bao nhiêu chứ không hệ tải làm gì với nó. Đo trên cùng host sweep, nên số tuyệt đối mang cùng cảnh báo, dù tỷ số ít nhạy hơn mỗi thành phần. Và đó là một thao tác trên một cấu trúc dữ liệu. Tổng quát hóa thành "Rust nhanh 1.6× hơn" sẽ lặp chính lỗi luận văn này chỉ trích trong literature trường hợp trung bình.

6.4.3 Độ trễ, throughput và bão hòa qua sweep

Hình 6.4 vẽ p99 độ trễ so với tải đề nghị, và hình dạng là hình Hình 3.2 phác. Qua vùng tracking bốn variant leo cùng nhau và nhẹ, từ khoảng 240 μs ở tải nhẹ tới single-digit millisecond bởi 335 kops/s. Dải tin cậy ở đó đủ rộng để không curve tách khỏi curve khác ở vùng đó. Sau đó, trong một bước duy nhất của thang tốc độ tiếp theo, p99 mọi variant tăng hơn hai bậc magnitude cùng lúc, từ khoảng 6 ms tới vài giây. Đó là vách quá tải, và mức đột ngột đến là phát hiện. Một bước tăng tải đề nghị tách dịch vụ đáp ứng deadline millisecond khỏi dịch vụ lỡ ba bậc magnitude. Thiết kế với hàng đợi chặn có giới hạn không có suy giảm nhẹ nhàng ở giữa.

Hai tính chất phía xa vách giới hạn ý nghĩa số, và chúng là lý do Mục 4.4 không test percentile ở đó. Độ trễ qua knee là backlog lịch tích chứ không phải thời gian phục vụ. Và vì giai đoạn đo bị cắt ở chiều rộng cố định, percentile báo cáo bão hòa gần chiều rộng đó, làm mọi percentile trên knee là bound dưới. Điều còn diễn giải được trong regime này là mỗi variant vẫn hoàn tất bao nhiêu việc, và Mục 6.4.4 nói tiếp.

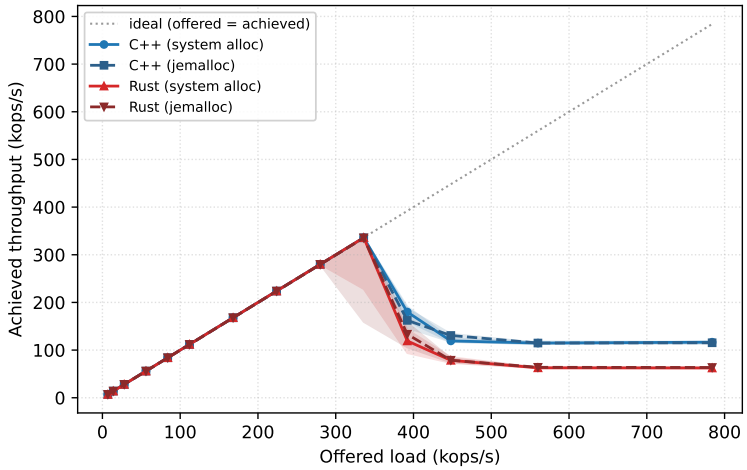
Hình 6.5 cho throughput đạt so với tải đề nghị, và là plot thông tin hơn. Throughput là trung bình cả lần chạy, làm nó ít nhạy hơn percentile với nhiễu rơi vào thao tác riêng lẻ. Mọi variant bám đường lý tưởng đề nghị-bằng-đạt tới 336 kops/s và bão hòa khoảng 335 kops/s. Hai điều theo ngay. Thứ nhất, throughput đạt khớp tải đề nghị dưới knee trong 0.1%, nên điểm vận hành Bảng 6.1 là tải *bền vững* thật chứ không danh nghĩa. Độ trễ báo ở đó không phải của hệ đã xếp hàng. Thứ hai, knee nằm cùng chỗ cho cả bốn variant, điều cấu trúc một luồng mỗi shard dự đoán. Nơi



Hình 6.4: Độ trễ đuôi (p99) so với tải đề nghị cho bốn variant build. Đường là median qua 20 lần lặp. Dải shaded là CI bootstrap 95%. Trục y log.

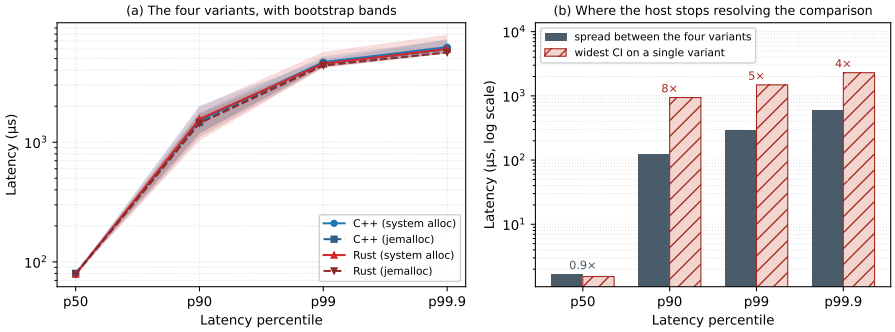
thiết kế không còn chịu là tính chất thiết kế.

Điều xảy ra *sau* knee không được chia sẻ. Bốn curve tách ngay khi tốc độ đề nghị không còn đáp ứng, và tách theo ngôn ngữ chứ không theo bộ cấp phát. Mục 6.4.4 đo phân tách đó thay vì để mắt.



Hình 6.5: Throughput đạt so với tải đề nghị. Đường chấm đánh dấu regime lý tưởng đề nghị-bằng-đạt. Throughput được đếm chỉ trong giai đoạn đo, nên warm-up không thiên cả mẫu số và tử số.

Hình 6.6 giữ tải cố định và sweep percentile. Bốn variant trùng ở median và tỏ ra khi percentile tăng. Đó là hình dạng mong đợi cho phân phối heavy-tailed, và là lý do methodology nhận biết percentile dùng xuyên suốt. Tỏ ra không phải hiệu ứng ngôn ngữ, và panel phải cho thấy vì sao. Nó đặt spread giữa bốn variant ở mỗi percentile so với khoảng rộng nhất một variant mang ở đó. Ở median spread là lớn hơn của hai, regime trong đó thứ tự có nghĩa. Từ p90 trở lên khoảng vượt spread nhiều lần. Sự tỏ ra do đó đánh dấu nơi phép đo mất độ phân giải, không nơi ngôn ngữ khác.



Hình 6.6: Độ trễ mỗi thao tác qua dải percentile ở 56 kops/s. **(a)** Bốn variant là median qua 20 lần lặp, mỗi cái với dải bootstrap 95%. Dải là điểm: vẽ đường trần các curve chồng lên và đọc như tuyên bố tương đương, không phải điều phép đo không thể tách đã chứng minh. **(b)** Vì sao không thể tách trên median. Spread giữa bốn variant được đặt so với khoảng tin cậy rộng nhất trên bất kỳ một variant, với tỷ số chú thích trên mỗi cặp thanh. Khi tỷ số vượt một, uncertainty về một variant lớn hơn khoảng giữa cả bốn và không thứ tự nào đọc được.

6.4.4 Hành vi qua bão hòa (RQ1)

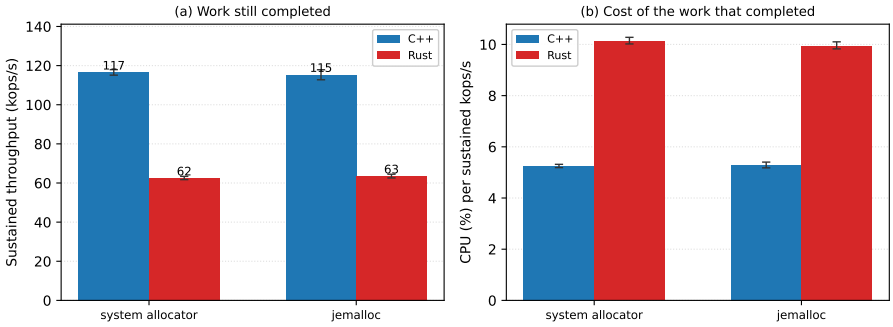
RQ1 hỏi hai ngôn ngữ so với nhau thế nào khi tải tăng *và quá tải*, và mọi thứ báo trước đó ở dưới knee. Mục này là nửa kia. Điểm vận hành là 784 kops/s đề nghị, định nghĩa là tải sweep sâu nhất mà mọi variant đã ngừng đáp ứng tốc độ đề nghị, và metric là hai cái còn diễn giải được ở đó (Mục 4.4). Bảng 6.5 cho mỗi variant vẫn duy trì gì và Bảng 6.6 test so sánh.

Ngôn ngữ không tương đương trong regime này, và margin không nhỏ. C++ duy trì 116 kops/s trong khi Rust duy trì 63, hệ số 1.8. Hiệu ghép ít nhất 50 kops/s dưới cả hai bộ cấp phát, giữ trong 20/20 lần lặp, và p Holm của nó ở sàn hiệu chỉnh của cả họ 14 test. Đây là hiệu ứng ngôn ngữ lớn nhất và tái lập nhất trong nghiên cứu, và là cái duy nhất nói trực tiếp câu hỏi RQ1 được viết để hỏi.

Cột CPU là điều làm kết quả diễn giải được chứ không chỉ lớn, vì nó loại cách đọc tự nhiên nếu không. Variant hoàn tất ít việc hơn có thể đơn giản làm ít hơn: bị throttle, chặn, rảnh. Không phải vậy. Rust dùng *nhiều* CPU tổng hơn C++ ở đây, tái lập và dưới cả hai bộ cấp phát, đã đảo dấu nó mang ở tải đại diện Bảng 6.2. Chia qua việc thực sự hoàn tất, chi phí CPU mỗi thao tác duy trì là 1.9 lần bên kia. Không variant gần capacity máy, vì cả hai ở khoảng sáu trong mười bốn lõi ghim. Gap do đó không phải giới hạn capacity và không phải rảnh. Đó là effort tiêu không hoàn tất thao tác, dấu hiệu đường đồng bộ quay, hoặc park và wake lặp, nơi bên kia chặn một lần và ngủ.

Hình 6.7 đặt cả hai nửa cách đọc cạnh nhau, và nhóm mang thứ ba. Phân tách gần như hoàn toàn trong nhóm bộ cấp phát, nghĩa hiệu ứng theo biến nghiên cứu thao tác chứ không biến đối chứng.

Effort đi đâu được thu hẹp bằng phép đo chứ không để mở. Qua bão hòa producer tiêu gần hết thời gian ở hai chỗ ghi trong Bảng 5.2: hàng đợi chặn có giới hạn và tay cầm hoàn tất. Mục 6.4.5 cô lập các nguyên thủy đó, và chúng không tái tạo thứ



Hình 6.7: Hành vi ở 784 kops/s đề nghị, xa quá knee 335 kops/s. Thanh là median qua 20 lần lặp với khoảng bootstrap 95%, nhóm sao cho hai thanh trong nhóm chỉ khác ngôn ngữ. **(a)** Mỗi variant vẫn hoàn tất: C++ duy trì hệ số 1.8 hơn Rust. **(b)** Tổng CPU chia cho việc thực sự hoàn tất, điều loại cách đọc variant giao ít hơn đơn giản làm ít hơn. Nó tiêu 1.9 lần CPU mỗi thao tác duy trì, và số CPU được cộng qua luồng.

tự full-service ở mức ý nghĩa. Điều đó loại chúng *riêng lẻ* làm toàn bộ giải thích cho gap 1.8×. Như Mục 6.4.2, không gì trong borrow checker, kiểm tra bound, hay ownership bị liên quan, và phát hiện không nên trích như là.

Ba điều lại giới hạn phát hiện cho phép gì. Đó là một thiết kế, dịch vụ shared-nothing với một yêu cần outstanding mỗi producer và giới hạn hàng đợi 1 024, và độ sâu sụp đổ phụ thuộc cả ba. Đo với harness có producer chặn mỗi yêu cầu, nên nó mô tả client hình dạng này trải nghiệm chứ không client pipeline. Và số tuyệt đối thuộc host này. Điều đi theo là thứ tự và nguyên nhân. Dưới bão hòa lựa chọn ngôn ngữ không quan trọng. Trên nó nguyên thủy đồng bộ thư viện chuẩn quan trọng, và crossover đủ sắc để so sánh dừng ở tải danh nghĩa sẽ kết luận ngược.

Crossover cũng liên quan công trình trước gần nhất, báo suy giảm tương đương qua ngưỡng tốc độ yêu cầu và gán cho runtime async [55]. Mục 7.2 nói ý nghĩa khi cùng hình dạng xuất hiện ở đây không có runtime như vậy ở cả hai bên.

Bảng 6.5: Mỗi variant vẫn duy trì ở 784 kops/s đề nghị, xa quá knee 335 kops/s: median qua 20 lần lặp trên khoảng tin cậy bootstrap 95%. Dưới bão hòa throughput đạt bị ghim tốc độ đề nghị theo cấu trúc và không thể nói gì về ngôn ngữ. Ở đây đó là thứ duy nhất có thể. Sử dụng CPU vượt 100% vì được cộng qua luồng, nên 600% là sáu lõi bận trong mười bốn ghim.

Variant	sustained throughput (kops/s)	CPU (%)
C++ (system alloc)	116.6	611.6
	[115.1, 117.9]	[610.0, 612.2]
C++ (jemalloc)	115.4	609.4
	[112.8, 117.9]	[607.8, 611.9]
Rust (system alloc)	62.4	632.7
	[61.7, 63.4]	[630.8, 634.2]
Rust (jemalloc)	63.5	633.1
	[62.6, 64.5]	[631.6, 634.5]

Bảng 6.6: Test ghép hoán vị chính xác C++ so với Rust ở 784 kops/s, bộ cấp phát giữ cố định, hiệu chỉnh Holm trên cùng họ 14 test như Bảng 6.2 chứ không là họ riêng. "Repeats" đếm bao nhiêu trong 20 hiệu ghép đồng ý dấu median, điều statistic tính từ mean không thể hiện. Không percentile độ trễ được test ở đây, vì lý do cố định trước Mục 4.4.

Metric	Allocator	Effect (Rust − C++)	Repeats	p	Holm
Sustained throughput (kops/s)	system	−54.0 [−55.8, −51.9]	20/20	0.0000019	0.000027
Sustained throughput (kops/s)	jemalloc	−52.2 [−53.1, −49.6]	20/20	0.0000019	0.000027
CPU utilisation (%)	system	+20.6 [+19.1, +23.3]	20/20	0.0000019	0.000027
CPU utilisation (%)	jemalloc	+23.6 [+20.8, +25.2]	20/20	0.0000019	0.000027

6.4.5 Hàng đợi tranh chấp và các primitive hoàn thành (RQ1)

Kết quả quá tải quy cho khoảng cách hai cấu trúc đồng bộ idiom bắt buộc, nhưng sự quy kết đó được lập luận từ hình dạng đo toàn dịch vụ. Tiểu mục này cô lập riêng các cấu trúc đó.

7 luồng producer đập vào một hàng đợi có giới hạn dung lượng 1 024 trong khi một consumer duy nhất rút khỏi đó. Mỗi producer gửi một job không làm gì mang tay cầm hoàn thành một lần và chặn cho đến khi consumer báo hiệu. Không có công việc `ShardCore` nào chạy trên đường đi. C++ dùng `BoundedQueue` viết tay của Mục 5.4 với `std::promise/std::future`. Rust dùng `sync_channel` và một `mpsc::channel()` mới mỗi yêu cầu. Mỗi cửa sổ đo kéo dài hai giây sau khởi động nóng. 20 lần lặp xen kẽ được tóm tắt bằng trung vị với khoảng bootstrap và kiểm định bằng thủ tục hoán vị ghép cặp chính xác của Mục 4.4. Bảng 6.7 báo cáo kết quả.

Bảng 6.7: Thông lượng qua hàng đợi có giới hạn và các primitive hoàn thành riêng, với 7 producer tranh chấp và dung lượng hàng đợi 1 024. Giá trị là trung vị trên 20 lần lặp xen kẽ phía trên khoảng tin cậy bootstrap 95%. Kiểm định ghép cặp so C++ với Rust trên thông lượng bền vững (kops/s). Đo trên máy chủ của Mục 6.1.

Ngôn ngữ	Thông lượng (kops/s)	Trung bình ns/op	p ghép cặp
C++	55.0 [53.7, 58.4]	18178 [17110, 18632]	—
Rust	58.7 [56.8, 61.8]	17045 [16169, 17594]	0.226

Các primitive cô lập không tái tạo thứ tự quá tải. C++ duy trì 55 kops/s và Rust 59 kops/s, khoảng cách chỉ 1.1× với $p = 0.226$. Toàn dịch vụ ở 784 kops/s tách 1.8×, với p đã hiệu chỉnh ở sàn của họ. Cách đọc trung thực do đó là sự phân kỳ quá tải gồm thêm công việc mà microbenchmark loại bỏ: `Request` gắn thẻ lớn hơn phía C++, điều phối variant của worker, và các thao tác bản đồ vẫn chạy trong chế độ quá tải. Điều kết quả vẫn thiết lập hữu ích. Khác biệt bậc độ lớn chỉ ở primitive bị

loại trừ, và tìm cơ chế chuyển sang phần còn lại của đường bàn giao chứ không sang mô hình an toàn.

6.4.6 Sử dụng tài nguyên (RQ3)

Bảng 6.8 báo cáo sử dụng tài nguyên ở cùng tải bền vững. Peak resident set size tách bốn biến thể rõ ràng, và tách vậy dù đọc họ theo cách nào. Khoảng của nó loại trừ không, và hiệu ứng giữ dấu dưới cả hai bộ cấp phát. Hình 6.8 vẽ cùng phép đo. Panel giữa là nơi phát hiện của mục này nằm, với hai hiệu ứng bên trong peak RSS vẽ thành khác biệt ghép cặp trên một trục chung.

Hai hiệu ứng có trong đó, không phải một, và chúng không cùng cỡ. Liên kết `jemalloc` tốn 32.9–34.4 MiB ở cả hai ngôn ngữ. Bản build Rust tốn thêm 1.9–3.4 MiB so với bản build C++ dưới cả hai bộ cấp phát, với khoảng bootstrap 1.8 đến 3.5 MiB. Sử dụng tài nguyên theo cả hai, và hiệu ứng bộ cấp phát lớn hơn một bậc độ lớn. Thứ tự đó phần nào đến từ kiểm soát công bằng chứ không phải sự thật chung về `jemalloc`. Cả hai build ghim số arena giống nhau, để mặc định của không ngôn ngữ nào thành lợi thế tình cờ, và ghim cao là điều làm metadata arena thành chi phí cố định lớn hai bên. Triển khai tinh chỉnh `jemalloc` cho dấu chân thay vì so sánh công bằng sẽ không trả chi phí đó.

Hàng duy nhất trong khối trên của Bảng 6.2 không vượt hiệu chỉnh là peak RSS dưới `jemalloc`, và lý do nằm ở thống kê chứ không phải phần mềm. Khoảng ở đó chặt, xa không, và 18 trong 20 lần lặp ghép cặp đồng ý. Điều đánh bại kiểm định là vài lần chạy báo resident set thấp hơn trung vị biến thể khoảng 30 MiB trong khi vẫn không phân biệt được về thông lượng, thời gian tường và số lỗi. Thống kê hoán vị là trung bình khác biệt ghép cặp, và một lần chạy như vậy chi phối nó. Đây là trường hợp quy tắc đọc của Mục 4.4 được viết trước để bao phủ, và Mục 7.6 tiếp nhận cả quy tắc và khuyết điểm đo đằng sau nó.

Sử dụng CPU hành xử khác cả hai, và khác biệt phụ thuộc chế độ chứ không phải nhiễu. Ở điểm vận hành dưới bão hoà này Rust tiêu thụ CPU *ít* hơn một chút, tối đa 3.4 điểm phần trăm, nhất quán dưới cả hai bộ cấp phát và vượt hiệu chỉnh dưới cả hai. Mục 6.4.4 thấy dấu đảo qua bão hoà và biên độ lớn hơn nhiều. Thông lượng đạt được không thêm gì ở đây, vì lý do Mục 4.4 nêu. Ở điểm dưới bão hoà nó bị ghim vào tải đề nghị, và chỉ báo cáo để xác nhận điểm được duy trì chứ không chỉ danh nghĩa, và nó vậy trong 0.1%.

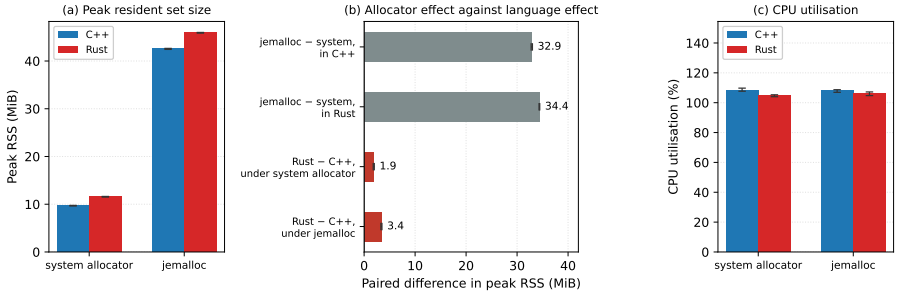
Hai điều kiện giữ phát hiện bộ nhớ đúng cỡ thực tế. Đó là khác biệt chi phí cố định chứ không phải chi phí mỗi ngữ cảnh. Ở 5 000 UE hoạt động dùng trong quét, khác biệt bố cục tám byte đo ở Mục 5.4.1 chiếm khoảng 40 kB, thiếu hai bậc độ lớn so với khoảng cách. Phần lớn khoảng cách do đó là overhead runtime và arena bộ cấp phát, không tỷ lệ với khối lượng công việc và sẽ quan trọng tỷ lệ ít hơn trong triển khai lớn hơn, không phải nhiều hơn. Điều kiện thứ hai là phép đo thiết lập khác biệt mà không giải thích. Không thu bộ đếm phần cứng hay hồ sơ cấp phát, nên phần chia giữa dự trữ arena, bộ nhớ đệm mỗi luồng và dấu chân tĩnh không được giải quyết ở

đây. Cả hai điểm được xem lại ở Chương 7.

Phát hiện RQ3 do đó không phải "không phạt tài nguyên" phẳng mà khoảng chồng lấn gợi ý, cũng không phải phạt hiệu năng chung. Rust tốn thêm vài megabyte dấu chân cố định, tái lập được và dưới cả hai bộ cấp phát, nhưng nhỏ hơn một bậc độ lớn so với chính lựa chọn bộ cấp phát. Nó rẻ hơn một chút về CPU khi dịch vụ còn chịu tải, và đắt hơn đáng kể khi không còn. Trên thao tác trần của Mục 6.4.2 nó là ngôn ngữ nhanh hơn trong hai.

Bảng 6.8: Sử dụng tài nguyên ở tải đề nghị bền vững 56 kops/s: thông lượng đạt được, peak RSS và tổng sử dụng CPU. Trung vị trên 20 lần lặp phía trên khoảng tin cậy bootstrap 95%. Peak RSS là chỉ số duy nhất ở đây mà khoảng tách bốn biến thể rõ ràng.

Biến thể	thông lượng (kops/s)	peak RSS (MiB)	CPU (%)
C++ (system alloc)	55.99	9.7	108.5
	[55.99, 55.99]	[9.6, 9.8]	[107.7, 109.8]
C++ (jemalloc)	55.94	42.6	108.0
	[55.93, 55.95]	[42.4, 42.6]	[106.9, 108.7]
Rust (system alloc)	55.99	11.6	105.0
	[55.99, 55.99]	[11.5, 11.6]	[104.0, 105.3]
Rust (jemalloc)	55.94	45.9	106.3
	[55.94, 55.95]	[45.8, 46.0]	[104.8, 107.2]



Hình 6.8: Sử dụng tài nguyên ở tải đề nghị bền vững 56 kops/s, nhóm theo bộ cấp phát để hai cột trong nhóm chỉ khác ngôn ngữ và hai nhóm chỉ khác bộ cấp phát. Trung vị trên 20 lần lặp với khoảng bootstrap 95%. **(a)** Kích thước resident set đỉnh. **(b)** Hai hiệu ứng nó chứa, mỗi cái là khác biệt ghép cặp trên cùng lần lặp: liên kết `jemalloc` tốn 32.9–34.4 MiB, trong khi bản build Rust tốn thêm 1.9–3.4 MiB so với bản C++. Cả hai đều thật, và trên trục chung hiệu ứng bộ cấp phát lớn hơn một bậc độ lớn. **(c)** Sử dụng CPU, cộng qua các luồng, nơi khác biệt chạy chiều ngược và nhỏ.

6.4.7 An toàn qua tiêm lỗi (RQ2)

Nghiên cứu này không tuyên bố Rust không có lỗi. Nó tiêm danh mục 14 lỗi được nêu trong Bảng 3.1, mỗi lỗi là chương trình nhỏ tự chứa có hương miền của phân hệ để ánh xạ lên lỗi hợp lý trong dịch vụ. Sự chia thành hai nhóm là điều phương pháp của Mục 4.2.2 bật: bảy lớp từ bên trong vùng Rust tuyên bố quản lý, và bảy chọn chính xác vì Rust an toàn *không* được kỳ vọng ngăn chúng. Không có nhóm thứ hai kết quả sẽ do lựa chọn ca kiểm thử quyết định, xác nhận kết quả kỳ vọng và không bao giờ làm lung lay nó. Với nhóm đó, nghiên cứu đo một biên thay vì chứng minh một tuyên bố. Bộ chạy lỗi giữ kỳ vọng ghi trước cho mỗi ca và báo vi phạm theo

cả hai chiều. Một ca biên dịch bất ngờ, hoặc không biên dịch bất ngờ, do đó nổi lên thay vì bị hấp thụ vào kết luận.

Thủ tục. Với mỗi lỗi C++ nghiên cứu build và chạy chương trình dưới AddressSanitizer (ASan), ThreadSanitizer (TSan) và UndefinedBehaviorSanitizer (UBSan), và ghi công cụ nào, nếu có, phát hiện nó. Công cụ thứ ba không phải tinh chỉnh hai cái kia mà là trục riêng. Tràn số có dấu vô hình với cả ASan và TSan, nên không có cột riêng lớp tràn số nguyên sẽ được ghi là không phát hiện ở C++ trong khi thực tế bị bắt. Không chạy phân tích tĩnh, vì lý do Mục 4.2.2 nêu. Đầu ra của nó không phải từ chối biên dịch cũng không phải quan sát thực thi, nên không thuộc ba kết quả phân loại ghi. Cột C++ của Bảng 6.9 do đó báo cáo công cụ nghiên cứu này chạy, không phải trần công cụ C++ có thể đạt. Cấu trúc khớp được viết bằng Rust an toàn và biên dịch. Nơi biên dịch, nó được chạy dưới timeout có giới hạn và hành vi quan sát được ghi, vì "biên dịch được" một mình không thiết lập liệu lỗi sống sót vào chương trình đang chạy. Danh mục, bộ chạy, và nhật ký công cụ thô được công bố cùng luận văn.

Kết quả. Kết quả trong Bảng 6.9, đo bằng `g++ 13.3` (`-std=c++23`, `-O0` để optimizer không thể loại bỏ lỗi tiêm) và `rustc 1.97` (edition 2021). Chuỗi công cụ C++ đó là của máy phát triển cục bộ chứ không phải `g++ 15.1.1` của máy đo, và danh mục được chạy ở đó có chủ ý. Liệu chương trình biên dịch, và bộ dò lỗi báo gì khi chạy, là thuộc tính của chuỗi công cụ chứ không của máy chủ, đó là nội dung cụ thể của việc gọi kết quả này không nhạy môi trường ở Mục 6.1.

Trong phạm vi của nó kết quả không mơ hồ. Cả bảy lỗi an toàn bộ nhớ và tranh chấp dữ liệu đều là lỗi biên dịch trong Rust an toàn, nên không cái nào có thể viết cả. Ở C++ mỗi lỗi là mã hợp lệ mà lỗi chỉ lộ lúc runtime, và chỉ nếu bài kiểm thử thực sự đi qua đường lỗi. ASan bắt lỗi an toàn bộ nhớ. Lỗi đồng thời chỉ bị TSan bắt và vô hình với ASan. Tràn trong nhóm biên chỉ bị UBSan bắt và vô hình với cả hai. Không có bộ dò lỗi C++ đơn lẻ bao phủ danh mục, và ba cái vẫn để 4 trong 14 lớp không chạm, với lớp thứ năm chỉ báo cáo gián đoạn. Phát hiện runtime cũng có nghĩa build nhị phân có công cụ riêng và trả overhead thời gian và bộ nhớ của bộ dò lỗi, trong khi lỗi Rust được báo trên mọi build thường bất kể độ phủ kiểm thử. Mã trong ngoặc là chẩn đoán `rustc` kích hoạt. `E0499` là mượn mutable xung đột, `E0502` mượn mutable khi mượn shared còn sống, `E0382` dùng giá trị đã move, và `E0373` closure sống quá dữ liệu mượn khi thoát sang luồng khác.

Hai hàng trong nhóm đó nói điều năm hàng kia không thể, và cả hai được thêm để kiểm lập luận chi phí chứ không lặp lại. Use-after-move là lớp trong bảo đảm duy nhất mà không bộ dò lỗi C++ nào báo cáo. Admission trả lời yêu cầu từ ngữ cảnh vừa move vào store, và đối tượng đã move hợp lệ nhưng không xác định, nên không gì bị hỏng, không gì undefined, và cả ba công cụ chạy sạch trong khi xác nhận đi ra dưới danh tính rỗng. Mọi lớp khác trong nhóm này bị ít nhất một trong số chúng bắt, điều làm lớp này thành dạng sắc hơn của lập luận: build có công cụ không chỉ

đắt, trên lớp này nó trống. Handle không an toàn luồng là cái kia. Chẩn đoán là `E0277`, ràng buộc trait chứ không phải lỗi mượn, vì từ chối đến từ `Send` chứ không từ borrow checker. Handle C++ mang quy tắc gắn luồng trong comment, là chỗ duy nhất C++ có để đặt. Trong Rust quy tắc đó là phần kiểu và `thread::spawn` yêu cầu nó trong chữ ký, nên trình biên dịch từ chối mà không phân tích thân. Một cảnh báo thuộc với cái đầu. Phân tích tĩnh cấu hình cho nó có thể đánh dấu use-after-move hợp lý, và `clang-tidy` có sẵn check cho đúng mẫu này, mà tài liệu riêng ghi rằng nó theo luồng điều khiển nhưng không theo đường, và chỉ xét biến cục bộ và tham số hàm [66]. Không cái nào chạy ở đây, vì lý do Mục 4.2.2, nên hàng ghi những gì bộ dò lỗi báo chứ không giới hạn công cụ C++ có thể chạm.

Cả bảy ca biên hành xử như dự đoán, và chúng đánh dấu cạnh của tuyên bố. Mỗi cái biên dịch trong Rust an toàn và mỗi cái hành xử sai. Ca deadlock treo cả hai ngôn ngữ và phải bị bộ điều khiển giết. Ca lost-update kết thúc cả hai ngôn ngữ bằng câu trả lời sai im lặng. So với tổng kỳ vọng 40 000 lần tăng chương trình Rust an toàn ghi 25 734, mất 14 266, và chương trình C++ dưới ThreadSanitizer ghi 27 767, mất 12 233. Không ngôn ngữ có lợi đáng kể ở đây, đó là điểm của việc mang lớp này. Chu kỳ tham chiếu rò rỉ cả hai, tràn bản build release tạo TEID sai cả hai, và lỗi trong một shard worker làm sập hơn shard đó cả hai.

Forgotten guard tách hai thứ cụm "RAII ngăn điều này" gộp lại. Rust loại bỏ dạng thường của lỗi: bản C++ lỗi trên đường xác thực trả về trước unlock viết tay, và bản Rust không có unlock để bỏ qua. Nó không loại bỏ lớp, vì lý do Mục 3.2 nêu. Cả hai chương trình phải bị bộ điều khiển giết, và công cụ không ngôn ngữ nào nói gì: không bộ dò lỗi báo bản C++, vì khóa không bao giờ được thả không phải race cũng không phải chu kỳ, và không có gì cho bộ phát hiện thứ tự khóa đóng. Đọc cạnh rò rỉ chu kỳ tham chiếu cặp là cùng bài học hai lần, tốn bộ nhớ ở đó và ngữ cảnh ở đây.

Seam double-free là ca không báo lớp lỗi mới, và nó có trong danh mục vì lý do đó. Nó là double-free của nhóm trong bảo đảm, chuyển sang phía kia quy tắc bằng cách viết theo cách ranh giới hàm nước ngoài phải viết. Rust an toàn từ chối giải phóng kép bằng `E0382`, vì ownership theo dõi giá trị. Đưa con trỏ thô cho caller C chính là thao tác kết thúc theo dõi đó, và từ đó `Box::from_raw` khẳng định ownership địa chỉ được cho. Khẳng định hai lần giải phóng lưu trữ hai lần, và chương trình abort trên check của bộ cấp phát. Nửa C++ của cặp cố ý bình thường và không đổi, nên chỉ cột Rust di chuyển. Bảo đảm không phải thuộc tính của ngôn ngữ xét riêng. Nó là thuộc tính của ngôn ngữ cùng ranh giới thành phần được triển khai phía sau, và ranh giới đó là cái Mục 6.4.8 đo.

Hai trong bảy là ca mà công cụ C++ *dẫn trước*, đó là kết quả danh mục hai phía được xây để có thể tạo ra. LeakSanitizer, đi kèm ASan, báo chu kỳ tham chiếu C++ là cấp phát không thể đến khi thoát. Chu kỳ Rust an toàn rò rỉ im lặng, vì rò rỉ an toàn trong Rust theo thiết kế rõ ràng và không gì trong chuỗi công cụ mặc định tìm nó. Điều này thuộc ngôn ngữ, không phải artefact của ví dụ tiêm, thể hiện ở công việc đang làm để đổi nó. Đề xuất thêm garbage collection cho Rust được thúc đẩy

chính xác bởi cấu trúc sở hữu chung mà đếm tham chiếu diễn đạt "vụng về, kém hiệu quả, hoặc cả hai" [67]. Nơi `unsafe` đánh dấu ranh giới an toàn bộ nhớ rõ ràng, ranh giới rò rỉ ngầm và trình biên dịch im lặng, đó là tiền đề của bộ kiểm tra rò rỉ tĩnh tìm lớp trong 19 trong hơn 1,200 crate công bố [68]. UBSan báo tràn C++, vì tràn có dấu undefined ở đó, trong khi tràn Rust quấn theo định nghĩa ở `opt-level = 2` và qua không đáng kể (Mục 3.2). Tương phản thứ hai cần nói cẩn thận, vì nó không rút về ngôn ngữ nào an toàn hơn. Sai nhưng xác định dễ debug hơn sai undefined, và không kém sai, và không cái nào là điều "an toàn bộ nhớ" thường được hiểu hứa. Quấn vòng cố ý cũng phổ biến đủ trong C và C++ để checker không thể coi mọi tràn là lỗi, và tràn undefined đã phá chương trình đang chạy khi compiler cải thiện [69].

Kết quả deadlock cần điều kiện mà cách đọc trước không có. ThreadSanitizer đánh dấu đảo chiều ở C++, nhưng một số lần chạy chứ không phải mọi lần: 5 trong 12 lần lặp trong lần chạy sau bảng này, trong khi chương trình deadlock mọi lần. Tỷ lệ bản thân không ổn định giữa các lần chạy và không nên đọc như một, vì các chiến dịch lặp trên máy này trả số hai phía nửa. Điều tái lập là tính gián đoạn. Cơ chế cụ thể, và nó tổng quát hóa ngoài danh mục này. TSan xây đồ thị thứ tự khóa từ acquisition *thành công*, nên chu kỳ chỉ đóng khi một luồng hoàn thành đảo chiều mà không chặn. Một lần chạy mà đảo chiều đầu chặn tạo một cạnh và không bao giờ cạnh thứ hai, và bộ phát hiện im lặng đúng lúc lỗi cắn. Điều khiển dương đảo cùng hai khóa mà không bao giờ chặn được báo mọi lần, điều thiết lập bộ phát hiện sống và không cấu hình sai. Tuyên bố trung thực cho lớp này do đó hẹp. Công cụ bắt đảo chiều mà chương trình sống sót, và nó không đáng tin trên cái deadlock, đó là đề xả khác nhiều cho pipeline tích hợp liên tục so với bắt sạch.

Sự phân biệt tất cả điều này bộc lộ chính xác, và đó là điều cụm "an toàn luồng" thường làm mờ. Bảo đảm đồng thời của Rust là vắng mặt *tranh chấp dữ liệu*, nghĩa là truy cập đồng thời không đồng bộ cùng vị trí. Nó không bảo đảm về *thứ tự acquisition khóa*, cũng không về *tính nguyên tử của thao tác ghép*, cũng không về rò rỉ, số học quấn vòng hay ngăn chứa lỗi, và nó bị treo ở mọi nơi chương trình viết `unsafe`. Lost update vẫn là ca im lặng nhất, vì không bộ dò lỗi báo nó. Với mọi truy cập khóa đúng không có race để bộ phát hiện race tìm.

Nơi các lớp biên bị bắt, và với chi phí gì. RQ2 hỏi lỗi nào được ngăn, và cũng ở giai đoạn nào phần còn lại được phát hiện. Với nhóm biên các cột bộ dò lỗi để phần thứ hai phần lớn trống. Hai bộ kiểm tra mô hình Rust trả lời, và phân công lao động giữa chúng là điều hệ Rust sản xuất đã được xác thực. Loom khám phá xen kẽ một cách sound trong mô hình bộ nhớ release/acquire, dùng giảm bán thứ tự một phần để giữ không gian trạng thái. Shuttle lấy mẫu chúng bằng kiểm thử đồng thời xác suất, mở rộng tới ca vượt tầm Loom [70]. Cả hai báo cả hai lỗi biên gốc. Loom báo deadlock là chu kỳ đóng và fail assertion lost-update, và Shuttle làm tương tự. Mỗi cái ghép với điều khiển mà lỗi được sửa, bằng thứ tự khóa nhất quán và giữ read-modify-write trong một vùng quan trọng, và cả hai điều khiển ra sạch. Báo cáo do đó đến từ lỗi chứ không từ công cụ.

Chi phí là phần đáng ghi. Không chương trình lỗi nào chạy dưới checker nào như viết. Mỗi cái phải import lại theo kiểu đồng bộ và luồng riêng của checker, vì công cụ không chặn primitive không thể lập lịch nó. Mỗi cái cũng phải thu nhỏ cỡ mô hình. Mô hình lost-update giảm từ hai vạn lần lặp mỗi luồng xuống một, và mô hình deadlock mất sleep mở rộng cửa sổ trên phần cứng thật. Câu trả lời cho nửa Rust của biên do đó là *phát hiện lúc kiểm thử, sau khi viết lại.* Ở nửa C++ bộ dò lỗi nhận chương trình không đổi và, với hai lớp này, tìm một không đáng tin và cái kia không hề. Hai nửa chỉ hướng ngược nhau, và câu chuyện không ngôn ngữ nào đơn giản. Hệ quả thực hành được phát triển ở Chương 7.

Seam double-free được trả lời bằng công cụ khác và đi kèm giới hạn sắc hơn. Miri diễn giải chương trình theo mô hình aliasing Stacked Borrows, và báo giải phóng thứ hai chính xác, như box treo tái tạo từ cấp phát đã giải phóng. Điều khiển giải phóng cùng handle một lần sạch, nên báo cáo đến từ lỗi chứ không từ bất cứ điều `Box::from_raw` làm nói chung. Giai đoạn cho lớp này do đó là kiểm thử lần nữa, và lần này không viết lại. Giới hạn là điều di chuyển phải cân. Miri diễn giải Rust và không thực thi host C++, nên chương trình nó kiểm tra dùng stand-in Rust cho host. Công cụ tìm lỗi này không thể trỏ vào cấu hình lỗi được giao, đó là đường nối với C++ ở phía kia. Không gì trong chuỗi công cụ kiểm tra ranh giới đó, và câu trả lời thực hành là điều Mục 6.4.8 lập luận từ nguyên mẫu: giữ đường nối đủ nhỏ để review bằng tay, vì review là điều còn lại.

Một lỗi không được tiêm. Mọi kết quả trên đến từ lỗi viết có chủ đích, và đó là điểm yếu cố định của phương pháp. Ai lắp danh mục đã biết Rust an toàn ngăn gì. Một ca trong nghiên cứu này không mang điểm yếu đó. Review thiết kế đặc tả thấy phần đồng thời cấm gửi thao tác ngoài cửa sổ chạy của dịch vụ (trước `start()`, hoặc sau `shutdown()`), trong khi không hiện thực nào kiểm tra điều đó. Lệnh cấm hướng tới caller không có cơ chế đằng sau, và trong khoảng trống đó hai ngôn ngữ không lỗi giống nhau. Đường submit C++ lập chỉ mục vector shard mà `shutdown()` đã xoá, mà AddressSanitizer báo là heap-use-after-free. Gọi trước `start()` là segmentation fault trên trang zero. Trong Rust cả hai lời gọi đã là panic, từ bounds check thường trên cùng thao tác lập chỉ mục, và không ai thiết kế cho nó.

Điều làm ca khác về mặt bằng chứng là nguồn gốc, không mức độ nghiêm trọng. Nó nằm trong mã viết để đúng, trong nhánh mà tuyên bố an toàn dựa vào ThreadSanitizer, có thể đến qua API công khai bằng cách gọi thao tác được ghi theo thứ tự được ghi. Nó cũng sống sót mọi kiểm tra nghiên cứu này chạy: 6 nhị phân kiểm thử C++, 27 kiểm thử Rust, 223 thao tác tương đương viết tay và 793 000 sinh, và ba cấu hình bộ dò lỗi báo sạch. Nó sống sót vì một lý do. Không cái nào gọi thao tác ngoài cửa sổ chạy. ASan tìm lỗi này chỉ khi bài kiểm thử lái nó, đúng tính chất lập luận chi phí trên gán cho phát hiện runtime, thấy ở đây trên artefact của chính nghiên cứu thay vì trên lỗi gieo.

Hướng không được đọc quá mức là hướng tôn vinh. Rust không ngăn điều này lúc biên dịch, và không gì trong borrow checker biết dịch vụ phải khởi động trước khi

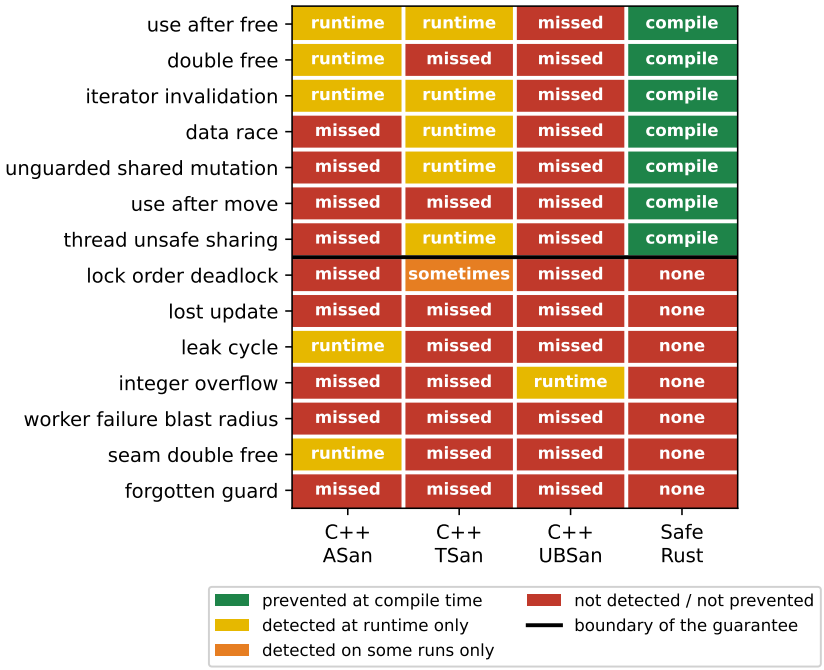
dùng. Điều biến undefined behaviour thành lỗi chẩn đoán được là bounds check slice. Đó là bảo đảm runtime, nên cùng loại bảo đảm bộ dò lỗi C++ cung cấp chứ không phải loại biên dịch Bảng 6.9 ghi. Tuyên bố trung thực là kiểm tra mặc định biến khoảng trống đặc tả thành panic, không phải hệ kiểu bắt lỗi vòng đời. Điều còn lại vẫn là phân biệt chương này lặp lại: panic nêu nguyên nhân mọi build, so với heap-use-after-free cần nhị phân có công cụ và bài kiểm thử nghĩ tới việc nhìn. Cả hai hiện thực giờ từ chối lời gọi rõ ràng, cả hai chiều, và cả hai bộ kiểm thử bao phủ nó.

Bảng 6.9: Phân loại lớp lỗi: kết quả đo cho danh mục tiêm. C++ dưới `g++ 13.3` với ASan/TSan/UBSan. Rust dưới `rustc 1.97`, edition 2021. "Lỗi biên dịch" nghĩa chương trình Rust an toàn không build, nên lỗi không thể viết. Khối dưới là nhóm biên: lớp lỗi chọn vì Rust an toàn *không* được kỳ vọng ngăn, và không ngăn. Hai trong bảy lớp đó là nơi công cụ C++ dẫn trước. Thân bảng sinh từ `faults/results/taxonomy.csv`, nên không thể mâu thuẫn lần chạy tạo ra nó.

Lớp lỗi	ASan	TSan	UBSan	Kết quả Rust an toàn
Within the guarantee				
Use-after-free	detected	detected	not detected	compile error (E0499)
Double-free	detected	not detected	not detected	compile error (E0382)
Iterator invalidation	detected	detected	not detected	compile error (E0502)
Data race	not detected	detected	not detected	compile error (E0373†)
Unguarded shared mutation	not detected	detected	not detected	compile error (E0502†)
Use-after-move	not detected	not detected	not detected	compile error (E0382)
Thread-unsafe handle shared	not detected	detected	not detected	compile error (E0277)
Boundary of the guarantee				
Lock-order deadlock‡	not detected	intermittent§	not detected	compiles, deadlocks
Lost update	not detected	not detected	not detected	compiles, wrong result
Reference-cycle leak	detected	not detected	not detected	compiles, leaks
Integer overflow (release)	not detected	not detected	detected	compiles, wrong result
Worker-failure blast radius	not detected	not detected	not detected	compiles, shard outage
Double-free across the seam	detected	not detected	not detected	compiles, aborts
Forgotten lock guard‡	not detected	not detected	not detected	compiles, deadlocks

† Nhiều chẩn đoán kích hoạt cho các ca này. Cái đầu được hiện (race dữ liệu cũng nâng `E0382` và `E0502`, đột biến không bảo vệ `E0524` và `E0597`). ‡ Ở các hàng này nhị phân có công cụ bị bộ điều khiển giết bởi timeout thay vì báo gì. Không gì bị hỏng và không gì undefined, nên công cụ phát hiện hỏng không có gì nói. Forgotten guard im lặng dưới cả ba, và ca thứ tự khóa là ca TSan đôi khi bắt, như ghi chú sau ghi. § ThreadSanitizer báo đảo chiều thứ tự khóa một số lần chạy chứ không phải mọi lần, đo ở 5 trong 12 lần lặp, trong khi chương trình deadlock mọi lần. Bộ phát hiện xây đồ thị thứ tự khóa từ acquisition *thành công*, nên lần chạy mà đảo chiều đầu chặn không bao giờ hoàn thành cạnh thứ hai của chu kỳ.

Đọc cả bảng làm hai điều thấy cùng lúc. Trên quy tắc biên toàn cột Rust là từ chối biên dịch, trong khi cột C++ là vá runtime và bỏ sót hẳn. Dưới quy tắc cột Rust tham gia, và trên hai hàng nó tụt lại. Hình 6.9 vẽ cùng 14 kết quả thành ma trận, nơi thay đổi tính chất tại quy tắc là hình dạng hình chứ không phải điều đọc từng hàng. Kết quả sinh từ nhật ký máy đọc được của bộ chạy (`faults/results/taxonomy.csv`), kết quả kiểm tra mô hình từ `faults/results/model_checking.csv` và của interpreter từ `faults/results/miri.csv`. Bảng và hình từ cùng nhật ký, nên không cái nào lệch cái kia.



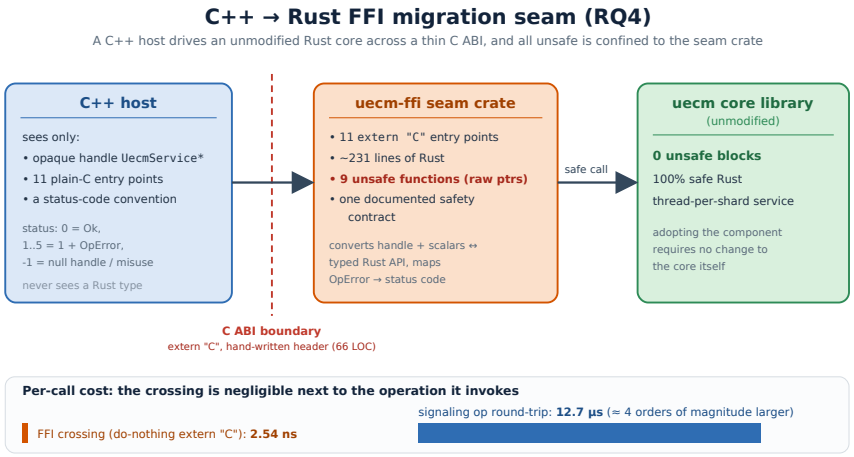
Hình 6.9: Kết quả của Bảng 6.9 vẽ thành ma trận, sinh từ cùng `faults/results/taxonomy.csv`. Trên quy tắc cột Rust là một màu: mọi lỗi là lỗi biên dịch, nên không cái nào viết được. Cột C++ ở đó là vá thay vì một bộ dò lỗi bao phủ. Hàng use-after-move là hàng không bộ dò lỗi nào bao phủ hết. Dưới quy tắc cột Rust tham gia, và trên rò rỉ chu kỳ tham chiếu và tràn bản build release công cụ C++ dẫn trước. Deadlock thứ tự khóa là ô công cụ báo gián đoạn: ThreadSanitizer đánh dấu đảo chiều trong 5 trong 12 lần chạy trong khi chương trình deadlock mọi lần.

6.4.8 Di chuyển (RQ4)

Áp dụng tăng dần được đánh giá bằng nguyên mẫu hoạt động, xây theo hướng di chuyển thực tế: host C++ hiện có giữ thành phần Rust sau ABI C. Dịch vụ Rust được expose qua đường nối `extern "C"` mỏng trong crate *riêng* phụ thuộc thư viện lõi không sửa, nên áp dụng thành phần không cần đổi chính thành phần. Host C++ chỉ thấy handle mờ và tập điểm vào C thuần (header viết tay, không binding sinh), và lái toàn vòng đời UE qua ranh giới. Mô hình lỗi vượt qua nguyên vẹn, nên thiết lập kết nối trùng trả `InvalidState` qua đường nối. Hình 6.10 cho thấy cấu trúc kết quả và chỗ `unsafe` nằm.

Nguyên mẫu cho số liệu liên vận cụ thể, tóm tắt trong Bảng 6.10. Đường nối nhỏ: 11 điểm vào ABI C trong 231 dòng Rust, cộng header C 66 dòng, không phụ thuộc FFI bên thứ ba. Ranh giới an toàn sạch. Thư viện lõi không `unsafe`, và xử lý con trỏ thô mà ABI C yêu cầu gói trong crate đường nối, trong 9 hàm `unsafe`, dưới một hợp đồng an toàn được ghi. Phía C++ không bao giờ thấy kiểu Rust.

Chi phí mỗi lần gọi được đo hai lần, vì hai phép đo trả lời câu hỏi khác và chỉ cái thứ hai trả lời RQ4. Cái đầu là bản thân vượt qua. Gọi `extern "C"` không làm gì tốn 2.54 ns, là lời gọi gián tiếp thường và không đáng kể cạnh vòng micro giây của thao tác báo hiệu. Điều đó giới hạn vượt qua và không nói đường nối làm gì với thao tác thật, vì lời gọi rỗng và vòng đầy đủ không phải cùng thí nghiệm. Vòng bị chi phối

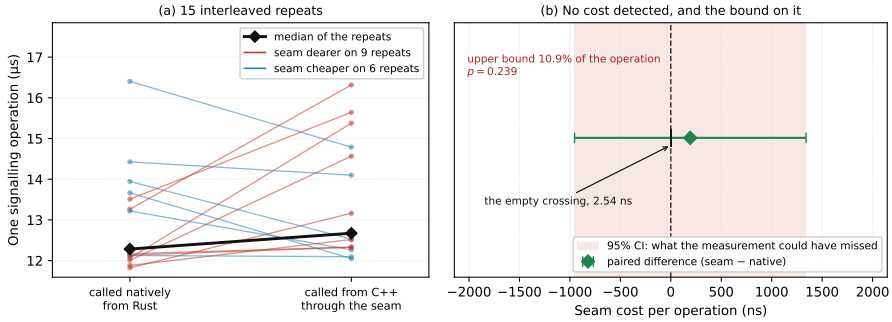


Hình 6.10: Đường nối di chuyển FFI C++-sang-Rust. Host C++ chỉ thấy handle mờ, 11 điểm vào C thuần, và quy ước mã trạng thái. Crate đường nối `extern "C"` mỏng tập trung toàn bộ `unsafe` con trỏ thô (9 hàm) và gọi lõi Rust không sửa, vẫn không `unsafe`. Chi phí vượt không làm gì 2.54 ns so với thao tác 12.7 µs. Đo so với cùng thao tác báo hiệu gọi bản địa, chi phí đường nối mỗi thao tác được giới hạn ở 10.91%.

bởi bàn giao giữa luồng, và lập luận bàn giao "sẽ trả giống hệt" là suy luận thay cho phép đo.

Phép đo do đó làm trực tiếp. Cùng thao tác báo hiệu, trên cùng cấu hình dịch vụ và cùng vòng, được gọi bản địa từ Rust rồi từ C++ qua đường nối, xen kẽ qua 15 lần lặp để cặp khớp. Khác biệt ghép cặp là +192 ns mỗi thao tác, với khoảng bootstrap $[-954, +1341]$ ns và p ghép cặp chính xác 0.239. Không phát hiện chi phí. Hữu ích hơn, khoảng giới hạn mọi chi phí phép đo có thể bỏ lỡ ở 10.91% thao tác. Xen kẽ không tùy chọn ở đây. Một lượt mỗi bên, chạy khi chuẩn bị mục này, làm đường nối trông *nhanh hơn* gọi bản địa một phần ba, vì độ lan giữa các lần chạy của bàn giao trên host chia sẻ rộng hơn nhiều so với bất cứ gì ABI C có thể đóng góp. Hình 6.11 cho thấy lần lặp tạo các số đó, và panel trái là độ lan đó. Các cặp giao nhau, và dải chúng phủ rộng hơn một bậc độ lớn so với khác biệt đang kiểm.

Nỗ lực kỹ thuật chính do đó vào *thiết kế* bề mặt API tương thích C, với handle mờ, thao tác kiểu vô hướng và quy ước trạng thái. Nó không vào overhead runtime. Chiều ngược (Rust gọi vào C++) đối xứng, nhưng cần wrapper ABI C phía C++ và đưa `unsafe` trở lại caller Rust. Nguyên mẫu và phép đo được công bố cùng luận văn, và chúng thông tin cho hướng dẫn di chuyển của Chương 7.



Hình 6.11: Chi phí liên vận đo trên thao tác thật thay vì suy từ lời gọi rỗng. **(a)** Cùng thao tác báo hiệu gọi bản địa từ Rust và từ C++ qua đường nối, một dòng mỗi lần lặp xen kẽ, trung vị màu đen. **(b)** Các cặp rút thành khác biệt: +192 ns mỗi thao tác với khoảng bootstrap 95% $[-954, +1341]$ ns và p ghép cặp chính xác 0.239. Không phát hiện chi phí, và đầu trên của khoảng giới hạn chi phí phép đo có thể bỏ lỡ ở 10.91% thao tác. Vượt `extern "C"` rỗng, 2.54 ns, được đánh dấu để tỷ lệ chứ không phải câu trả lời: không phân biệt được với không trên trục thao tác được đo.

Bảng 6.10: Nguyên mẫu di chuyển FFI: kích thước đường nối, ranh giới an toàn và chi phí liên vận mỗi lần gọi (host C++ gọi thành phần Rust qua ABI C, `-O2`). Thư viện lõi không sửa và không `unsafe`. Mọi liên vận gói trong crate đường nối. Ba hàng cuối là so sánh trực tiếp: một thao tác báo hiệu trên cùng cấu hình dịch vụ và cùng vòng, gọi bản địa rồi qua đường nối, để khác biệt tách phần ranh giới thêm chứ không phải thao tác tốn.

Chỉ số	Giá trị
C-ABI entry points	11
Seam size (Rust `lib.rs`)	231 LOC
Hand-written C header	66 LOC
`unsafe` functions in core library	0
`unsafe` functions in FFI seam	9
Raw FFI crossing	2.54 ns/call
Signaling op, native caller	12.3 µs/call
Signaling op, through the seam	12.7 µs/call
Seam cost per operation (paired)	+192 ns $[-954, +1341]$
as a fraction of the operation	+10.91% at the upper bound

Ba trục chi phí đường nối không chạm. RQ4 hỏi áp dụng tăng dần tốn gì về công sức, liên vận và rủi ro. Đường nối trả lời hạng giữa bằng phép đo, và để hai hạng kia cho số dòng riêng, mô tả một thành phần và không nói công việc xây nó. Ba trục nữa được đo ở đây, chọn vì đội hỏi từng cái trước mọi câu đã trả lời: chuỗi công cụ mất bao lâu, artefact lớn cỡ nào, và bao nhiêu mã nguồn không rõ gốc cấu hình kéo theo. Bảng 6.11 gom chúng.

Bảng 6.11: Trục chi phí áp dụng ngoài đường nối. Thời gian build là trung vị 15 lần lặp xen kẽ ở 14 job song song trên máy phát triển cục bộ (`g++ 13.3`, `rustc 1.97`), bỏ lần khởi động nóng. "Chỉ first-party" build lại cùng target Rust với cây phụ thuộc còn ấm, nên phần chênh phía trên là hệ sinh thái tốn chứ không phải compiler. Dấu chân là nhị phân `conformance_runner`, cặp thực thi khớp nhất trong nghiên cứu, báo ở liên kết mặc định mỗi toolchain rồi ở cả hai chính sách khớp, vì mặc định khác và thứ tự phụ thuộc chính sách giữ. Số crate là bắc cầu từ `Cargo.lock`. Ô `unsafe` trống với C++ vì ngôn ngữ không đánh dấu vùng như vậy, nên không có số so sánh.

Đo lường	C++	Rust
Clean build, core library only (first-party source on both sides)		
Wall clock (s)	1.9	1.2
CPU time (s)	2.6	3.9
Clean build, every default target		
Wall clock (s)	5.7	8.1
CPU time (s)	35.9	64.8
CPU time, first-party only (s)	35.9	29.6
Incremental rebuild after one edit		
Source file changed (s)	1.2	3.6
Core header changed (s)	5.8	3.6
Static footprint, `conformance_runner` (KiB)		
Shipped, each toolchain's default linkage	106	723
Matched: standard library linked statically	1577	723
Matched: standard library resolved dynamically	106	269
Shared standard library left on the host	2531	5295
Third-party code resolved		
Transitive crates, compared component	0	0
Transitive crates, matched-allocator arm	0	8
Transitive crates, benchmark harness	0	17
`unsafe` sites in that third-party code	—	1064

Hai trong ba không nhạy môi trường theo nghĩa Mục 6.4.7 cho danh mục lỗi. Kích thước nhị phân và số phụ thuộc là thuộc tính toolchain chứ không của máy chủ, nên chúng mang từ phép đo này không đổi. Thời gian build thì không, và được xử lý tương ứng. Lấy trên máy phát triển cục bộ qua 15 lần lặp xen kẽ ở 14 job song song, bỏ lần đầu để bộ nhớ đệm trang lạnh không bị đọc như thuộc tính compiler. Giây tuyệt đối mô tả máy đó. Điều mang theo là tỷ lệ ghép cặp giữa hai toolchain đo luân phiên trên một host, cùng lý do chiến dịch xen kẽ biến thể.

Thời gian biên dịch tách compiler khỏi hệ sinh thái. Trên phạm vi khớp, chỉ thư viện lõi không mã bên thứ ba hai bên, Rust xong sớm hơn và C++ tiêu ít thời gian máy hơn: 1.9 s so với 1.2 s đồng hồ tường, và 2.6 s so với 3.9 s CPU. Hai thứ tự không đồng ý vì đo khác nhau, và ở đây đồng hồ tường yếu hơn. Thư viện C++ là hai đơn vị dịch, nên ở 14 job chỉ hai chạy song song, trong khi `rustc` tách một crate thành đơn vị codegen và giữ khoảng ba lõi bận. Đồng hồ tường ở phạm vi này do đó thưởng bố cục file của codebase này cũng như toolchain. Thời gian CPU là so sánh không phụ thuộc nó.

Build đầy đủ là nơi phản đối thực tế áp dụng Rust thường đáp, và đọc lần đầu nó

đứng. Mọi target mặc định tốn 64.8 s CPU so với 35.9 s, hệ số 1.8. Tách con số đó chuyển nó tới chỗ hữu ích hơn. Build lại với cây phụ thuộc còn ấm, cùng target Rust tốn 29.6 s. Vậy 54% build sạch là mã bên thứ ba, và mã riêng dự án biên dịch bằng 0.8 lần CPU `g++` cần cho tương đương. Trên codebase này compiler không phải thứ làm build chậm. Hai điều kiện giới hạn cách đọc đó. Hai tập target gần chứ không giống hệt. Và phụ thuộc C++ không vắng, chúng được biên dịch sẵn vào bản phân phối dưới dạng thư viện chia sẻ, đó là lợi ích thật của hệ sinh thái trưởng thành chứ không phải artefact kế toán.

Vòng sửa-biên dịch nhanh hơn ở C++ và kém dự đoán hơn. Sau một sửa file hiện thực, C++ build lại trong 1.2 s, nhanh hơn 3.1 lần so với 3.6 s của Rust. Sau một sửa header lõi mất 5.8 s. Đó chậm hơn Rust 1.6 lần và, đo bằng CPU, không phân biệt được với build cây từ đầu. Số Rust không đổi. Chạm mọi file trong crate tốn 3.8 s so với 3.6 s cho chạm một, vì đơn vị biên dịch là crate chứ không phải file. Vòng sửa-biên dịch nhanh hơn ở C++ khi sửa nằm trong đơn vị dịch, và chậm hơn khi không. Trường hợp nào chi phối ngày làm việc phụ thuộc kỷ luật header của codebase, không phụ thuộc ngôn ngữ. Hình dạng kết quả đó quen từ RQ1. Số C++ tốt hơn ở trung vị và tệ hơn ở đuôi.

Thứ tự dấu chân đảo khi chính sách liên kết đổi. Build theo cách mỗi toolchain giao mặc định, nhị phân Rust `conformance_runner` là 6.8 lần C++, 723 KiB so với 106. So sánh đó giữa hai chính sách liên kết chứ không giữa hai ngôn ngữ, vì `g++` để `libstdc++` cho loader động trong khi `rustc` liên kết thư viện chuẩn vào nhị phân. Đo ở chính sách khớp khoảng cách hẹp rồi đổi dấu. Cả hai thư viện chuẩn giải quyết động, C++ nhỏ hơn 2.5 lần. Cả hai liên kết tĩnh, Rust nhỏ hơn 2.2 lần.

Sự đảo có nguyên nhân đo được chứ không lập luận. 47% symbol có kích thước trong nhị phân C++ liên kết tĩnh là locale và facet stream, trong đó facet tiền, ngày và ký tự rộng chương trình này không bao giờ khởi tạo, vì liên kết tĩnh `libstdc++` mang bảng facet vào như một tập. Không cấu hình động miễn phí, vì mỗi cái để thư viện trên host trả một lần: 2531 KiB cho `libstdc++` so với 5295 KiB cho thư viện chuẩn chia sẻ Rust. Kết luận đáng mang theo là câu hỏi có ba câu trả lời, và số 6.8× ít thông tin nhất. Với artefact tự chứa, điều image container hay thiết bị muốn, thứ tự ủng hộ Rust.

Thành phần so sánh không giải quyết mã bên thứ ba. Phản đối chuẩn Rust trong codebase được quản lý là rủi ro phụ thuộc bắc cầu. Trên artefact nghiên cứu này so sánh nó không phát sinh: 0 crate và 0 site `unsafe`, biến tuyên bố không phụ thuộc của Mục 5.4.4 thành phép đo thay vì khẳng định. Bề mặt xuất hiện trong cấu hình quanh thành phần đó. Liên kết bộ cấp phát khớp giải quyết 8 crate. Trong số đó, 5 tới artefact và 3 chỉ chạy khi đang build. Phân biệt đó quan trọng, vì phụ thuộc build không đưa lệnh vào nhị phân nhưng vẫn chạy mã trên máy build. Bộ

điều khiển benchmark giải quyết 17. Đếm bằng một quy tắc cú pháp trên nguồn, hai cây đó giữ 623 và 1064 site `unsafe`.

Hai cách đọc nên tách. Số giới hạn nơi audit phải nhìn, và không phải điểm rủi ro, vì crate tập trung `unsafe` trong một khối lớn điểm thấp hơn crate rải cùng công việc qua nhiều khối nhỏ. Cột C++ trống theo cấu trúc chứ không theo lợi thế. Ngôn ngữ không đánh dấu vùng như vậy, nên toàn chương trình đứng chỗ khối `unsafe` của Rust đứng, và ghi không ở đó là lỗi loại. Trục thiết lập là nơi mã chưa audit trong dự án thực sự nằm. Thành phần sẽ giao không mang gì, và bề mặt lớn nhất thuộc bộ máy tạo các phép đo.

7 Thảo luận

7.1 Tóm tắt phát hiện

Bảng 7.1 gom phát hiện chính cho mỗi câu hỏi nghiên cứu, cùng độ mạnh bằng chứng hiện tại. Kết quả an toàn (RQ2) không phụ thuộc môi trường đo chút nào. Kết quả di chuyển (RQ4) phụ thuộc theo một trục duy nhất, vì thời gian build là phép đo thời gian, nên được báo dạng tỷ lệ ghép cặp. Kết quả hiệu năng và tài nguyên phụ thuộc môi trường, và phép chia trung thực giờ nằm trong RQ1 chứ không giữa các chương. Điểm gối bão hoà, so sánh quá tải, và so sánh tài nguyên được giải quyết trên máy chủ Mục 6.1, và tái lập qua kiểm soát bộ cấp phát. Đuôi xa dưới bão hoà vẫn chưa giải quyết, vì lý do thuộc máy chứ không phải phân tích.

Bảng 7.1: Tóm tắt phát hiện theo câu hỏi nghiên cứu và độ mạnh bằng chứng hiện tại.

RQ	Phát hiện chính	Trạng thái bằng chứng
RQ1	Câu trả lời chia tại bão hoà. Dưới đó các biến thể đồng ý trong 2 μs (2.1%) ở trung vị và không có thứ tự đuôi nào phân giải được. Số đầu-cuối chỉ 0.250% thao tác, nên không thể giới hạn hiệu ứng ngôn ngữ trong mọi trường hợp, trong khi thao tác đo riêng nhanh hơn 1.6× ở Rust. Qua bão hoà chúng phân kỳ dứt khoát: C++ duy trì 1.8× thông lượng của Rust ở 784 kops/s, trong 20/20 lần lặp dưới cả hai bộ cấp phát.	Quá tải và thao tác: đã giải quyết. Đuôi xa: chưa giải quyết trên máy này. Cơ chế: đã kiểm. Primitive riêng không đủ
RQ2	Cả bảy lớp an toàn bộ nhớ và tranh chấp dữ liệu tiêm được ngăn lúc biên dịch trong Rust an toàn, trong khi ở C++ mỗi lớp biên dịch và chỉ bị bắt, nếu có, lúc runtime và không bộ dò lỗi đơn lẻ nào. Bảo đảm dừng ở đó: bảy lớp biên biên dịch cả hai ngôn ngữ, và trên rò rỉ và tràn bản build release công cụ C++ báo điều Rust không.	Cuối cùng
RQ3	Phạt bộ nhớ tái lập: thêm 1.9–3.4 MiB peak RSS trong bản build Rust dưới *cả hai* bộ cấp phát, nhỏ hơn một bậc độ lớn so với 32.9–34.4 MiB của bộ cấp phát. CPU đảo giữa các chế độ: Rust tiêu tối đa 3.4 điểm phần trăm ít hơn dưới bão hoà và 1.9× nhiều hơn mỗi thao tác hoàn thành phía trên.	Bộ nhớ: đã giải quyết, chưa giải thích. CPU: đã giải quyết cả hai chế độ, dấu đối
RQ4	Đường nối nhỏ và giữ lõi không `unsafe` lúc biên dịch. Đo so với cùng thao tác gọi bản địa, không phát hiện chi phí và mọi thuế được giới hạn ở 10.91% một thao tác. Chi phí thật là thiết kế API. Lõi vẫn lộ lúc chạy với lỗi bộ nhớ ở host C++ chia không gian địa chỉ. Trên các trục áp dụng ngoài đường nối, 54% build sạch Rust là mã bên thứ ba và mã riêng dự án biên dịch bằng 0.8 lần CPU `g++` cần, thứ tự dấu chân đảo theo chính sách liên kết, và thành phần so sánh giải quyết 0 crate.	Cuối cùng. Giây build tuyệt đối gắn host, tỷ lệ thì không

7.2 Thảo luận về các câu hỏi nghiên cứu

RQ1 (độ dự đoán được). RQ1 có hai nửa, dưới bão hoà và qua nó, và chiến dịch trả lời chúng đủ khác để báo cáo một mình sẽ bóp méo kết quả.

Dưới bão hoà hai ngôn ngữ tương đương về hành vi trung tâm, với biên độ không triển khai nào để ý. Khác biệt ghép cặp nhỏ ở trung vị vượt hiệu chỉnh là một phần nhỏ của thao tác 80 μs. Đuôi xa trong chế độ này là chuyện khác, và triển khai cũng quan tâm. Ở đó kết quả là null mà máy này không thể biến thành thứ tự, nên tương đương tuyên bố ở đây không chạm tới nó. Kết quả trung tâm nhất quán với bằng chứng rộng hơn rằng kiểm tra an toàn của Rust phần lớn được CPU out-of-order hiện đại hấp thụ [26, 71]. Nó cũng yếu hơn trông, vì lý do Mục 6.4.2 nêu. Thao tác hai ngôn ngữ thực thi là 0.250% những gì quét đo. Null đầu-cuối trong thiết kế chi phối bởi đánh thức giữa luồng do đó nói về thiết kế chứ không về ngôn ngữ. Đo riêng, thao tác khác 1.6×.

Qua bão hoà câu trả lời đổi loại, và đây là kết quả duy nhất trong nghiên cứu trả lời RQ1 đúng theo nghĩa RQ1. Bản build C++ duy trì 1.8 lần thông lượng của bản Rust trong khi tiêu nhiều CPU hơn mỗi thao tác hoàn thành, tái lập được và ở sàn đã hiệu chỉnh của cả họ kiểm định (Mục 6.4.4). Vậy dạng thực tế câu trả lời RQ1 không phải lựa chọn ngôn ngữ vô quan với độ dự đoán. Nó vô quan *cho đến khi dịch vụ không còn chịu tải*, và lúc đó nó quyết định bao nhiêu công việc vẫn được làm. So sánh dừng ở tải danh nghĩa, như hầu hết trong tài liệu này, sẽ kết luận ngược với cùng bộ máy.

Điều đó đặt lại công trình tiền nhiệm gần nhất thay vì chỉ đồng ý. Arvidsson và Kaller đo đại lượng này cho thủ tục mặt điều khiển 5G và thấy phần trăm thứ 99 của Rust suy giảm nhanh hơn qua ngưỡng tốc độ yêu cầu. Họ truy về `async/await` không stack, cho phép chuyển ngữ cảnh ngay trước khi thao tác I/O bắt đầu [55]. Mục 7.5 chấp nhận sự quy kết đó, và nghiên cứu này được xây để loại nhiễu đằng sau: một worker mỗi shard qua hàng đợi chặn có giới hạn, và không runtime bất đồng bộ hai bên.

Mục 1.3 cam kết trước mỗi kết quả sẽ có nghĩa gì, và nhánh thứ hai là nhánh kích hoạt. Suy giảm qua ngưỡng tốc độ xuất hiện với mô hình đồng thời giữ cố định, nên runtime bất đồng bộ không thể là nơi duy nhất hiệu ứng như vậy bắt đầu. Dự đoán cũng đặt tên nơi nhìn trước nếu điều đó xảy ra: kiểm tra biên và tràn [72, 73]. Phần đó sai. Hiệu ứng chỉ xuất hiện khi đường bàn giao tranh chấp, trỏ thay vào hai primitive chặn của hai thư viện chuẩn ghi trong Bảng 5.2. Mục 6.4.5 rồi kiểm tra liệu các primitive đó riêng giải thích khoảng cách, và chúng không tái tạo thứ tự toàn dịch vụ ở mức ý nghĩa. Điều đó thu hẹp nguyên nhân vào công việc thêm trên đường bàn giao C++ chứ không phải mô hình an toàn. Ghi rằng dự đoán giải quyết ngược giả thuyết đầu của chính nó, và microbenchmark theo sau tinh chỉnh quy kết thay vì xác nhận, hữu ích hơn cho nghiên cứu sau hơn là lặng thay bằng lời giải thích còn sống.

RQ2 (an toàn). Kết quả không phụ thuộc môi trường. Điều làm nó thú vị không phải nửa xác nhận kỳ vọng, mà nửa giới hạn nó. Trong phạm vi bảo đảm tuyệt đối theo một nghĩa cụ thể, và khác biệt với C++ là khác loại chứ không phải mức độ. Trình biên dịch từ chối build loại trừ lỗi khỏi mọi thực thi. Bộ dò lỗi chỉ loại trừ khỏi thực thi bài kiểm thử có công cụ tình cờ đi qua. Điều tuyệt đối là độ phủ thực thi, không phải độ rộng danh mục. Phạm vi vẫn là 14 chương trình viết cho nó. Điều đó khớp bằng chứng CVE rằng lỗi an toàn bộ nhớ Rust luôn liên quan `unsafe` [42], và cảnh báo unsafety vẫn lan qua phụ thuộc [43], đó là lý do giữ lõi không `unsafe` quan trọng. Seam double-free là bằng chứng đó tái tạo trong điều kiện kiểm soát: lớp duy nhất trong danh mục mà Rust an toàn từ chối và Rust unsafe không.

Ca duy nhất vượt chương trình đã viết là lỗi vòng đời của Mục 6.4.7. Nó được tìm trong hiện thực, không viết cho danh mục, và sửa bức tranh hai hướng. Nó củng cố lập luận chi phí. Một lỗi thật trong mã viết để đúng sống sót toàn bộ bộ xác minh của nghiên cứu này, tức kiểm thử, vector tương đương và ba bộ dò lỗi, chỉ vì không gì đi qua đường đó. Phát hiện runtime phụ thuộc độ phủ kiểm thử là đề xuất yếu hơn trong thực tế so với trong bảng mà mỗi hàng được lái có chủ đích.

Nó cũng thu hẹp điều có thể ghi cho Rust, vì đóng góp ngôn ngữ ở đó là bounds check chứ không phải chứng minh. Rust an toàn biến undefined behaviour thành panic xác định, đó là tuyên bố khác và kém hơn ngăn lỗi. Tổng quát hoá trung thực là bảo đảm đáng ngân sách là *kiểm tra mặc định* chứ không phải *loại trừ lúc biên dịch*. Nguyên nhân gốc đáng đặt tên, vì đó là rủi ro của phương pháp chứ không phải ngôn ngữ nào. Đặc tả nêu lệnh cấm mà không gán cơ chế để mỗi hiện thực lỗi theo idiom riêng. Hai idiom ở đây là panic và use-after-free.

Nơi bảo đảm dừng quan trọng không kém cho nhà cung cấp quyết định Rust mua gì, và hai lớp biên minh hoạ vì lý do khác nhau. Deadlock thứ tự khóa cho thấy Rust suy luận về *aliasing* chứ không về *thứ tự acquisition*. Guard `Mutex` là giá trị thường mà borrow checker theo dõi vòng đời hoàn hảo, và theo dõi hoàn hảo không nói hai luồng lấy khóa theo thứ tự tương thích. Công cụ quanh ngôn ngữ xác nhận đây là biên chứ không phải artefact ví dụ tiêm. Phát hiện deadlock tĩnh cho Rust là bài toán nghiên cứu đang hoạt động riêng [74], và bộ kiểm tra mô hình cho mã kernel Rust được thúc đẩy rõ ràng bởi quan sát ownership và borrowing để lại tranh chấp dữ liệu, deadlock và livelock vẫn chạm tới [75].

Lost update sắc hơn nữa. Nó thoát ThreadSanitizer đúng lý do thoát borrow checker, như Mục 6.4.7 nêu. Với mọi truy cập đồng bộ đúng không có truy cập không đồng bộ để tìm. Cũng không lớp lề. Trong nghiên cứu thực địa 105 lỗi đồng thời thật, khoảng một phần ba ca không deadlock vi phạm thứ tự lập trình viên định, không phải khóa của họ, và tác giả ghi ý định như vậy không diễn đạt dễ qua khóa [76]. Phân biệt nó bộc lộ, giữa race dữ liệu và điều kiện đua, là điều cụm "an toàn luồng" thường gộp, và Rust chỉ loại bỏ cái đầu.

Hệ quả thực hành là áp dụng Rust loại bỏ hoàn toàn một loại lỗi đồng thời trong khi để nguyên loại kia. Loại còn lại là loại thất bại im lặng, làm hỏng bộ đếm hay quyết định trạng thái thay vì crash. Kế hoạch di chuyển coi "an toàn bộ nhớ" là "an toàn

đồng thời" sẽ do đó lập ngân sách review và kiểm thử sai. Đó là lý do biên được báo cáo như kết quả hạng nhất chứ không phải điều kiện. Hình 7.1 vẽ chính biên, với hai nửa bảo đảm hai bên đường và cơ chế thực thi, hoặc vắng mặt của nó, được đặt tên mỗi panel. Nó mang hai ca đồng thời thay vì cả bảy ngoài bảo đảm, vì chúng là cặp làm rõ phân biệt, và Bảng 6.9 giữ toàn danh mục.



Hình 7.1: Nơi bảo đảm biên dịch dừng. Bên trái là các lớp Rust an toàn từ chối hẳn, nhóm theo *cơ chế* từ chối thay vì theo họ lỗi: ownership và borrowing cho lỗi vòng đời và aliasing, đó là lý do vô hiệu hoá iterator nằm cùng chúng, và kỷ luật `Send`/`Sync` cho truy cập không đồng bộ vào một vị trí. Bên phải là các lớp danh mục đưa vào để tìm cạnh, mà không trình biên dịch nào ngăn. Sơ đồ đặt tên một lớp mỗi cơ chế và không phải danh mục, mà Bảng 3.1 liệt kê đầy đủ: use-after-move thuộc các ca ownership bên trái và handle không an toàn luồng chia sẻ với `Send`/`Sync` bên đó, trong khi rò rỉ, tràn bản build release, lỗi worker, seam double-free và forgotten guard nằm cạnh hai ca bên phải. Cụm "an toàn luồng" trải cả hai bên đường, và bảo đảm chỉ bao trái. Đường lại di chuyển thêm một lần nữa so với sơ đồ thể hiện, vì chương trình viết `unsafe` vượt qua nó tự nguyện.

RQ3 (overhead). Câu trả lời chia theo tài nguyên, và các phép chia không cùng chiều. Về bộ nhớ có phạt. Bản build Rust mang vài MiB peak resident set hơn bản C++, và giữ dấu đó dưới cả hai bộ cấp phát. Hai điều làm nó đáng tin ngoài giá trị p. Peak RSS là max cấp lần chạy chứ không phải thống kê đuôi, nên jitter lập lịch phá p99 không chạm nó. Và hiệu ứng nhỏ hơn một bậc độ lớn so với của bộ cấp phát, đó là cách hữu ích để định cỡ nó. Đội chọn giữa hai ngôn ngữ này di chuyển ít bộ nhớ hơn đội chọn giữa hai bộ cấp phát này.

Về CPU câu trả lời phụ thuộc tải, và phụ thuộc đó chính là phát hiện chứ không phải biến chứng. Dưới bão hoà Rust tiêu *ít* hơn một chút, nhất quán dưới cả hai bộ cấp phát. Qua bão hoà dấu đảo và biên độ tăng (Mục 6.4.4). Một số duy nhất cho "chi phí CPU của mô hình an toàn Rust" sẽ trung bình hai chế độ hành xử ngược chiều, và không số nào là mô hình an toàn. Thông lượng chia cùng cách vì cùng lý do. Dưới bão hoà không có gì để đo, vì thông lượng đạt theo tải đề nghị theo cấu

trúc. Phía trên nằm hiệu ứng lớn nhất nghiên cứu.

Ba điều giới hạn điều kết quả bộ nhớ cấp phép. Nó nhỏ về tuyệt đối, và là chi phí cố định chứ không phải mỗi ngữ cảnh, nên phần dấu chân triển khai giảm khi triển khai lớn. Khác biệt bố cục đo giữa hai kiểu `UeContext` gần như không đóng góp. Nó cũng chưa giải thích. Tách nó giữa dự trữ arena bộ cấp phát, bộ nhớ đệm mỗi luồng và dấu chân tĩnh cần hồ sơ cấp phát và bộ đếm phần cứng nghiên cứu không thu, đó là khoảng trống dụng cụ đo đơn rõ nhất trong RQ3. Và không nên mở rộng thành tuyên bố về mô hình an toàn Rust, vì không gì ở đây tách bounds checking hay sao chép do ownership là nguyên nhân. Cơ chế tài liệu đã định vị [72, 73] là ứng viên kiểm chứ không phải kết luận nhập. Điều có thể nói hẹp hơn và vẫn hữu ích cho nhà cung cấp định cỡ nút mặt điều khiển. Áp dụng Rust ở đây tốn vài megabyte dấu chân cố định mỗi tiến trình, và đổi lấy thao tác lõi nhanh hơn (Mục 6.4.2) mà cấu trúc khối lượng công việc này không biến thành thông lượng dưới bão hoà, và thành thâm hụt phía trên.

Dấu kết quả đáng bảo vệ, vì một trong công trình tiền nhiệm gần nhất trong miền báo dấu ngược, trong khi hai cái khác đồng ý. Đồng ý trước, vì nói nhanh. Nghiên cứu thứ hai do Ericsson lưu trữ về phần mềm quản lý nút RAN thấy peak memory Rust thường cao hơn C [53], và cặp driver thiết bị user-space khớp ngoài viễn thông đặt dấu chân Rust khoảng 29% trên bản C [77]. Không cái nào tách bộ cấp phát khỏi ngôn ngữ, đó là phân biệt Mục 6.4.6 dựa vào, nên chúng ủng hộ hướng hiệu ứng chứ không cách nó chia.

Bất đồng là ca hướng dẫn hơn. Karlbäck và Orö đo peak resident set trên khối lượng RAN và thấy Rust *thấp nhất* trong ba ngôn ngữ và C++ cao nhất, khoảng cách nới khi đồng thời tăng. Họ gọi mẫu bất ngờ và quy tạm cho kích thước stack ban đầu của green thread [52]. Hai phát hiện tương thích khi rõ chúng đo đại lượng khác. C++ của họ chạy trên Boost.Fibers, cấp phát stack mỗi fiber, nên mười nghìn chuỗi đồng thời tốn mười nghìn stack. Rust của họ chạy trên Tokio, nơi task là future heap vài trăm byte. Số của họ do đó bắt chi phí mỗi ngữ cảnh của runtime đồng thời, đúng lý do nó tăng với số ngữ cảnh.

Thiết kế đo ở đây không có green thread hai bên. Pool cố định một luồng mỗi shard không để cấp phát mỗi ngữ cảnh khác, và còn lại là dấu chân cố định báo ở trên, hằng theo số UE chứ không tỷ lệ. Bất đồng của họ với kết quả này do đó giữa hai runtime đồng thời chứ không giữa hai ngôn ngữ, đó là minh chứng trực tiếp nhất tài liệu cho việc nghiên cứu này giữ mô hình đồng thời cố định. Nó cũng làm sắc điều kiện gắn với số hiện tại. Nó không cấp phép gì về hành vi bộ nhớ Rust dưới thiết kế một task mỗi UE, đó là thí nghiệm khác.

RQ4 (di chuyển). Nguyên mẫu cho thấy áp dụng tăng dần về mặt kỹ thuật rẻ. Host C++ có thể gọi lõi Rust không sửa qua đường nối ABI C mỏng có chi phí trên thao tác thật được giới hạn ở 10.91%, và mọi `unsafe` ở trong đường nối. Giới hạn đó từ so sánh khớp Mục 6.4.8, không từ đặt vượt rỗng so với vòng đầy đủ. Hai thứ không cùng thí nghiệm, và trên máy này một lượt không ghép mỗi bên đủ đảo thứ

tự biểu kiến. Chi phí chủ đạo do đó là con người chứ không phải máy: thiết kế bề mặt tương thích C và, như nghiên cứu áp dụng báo cáo [78, 79], leo đường cong học Rust. Điều đó biến rủi ro di chuyển thành mối quan tâm tổ chức chứ không phải hiệu năng, và hai áp dụng lớn nhất được ghi vào codebase C và C++ sẵn có báo cùng thứ tự chi phí [80, 81].

Một ranh giới trên kết quả đó phải vẽ rõ, vì cách đọc tự nhiên phóng đại nó. "Mọi `unsafe` gói trong đường nối" là tuyên bố về điều compiler xác minh, và đúng như đo. Crate lõi không chứa `unsafe`, và mọi thao tác con trỏ thô nằm trong 9 hàm có thể review. Nó không phải tuyên bố lõi được bảo vệ lúc chạy. Host và thành phần chia không gian địa chỉ, và lỗi bộ nhớ phía C++ có thể làm hỏng dữ liệu Rust dù phía Rust được kiểm cẩn thận. Đó là lý do biện pháp trong tài liệu cô lập hai heap bằng hỗ trợ phần cứng thay vì chỉ kỷ luật giao diện [47, 48], và lớp lỗi phát sinh tại ranh giới như vậy đã được phân loại riêng [49]. Di chuyển dạng này mua ít hơn "phần Rust an toàn". Nó mua thành phần đã di chuyển *ngừng đóng góp* lỗi an toàn bộ nhớ mới, trong khi vẫn lộ với lỗi của host cho đến khi host cũng di chuyển. Đó vẫn là hướng đi đúng, và là lý do xếp thứ tự di chuyển để các thành phần chia nhiều trạng thái mutable nhất cùng qua thay vì từng cái một.

Ba trục đo ngoài đường nối làm sắc cùng kết luận, và làm phức tạp hai phiên bản dân gian. Thời gian build là phản đối thường gặp nhất chống Rust trong codebase lớn, và nó chỉ sống sót ở tổng hợp. Build đầy đủ tốn 1.8 lần CPU. Nhưng 54% là mã bên thứ ba, nên mã nguồn riêng biên dịch bằng 0.8 lần những gì `g++` chi cho tương đương. Tổ chức lo ngại thời gian build nên nhìn ngân sách phụ thuộc trước compiler. Vòng sửa-biên dịch là trục C++ thực sự dẫn, và dẫn không đều. Lợi thế giữ với sửa trong đơn vị dịch và đảo với sửa header được include rộng.

Kích thước nhị phân hành xử tệ hơn lập luận dự đoán. Số 6.8× thường được trích dẫn so hai chính sách liên kết, và khi chính sách được giữ cố định thứ tự phụ thuộc cái nào được chọn. Trục quan trọng nhất cho codebase được quản lý cũng là trục có câu trả lời sạch nhất. Thành phần so sánh ở đây không giải quyết mã bên thứ ba và không chứa `unsafe`, nên lập luận phụ thuộc bắc cầu chống áp dụng Rust không chạm artefact đang bàn. Bề mặt chưa audit dự án mang thuộc bộ máy đo.

7.3 Xem lại các giả thuyết

Mục 1.3 cam kết ba giả thuyết trước mọi phép đo được lấy. Nêu rõ phán quyết của chúng, thay vì để kết quả tự nói, là điều phân biệt kiểm định với mô tả.

H1: được ủng hộ dưới bão hoà, bác bỏ phía trên. Giả thuyết có hai phần: hai ngôn ngữ tương đương hiệu năng dưới bão hoà, và mọi khác biệt chỉ ở đuôi xa. Phần đầu được ủng hộ. Phần hai bị bác bỏ, và không theo hướng cách diễn đạt dự đoán, vì lý do Mục 7.2 nêu. Đuôi xa dưới bão hoà, điều H1 thực sự đặt tên, vẫn chưa giải quyết trên máy này. Khác biệt xuất hiện nằm ở quá tải. Giả thuyết như viết sẽ dẫn chiến dịch nhìn sai chỗ, và kết quả chỉ

tìm được vì quét được đẩy qua bão hoà vì lý do phương pháp chứ không vì kỳ vọng nó.

Một phán quyết thứ hai là giả thuyết không được đặt tốt. "Tương đương hiệu năng" được viết như thể nó đặt tên một đại lượng. Mục 6.4.2 cho thấy nó đặt tên ít nhất ba. Có thao tác giao dưới bão hoà, nơi hai ngôn ngữ tương đương. Có mã compiler phát ra, nơi chúng không, và nơi đo đầu-cuối pha loãng khác biệt hơn hai bậc độ lớn. Và có công việc bền dưới quá tải, nơi chúng cũng không tương đương và khác biệt chạy chiều kia. Cả ba đều đúng, và giả thuyết không tách chúng là giả thuyết nghiên cứu này nên nêu cẩn thận hơn trước khi đo.

H2: được ủng hộ ở dạng chính xác, bác bỏ ở dạng lỏng. Giả thuyết là Rust an toàn ngăn lúc biên dịch "các lớp lỗi an toàn bộ nhớ và đồng thời mà C++ để cho phát hiện nỗ lực tốt nhất". Với bảy lớp tiêm loại đó nó đúng không ngoại lệ. Nghiên cứu tiêm lỗi vẫn bác bỏ cách đọc rộng mà cách diễn đạt mời gọi. Bảy lớp biên đúng loại C++ để cho phát hiện nỗ lực tốt nhất, Rust an toàn cũng không ngăn, và trên rò rỉ và tràn bản build release công cụ C++ là bên báo cáo. H2 do đó được chấp nhận cho an toàn bộ nhớ và tự do tranh chấp dữ liệu, và bị từ chối cho tính đúng đồng thời rộng. Nó cũng mang điều kiện tiền đề chưa nói, mà ca seam làm rõ: giả thuyết về Rust *an toàn*, và cùng lớp nó loại trừ trở lại ngay khi chương trình viết `unsafe`. Cách đọc thứ hai là điều thực hành thường giả định.

H3: được ủng hộ, và được kiểm đúng cách ở lần thử thứ hai. Giả thuyết là chi phí liên vận không đáng kể so với thao tác được gọi. Đo trực tiếp (Mục 6.4.8), không phát hiện chi phí và mọi thuế mỗi thao tác được giới hạn ở 10.91%. Điều ủng hộ giả thuyết là giới hạn, không phải không phát hiện khác biệt. Phiên bản đầu của phép đo này không mang điều đó. Nó đặt vượt rỗng so với vòng đầy đủ và suy câu trả lời, so hai thí nghiệm khác và sẽ cho kết luận đúng vì lý do sai. Tuyên bố kèm theo, chi phí thật là công sức kỹ thuật, được ủng hộ về cấu trúc chứ không thống kê. Đường nối nhỏ và cơ học, nhưng thiết kế bề mặt tương thích C là phần nguyên mẫu đòi quyết định thiết kế thực sự.

7.4 Hướng dẫn kỹ thuật cho áp dụng tăng dần

Kết quả nhằm dùng được, và nhà cung cấp cân nhắc Rust cho thành phần mặt điều khiển cần hơn một tập phép đo. Các hướng dẫn sau là dạng thực hành của bằng chứng trên. Mỗi cái gắn phát hiện ủng hộ nó, và giới hạn của ứng hỗ được nêu chứ không bóp. Hướng dẫn trích dẫn không kèm giới hạn là cách kết quả RQ2 thành "thành phần di chuyển an toàn" và RQ4 thành "di chuyển rẻ", và nghiên cứu này không thiết lập cái nào. Chúng theo thứ tự di chuyển đầu tiên sẽ chạy.

Chọn thành phần đầu theo giao diện, không theo kích thước. Chi phí runtime của ranh giới được giới hạn ở 10.91% một thao tác (RQ4), nên tiêu chí thành phần Rust đầu không phải nó nhỏ hay nóng đến đâu. Đó là giao diện hẹp và vô hướng đến đâu. Đường nối ở đây giữ 11 điểm vào vì API phân hệ đã hướng vòng đời, có kiểu, và không có cấu trúc mutable chia sẻ qua ranh giới. Thành phần giao diện truyền đối tượng C++ phong phú, template hay callback sẽ cần thiết kế bỏ chúng trước, và thiết kế lại đó là rủi ro lịch, không phải FFI.

Đặt ABI trong crate riêng và giữ lõi không `unsafe`. Tập trung mọi thao tác con trỏ thô trong crate đường nối riêng phụ thuộc lõi không sửa (Mục 6.4.8) đáng áp dụng như quy tắc, không coi là tình cờ của nguyên mẫu này. Nó giữ vấn đề lan phụ thuộc Evans et al. mô tả [43] xa mã quan trọng an toàn, và làm bề mặt có thể review của di chuyển thành thứ cố định, đếm được. Hai điều kiện đi kèm. Nó giới hạn điều phải *review*, không phải điều có thể *làm hỏng*. Khi host vẫn là C++ hai bên chia heap, nên đội cần tính chất mạnh hơn phải nhìn cô lập trong tiến trình [47, 48] chứ không chỉ kỷ luật giao diện. Và phần sinh của đường nối không miễn nghi ngờ, vì generator binding có lịch sử lỗi riêng [50]. Review nó như mã thay vì tin như đầu ra công cụ.

Đừng lập lại ngân sách review đồng thời dựa trên an toàn bộ nhớ. Điều này theo trực tiếp các ca biên. Áp dụng Rust loại bỏ use-after-free, double-free, vô hiệu hoá iterator và tranh chấp dữ liệu khỏi ngân sách lỗi hoàn toàn, và đó là các lớp chi phối thống kê bảo mật [8, 9]. Nó không bớt gì khỏi ngân sách deadlock và vi phạm tính nguyên tử. Kỷ luật thứ tự khóa, bất biến về khóa nào có thể giữ đồng thời, và review cho thao tác ghép nhưng không nguyên tử vẫn cần thiết như trong C++, và nên giữ trong checklist code review và chương trình design review sau di chuyển.

Ưu tiên cấu trúc không chia sẻ hơn chia sẻ-và-khóa, ở cả hai ngôn ngữ. Kết quả đồng thời mạnh nhất nghiên cứu này không thuộc ngôn ngữ nào. Thiết kế một luồng mỗi shard loại khóa khỏi đường nóng hoàn toàn, đó là lý do lỗi biên phải xây thành chương trình độc lập thay vì tìm trong hiện thực. Nơi trạng thái có thể phân vùng theo khóa tự nhiên, như trạng thái UE theo `ue_id`, phân vùng biến một lớp lỗi đồng thời thành không thể về cấu trúc cho cả hai ngôn ngữ, và làm vậy không cần trợ giúp compiler nào.

Ngân sách cho đường cong học tập, và đo nó. Chi phí chủ đạo xác định ở đây là con người, khớp nghiên cứu áp dụng báo cáo [78]. Luận văn này không định lượng chi phí đó, vì đây là nghiên cứu một tác giả, nên hướng dẫn tương ứng khiêm tốn. Coi thành phần đầu như đầu tư đào tạo với lịch cố ý rộng rãi, và gắn dụng cụ, ghi thời gian review và mật độ lỗi, để thành phần thứ hai lập kế hoạch trên công sức đo thay vì giả định.

Đo thành phần, không đo vòng, và đừng tổng quát hoá cả hai. Phát hiện hiệu năng mang cảnh báo phương pháp đi xa hơn số. Đo request/response đầu-cuối của thiết kế như thế này là 0.250% ngôn ngữ và phần còn lại scheduler (Mục 6.4.2). Dưới bão hoà nó do đó trả "không khác" giữa hầu hết hai hiện thực thành thạo, và đọc đó như bằng chứng về ngôn ngữ là sai lầm nghiên cứu này mắc trước khi đo thành phần riêng. Nhà cung cấp tự so sánh nên ngân sách cho cả hai: đo đầu-cuối, nói triển khai sẽ cảm thấy thế nào, và đo thành phần, nói ngôn ngữ đang làm gì.

Quét qua bão hoà, vì đó là nơi hai ngôn ngữ ngừng đồng ý. Đây là hướng dẫn nghiên cứu muốn đã theo từ đầu thay vì đến tới. Mọi kết luận về hiệu năng khi quét dừng ở tải danh nghĩa là null. Mỗi cái đúng ở điểm vận hành đó và im lặng về điểm hai hiện thực khác 1.8× (Mục 6.4.4). Điểm gối không là chi tiết ở rìa đồ thị. Với mặt điều khiển nó là vùng quyết định bão báo hiệu làm suy giảm dịch vụ hay kết thúc nó, và nơi thiết kế chặn có giới hạn không còn hành vi êm. So sánh dừng khi hệ thống vẫn đáp ứng tải đề nghị đã đo nửa dễ của câu hỏi. Quét qua điểm gối tốn thêm một buổi chiều thời gian máy.

Ngoài cả hai, artefact tái sử dụng là khung benchmark chứ không phải đầu ra hiện tại. Quyết định nó nên thông tin là "đo điều này trên thành phần của bạn, và đo qua điểm thành phần ngừng chịu tải", không phải "Rust đủ nhanh, tiếp tục".

7.5 Công trình liên quan

Chương 2 đặt nền công trình này đứng trên, theo bốn mảng. Phần sau định vị luận văn so với chúng rồi xử lý kỹ hai nghiên cứu nó phải trả lời. Điều quan trọng ở đây là tính chất bốn mảng chia sẻ.

Đo liên ngôn ngữ đặt ngôn ngữ hệ thống biên dịch gần nhau về thời gian và năng lượng [44, 82], và thấy an toàn Rust rẻ lúc chạy [26, 83]. Điều đó giữ nơi thời gian chặt nhất: hypervisor nhúng [84], hạ tầng đám mây sản xuất [21], và xử lý gói trong vài phần trăm so với tham chiếu C [71, 85]. Nơi chi phí xuất hiện nó đã được ghim xuống, vào biên, tràn và kiểm tra chia [72], vào cùng tập kiểm tra biên so với khả năng phần cứng [73], và vào anti-pattern lặp [86]. Bản port like-for-like một bộ từ C++ sang Rust đặt chi phí kiểm tra biên ở 34% của overhead trung bình hình học 46%. Nó cũng cảnh báo nơi compiler được ghi nhận loại bỏ kiểm tra rẻ, mã đo đôi khi khai báo hàm nóng `unsafe` [87]. Cảnh báo đó không áp dụng cho lõi không `unsafe` so sánh ở đây. Tất cả công trình này là trường hợp trung bình, và không cái nào báo hành vi qua điểm bão hoà, đó là chế độ luận văn này nhắm.

Bằng chứng an toàn cũng được giải quyết cùng cách một phần. Có nền tảng kiểm tra máy cho mảnh an toàn và cho kỷ luật aliasing mà mã `unsafe` phải tôn trọng [11, 39, 40]. Có xác nhận `unsafe` dùng tiết kiệm và CVE an toàn bộ nhớ luôn liên quan nó [27, 41, 42]. Có audit tìm lỗi ở quy mô dù vậy [88, 89, 90], và cảnh báo unsafety lan qua chuỗi phụ thuộc [43]. Và có báo cáo thúc đẩy chuyển dịch, đặt cạnh hồ sơ dài hỏng bộ nhớ chống lại giảm thiểu runtime [8, 9, 16, 18, 19, 33].

Mảng thứ ba điều kiện hoá lập luận áp dụng. Mục 2.6 nêu giới hạn trung tâm. Gói `unsafe` trong đường nối không bảo vệ phía Rust, vì mã nước ngoài trong cùng không gian địa chỉ vẫn có thể làm hỏng dữ liệu Rust [47, 48]. Đường nối có lớp lỗi riêng [49], và công cụ của nó có lỗi riêng [50]. Mô hình vượt qua nhẹ hơn tồn tại cho nhúng [91]. Điều mục đó không bao phủ là tầm với. Mẫu aliasing diễn đạt được trong C nhưng không trong Rust an toàn giới hạn dịch về nguyên tắc [92], liên vận là bài toán mở trung tâm trong Rust nhúng [93], và `unsafe` tập trung đúng các ranh giới này [94]. Hai áp dụng lớn nhất được ghi vào C và C++ sẵn có báo cáo ma sát đó bằng thực nghiệm [80, 81]. Tự động hoá viết lại là hướng đang hoạt động chứ không phải bài toán đã giải, vì translator hiện tại vẫn phát mã mà compiler không xác thực an toàn [95]. Đó là lý do chi phí di chuyển đo ở đây là chi phí đường nối viết tay.

Mảng thứ tư là miền và phương pháp: tài liệu độ trễ đuôi và quy mô micro giây [6, 7, 30, 96], cấu trúc một lõi mỗi luồng tự di chuyển đuôi [45], tách rời O-RAN [3], độ trễ mặt điều khiển là mục tiêu tích cực trong lõi mạng [97, 98], bằng chứng áp dụng trỏ vào đường cong học thay vì chi phí runtime [78, 79, 99], và hướng dẫn chuẩn về đánh giá lặp [32] và tiêm lỗi [59]. Rust có tiền lệ mạng di động ở datapath [100] và, phía mặt điều khiển, gNB-CU mã nguồn mở với CU-CP và CU-UP tách qua E1. Cái sau liên quan khoảng trống hơn phương pháp. Nó thiết lập bề mặt giao thức ASN.1 và SCTP không phải rào cản về nguyên tắc, nhưng nó là chứng minh tồn tại chứ không phải so sánh, không báo độ trễ và không mang đối tác C++ xây cùng đặc tả [101].

Bốn nghiên cứu hệ thống gần nhất về phương pháp, dù không về miền, vì chúng cũng xây cùng thành phần hai lần và quy khác biệt. Một so sánh cơ chế IPC hiện thực một chức năng mạng trong C++ và Rust [102]. RedLeaf được đánh giá so baseline C xây cho mục đích [103]. Bản dịch thủ công bộ benchmark giao dịch STAMP sang Rust là bản khớp công bố gần nhất với thiết kế nghiên cứu này, vì khối lượng đồng thời và cả hai phiên bản hiện thực một đặc tả [104]. Cái thứ tư đo driver tunnel UDP Rust trong kernel Linux so driver C tương đương [105]. Cái sau đáng xem lại vì phương pháp chứ không phải kết quả. Nó báo Rust chậm hơn 0,19% ở $p = 1{,}5 \times 10^{-15}$, khác biệt áp đảo về thống kê và không đáng kể về vận hành. Giữ hai phần tách là điều quy tắc khoảng-so với-p cam kết trước của Mục 4.4 tồn tại để làm. Như tài liệu rộng hơn, không cái nào trong bốn đo qua bão hoà.

Năm luận văn thạc sĩ, đều tại Ericsson, là công trình tiền nhiệm gần nhất theo miền và xứng được xử lý riêng. Không cái nào được phản biện, và không cái nào được lập chỉ mục bởi cơ sở dữ liệu trích dẫn nào, nên chúng được trích ở đây như hồ sơ nhà cung cấp đã thực sự thử chứ không phải kết quả đã thiết lập.

Karlbäck và Orö so sánh Rust và Go với C++ cho ứng dụng RAN trên x86 và ARM, kết hợp đo runtime với chỉ số hiệu quả phát triển, và kết luận C++ vẫn lựa chọn nơi hiệu năng quan trọng [52]. Kiểm soát của họ đáng trích nguyên văn chứ không diễn giải, vì đó là tuyên bố rõ nhất trong tài liệu này về nhiễu mà nghiên cứu hiện tại xây để loại. Tác giả ghi họ cố ý không viết cùng chương trình ba lần. Họ “nhằm giải pháp tốt nhất mỗi ngôn ngữ, thay vì cố làm mã giống nhau tối đa giữa ngôn ngữ”,

và họ "tránh dùng đúng cùng giải pháp mỗi ngôn ngữ" [52]. Với câu hỏi họ hỏi, đội viết mã idiom mỗi ngôn ngữ sẽ được gì, đó là lựa chọn đúng. Đó là kiểm soát sai cho câu hỏi ở đây. Runtime đồng thời rồi thay đổi theo ngôn ngữ (Boost.Fibers trên libevent, goroutine và Tokio), nên không khác biệt đo được chia giữa hai. Mục 7.2 quay lại với ca cụ thể, nơi kết quả tài nguyên của họ và kết quả báo ở đây có dấu đối, và bất đồng giải thành khác biệt runtime chứ không phải ngôn ngữ.

Olsson hỏi phiên bản hẹp hơn cùng câu hỏi hai năm sau, so Rust với C cho phần mềm quản lý nút RAN, và thấy tốc độ thực thi Rust ngang hoặc tốt hơn C, peak memory thường cao hơn, và an toàn cải thiện đáng kể [53]. Hai trong ba phát hiện đó đồng ý với kết quả báo ở đây, kể cả kết quả tài nguyên mà Karlbäck và Orö bất đồng với nghiên cứu này, được xử lý ở Mục 7.2. Điều nghiên cứu không làm là báo phần trăm, hay đẩy phần mềm qua sức chứa: so sánh là thời gian thực thi và peak memory ở tải ứng dụng chạy.

Olsson và Stenström là công trình tiền nhiệm gần nhất về mô hình đồng thời luận văn này áp dụng, và kết quả là minh hoạ sắc nhất trong tài liệu vì sao mô hình phải được giữ cố định. Họ xây bộ xử lý gói một lõi mỗi luồng trên Rust bất đồng bộ qua runtime Glommio, phản chiếu hai ứng dụng DPDK Eventdev hiện có, và đo thông lượng giữa 20% và 50% phiên bản DPDK với độ trễ họ mô tả tệ hơn nhiều [54]. Đọc như so sánh ngôn ngữ kết quả tệ hại. Đọc như điều nó là, so runtime bất đồng bộ trẻ với framework kernel-bypass tinh chỉnh một thập kỷ, nó gần như không nói gì về Rust và nói nhiều về runtime: hiệu ứng hai đến năm lần không phải quy mô mà sinh mã hoạt động. Đó cũng là lý do thiết kế đo ở đây dùng một luồng mỗi shard và không runtime bất đồng bộ hai bên, và Chương 8 ghi so sánh tách hai đúng cách.

Arvidsson và Kaller còn gần hơn. Họ đánh giá framework lập trình bất đồng bộ, C++ với Boost.Fibers, Rust với Tokio và Monoio, Go và Erlang, trên thủ tục mặt điều khiển 5G, và báo cả độ trễ trung vị và phần trăm thứ 99 qua quét tải [55]. Kết quả là thách thức trực tiếp nhất trong tài liệu với kỳ vọng luận văn này bắt đầu. Boost.Fibers có độ trễ thấp nhất mọi tải, và trong khi runtime Rust theo kịp ở trung vị, phần trăm thứ 99 của chúng tăng dốc hơn qua một tốc độ yêu cầu nhất định. Đó là phân kỳ đuôi đúng miền, và sẽ không trung thực nếu bỏ qua. Tuy nhiên đó cũng là so sánh mà runtime đồng biến với ngôn ngữ: tác giả quy hiệu ứng cho `async`/`await` không stack cho phép chuyển ngữ cảnh ngay trước khi thao tác I/O bắt đầu, đó là tính chất Tokio và Monoio chứ không phải Rust.

Sự quy kết đó không cần tin mù quáng, vì runtime sau đó được xem trực tiếp trong bối cảnh thời gian thực. Škoudlil et al. mổ mô hình thực thi của Rust bất đồng bộ so C++ dựa luồng tương đương. Họ báo chính sách lập lịch của runtime "phức tạp và khó hiểu vì lớp trừu tượng", và Tokio cụ thể "thiết kế tối đa hoá thông lượng thay vì tính xác định thời gian". Tái cấu trúc cùng ứng dụng lên luồng ưu tiên qua runtime khác cho thời gian phản hồi khớp phân tích thời gian phản hồi cổ điển, điều cấu hình work-stealing idiom không [46]. Công cụ xây quanh runtime chỉ cùng hướng. Lập lịch ưu tiên là thứ runtime Rust bất đồng bộ phải được thư viện cung cấp, không phải thứ nó đã có [106]. Hành vi đuôi đang bàn do đó là tính chất runtime, và đổi

khi runtime đổi. Nhiễu đó chính xác là điều đặc tả đóng băng và mô hình một luồng mỗi shard trong luận văn này thiết kế để loại. Hai nghiên cứu bổ sung chứ không mâu thuẫn. Của họ thiết lập khác biệt đuôi quan sát được trong miền này. Cái này được xây để xác định liệu ngôn ngữ có phải nguyên nhân.

Emberg và Wadsten gần nhất về phương pháp, và cách trung thực định vị luận văn là so với họ thay vì so các framework. Họ xây hai hiện thực một bộ giải mã ASN.1, C++ và Rust, cho cả hai gọi cùng mã C++ bên thứ ba, đến từ Rust bằng bọc C, và so hai bằng kiểm định rank-sum. Họ cũng phỏng vấn developer về trải nghiệm viết phía Rust [56]. Đó là cặp khớp, đường nối FFI, so sánh thống kê và đo công sức, tức bốn trong số việc luận văn này làm, cùng ngành, hai năm trước. Ba khác biệt còn lại, và chúng là điều mang khoảng trống. Đối tượng là biến đổi không trạng thái thay vì phân hệ đồng thời có trạng thái, nên không có mô hình đồng thời để giữ cố định và không trạng thái chia sẻ để tranh. Đo ở tải benchmark tình cờ tạo thay vì quét tới bão hoà và qua, nên không báo hành vi đuôi dưới quá tải. Và phía an toàn được lập luận từ phỏng vấn developer thay vì danh mục lỗi, nên không thiết lập lớp nào compiler thực sự từ chối.

Nghiên cứu của họ cũng cung cấp điều hữu ích hơn so sánh, đó là cảnh báo. Hai phiên benchmark của họ đảo phán quyết. Phiên đầu, sau code review loại thường làm trong thực tế, thấy hai hiện thực tương đương. Phiên thứ hai, sau khi hai bên tối ưu thêm, thấy bản Rust tệ hơn 85–91%. Tác giả lập luận phiên đầu đại diện hơn cho phần mềm nói chung, đó là cách đọc biện minh được. Quan sát quan trọng hơn là cặp khớp một tác giả có thể bị làm cho ra câu trả lời khác, tùy nỗ lực tối ưu rơi đâu. Mục 7.6 coi đó là mối đe doạ chính tới tính hợp lệ cấu trúc của chính nghiên cứu này và mô tả điều làm về nó.

Hai luận văn 2026 ngoài tập nhà cung cấp áp dụng phần phương pháp luận văn này cho đối tượng nhỏ hơn, và cả hai đáng ghi nhận vì đến câu trả lời tương thích độc lập [77, 107]. Không có đồng thời, khối lượng viễn thông hay quét tải, và cả hai là công trình cử nhân. Chúng được trích như bằng chứng hội tụ cho câu hỏi nhỏ hơn, không thay thế so sánh thử ở đây.

Bảng 7.2 đặt nghiên cứu này cạnh năm cái trực tiếp, và hai điều đáng đọc ra. Cột giữa cho thấy công trước không yếu hơn, chỉ nhắm khác. Bốn trong năm đo ngôn ngữ cùng framework nó viết trên, và cái thứ năm đo cặp khớp trên đối tượng không trạng thái ở tải cố định. Cột cuối cho thấy hệ quả. Rust có peak memory thấp nhất trong một nghiên cứu và cao hơn trong nghiên cứu khác. Ngang C trong một và bằng một phần năm thông lượng DPDK trong nghiên cứu khác. Theo kịp C++ ở trung vị trong một, và tương đương rồi tệ hơn nhiều qua hai phiên của một nghiên cứu. Các phát hiện không thể điều hoà với nhau, vì không hai cái giữ cố định cùng thứ. Đó là dấu hiệu tài liệu mà biến độc lập không được kiểm soát, và đó là lý do luận văn này xây đối tượng nghiên cứu thay vì tìm một.

Mục 2.7 ghi cả năm đến từ một nhà cung cấp. Đọc cạnh bảng, tập trung đó có hệ quả sắc hơn. “C++ vẫn lựa chọn nơi hiệu năng quan trọng” và “Rust bất đồng bộ chưa khả thi hiện tại” là phát hiện về phần mềm cụ thể trên hạ tầng cụ thể, và lặp

Bảng 7.2: Nghiên cứu này cạnh năm luận văn nhà cung cấp, đều tại Ericsson và không phản biện. Hàng sắp theo năm. Cột cuối báo phán quyết riêng của mỗi nghiên cứu theo thuật ngữ riêng, đó là lý do mục không khớp: chúng là câu trả lời câu hỏi khác, đo trên đối tượng và chế độ khác. Phán quyết ghi cho luận văn này là điều Mục 7.2 bảo vệ, kể cả phần máy đo không giải quyết. Hình 2.6 đặt cùng năm nghiên cứu trên trục khoảng trống thay vì, cạnh tài liệu đã phản biện.

Nghiên cứu	Đối tượng đo	Chế độ báo cáo	Phán quyết về Rust
Karlbäck & Orö, 2021 [52]	Ứng dụng RAN trên x86 và ARM, một hiện thực idiom mỗi ngôn ngữ	Runtime và tài nguyên ở tải thực hành	C++ được ưu tiên nơi hiệu năng quan trọng. Peak memory Rust thấp nhất trong ba ngôn ngữ
Olsson, 2023 [53]	Phần mềm quản lý nút RAN, Rust so C	Thời gian thực thi và peak memory	Tốc độ ngang hoặc tốt hơn C, peak memory cao hơn, an toàn cải thiện đáng kể
Olsson & Stenström, 2023 [54]	Xử lý gói: Rust bất đồng bộ trên Glommio so DPDK Eventdev C	Thông lượng và độ trễ theo số lõi	20–50% thông lượng DPDK với độ trễ tệ hơn nhiều. Rust bất đồng bộ được xét chưa khả thi hiện tại
Emberg & Wadsten, 2023 [56]	Bộ giải mã ASN.1 xây hai lần, phía Rust tới C++ qua wrapper C	Thời gian thực thi trong hai phiên benchmark	Tương đương phiên đầu, tệ hơn 85–91% phiên hai. Được ưu tiên thực tế vì lý do không hiệu năng
Arvidsson & Kaller, 2025 [55]	Thủ tục mặt điều khiển 5G qua năm framework bất đồng bộ	Trung vị và phần trăm thứ 99 qua quét tải	Trung vị theo C++. Phần trăm thứ 99 tăng dốc hơn qua ngưỡng tốc độ yêu cầu
Luận văn này, 2026	Dịch vụ Quản lý Ngữ cảnh UE xây hai lần từ một đặc tả đóng băng, mô hình đồng thời, bộ cấp phát và cấu hình build cố định	Trung vị, p99, p99.9 và max qua quét tới và qua bão hoà	Tương đương ở trung vị dưới bão hoà. Đuôi xa ở đó chưa giải quyết trên máy này. Qua bão hoà C++ duy trì 1.8× thông lượng của Rust

trong một tổ chức không thiết lập tính tổng quát. Câu hỏi do đó không được giải quyết nơi khác rồi nhập. Nó mở, chưa được khảo sát trong bối cảnh này, và câu trả lời ở đây là câu tổ chức độc lập có thể kiểm, vì đặc tả, cả hai hiện thực và bộ điều khiển được công bố cùng nó.

Không phải mọi bằng chứng cùng chiều, và phát hiện điều kiện được xử lý nghiêm ở đây như phát hiện thuận lợi. Ngay kết quả mạng tích cực ghi chi phí thật. Driver Rust idiom phát nhiều lệnh mỗi gói hơn tham chiếu C và chậm hơn đo được về tuyệt đối. Kiểm tra an toàn được CPU out-of-order hấp thụ, không hoàn toàn miễn phí [71]. Hai so sánh khớp khác chỉ cùng hướng trên khối lượng khác. Hiện thực Rust an toàn idiom của benchmark giao dịch STAMP chạy chậm hơn cả bản C gốc và Rust dịch máy [104], và nghiên cứu chung Rust so C báo đôi khi chậm hơn đáng kể [108]. Nghiên cứu lớn nhất Rust trong codebase C sẵn có đến kết luận tương tự từ hướng khác. Nó thấy tính năng an toàn Rust đưa overhead runtime và phức tạp phát triển khi không được quản lý cẩn thận [80].

Bằng chứng không chạy một chiều, dù vậy. Mã mức thấp an toàn không overhead runtime đã được chứng minh cho driver thiết bị mạng, cùng lớp chương trình Emmerich, khi nghĩa vụ an toàn được thực hiện qua bất biến kiểu thay vì kiểm tra [109]. Khung “zero-cost” [26] do đó nên đọc không phải đã thiết lập hay bác bỏ nói chung, mà là tính chất của một chương trình trên một khối lượng. Đó là lý do hành vi đuôi phải đo trên khối lượng mục tiêu chứ không giả định.

Câu chuyện an toàn cũng không tuyệt đối. Nghiên cứu thực nghiệm Rust thực tế thấy lỗi an toàn bộ nhớ và luồng vẫn xảy ra trong thực tế, trong đó lỗi chặn và đồng bộ lạm dụng [27, 90]. Chúng cũng thấy phần đáng kể hệ sinh thái không thể kiểm tra tĩnh đầy đủ, vì `unsafe` lan qua chuỗi phụ thuộc [43].

Các điều kiện này không lật đổ lập luận cho Rust, nhưng thúc đẩy hai lựa chọn thiết kế của luận văn. Thứ nhất là đo độ dự đoán và ngăn lỗi trực tiếp trên phân hệ thay vì nhập kết luận trung bình mục đích chung. Thứ hai là giữ lõi quan trọng an toàn không `unsafe`, để điều kiện lan phụ thuộc không áp dụng cho nó.

Ba trục tách luận văn này khỏi tất cả. Thứ nhất là liệu so sánh có kiểm soát thiết kế bằng cách xây cả hai hệ từ một đặc tả. Thứ hai là liệu nó có báo hành vi đuôi dưới quá tải thay vì hiệu năng trung bình. Thứ ba là liệu nó có đo chi phí áp dụng thay vì mô tả. Không trục nào trong ba trống một mình, như hiện thực khớp [56, 71, 102, 103, 104, 105, 109], quét phần trăm dừng ở yêu cầu độ trễ [55], và đo công sức [52, 78] trên hết cho thấy. Không nghiên cứu trước kết hợp ba cho khối lượng mặt điều khiển đồng thời có trạng thái, và đó là khoảng trống hẹp hơn công trình này chiếm.

7.6 Mối đe doạ tính hợp lệ và biện pháp giảm thiểu

Mọi kết luận ở đây quy khác biệt cho ngôn ngữ hiện thực. Độ tin cậy của nghiên cứu do đó dựa trên kiểm soát giữ mọi thứ khác bằng nhau, và trên tài khoản trung

thực nơi kiểm soát dừng. Bảng 7.3 ánh xạ mối đe doạ trước, theo loại suy luận mỗi cái làm nguy hiểm, để rủi ro còn lại đọc như một tập thay vì gom từng cái từ văn bản. Ba cái chịu tải. Nghiên cứu không thể loại trừ bất đối xứng kỹ năng một tác giả. Dụng cụ đầu-cuối sai cho phần RQ1 và RQ3 hỏi. Và phát hiện đơn mạnh nhất, phân kỳ quá tải, dựa trên quy kết không đo trực tiếp.

Tính hợp lệ nội tại: ngôn ngữ có thực sự là khác biệt duy nhất? Mối đe doạ trung tâm là thiên lệch hiện thực giữa hai phiên bản, giảm thiểu về cấu trúc bởi chiến lược biến kiểm soát Mục 5.1 và Bảng 4.2 và bởi kiểm thử tương đương cấp transcript Mục 5.6. Điều chưa kiểm soát là bất đối xứng kỹ năng một tác giả. Một người viết cả hai hiện thực và không thành thạo bằng nhau cả hai, nên lợi thế cho ngôn ngữ người đó biết hơn không thể loại trừ. Bộ tương đương giới hạn mối đe doạ này cho *hành vi* chứ không cho *hiệu năng*.

Rằng mối đe doạ thật chứ không chỉ có thể tưởng được được ghi trong cùng ngành, bởi hai phiên benchmark của Emberg và Wadsten đảo phán quyết của chính họ [56]. Một cặp khớp ủng hộ hai kết luận khác, tùy nỗ lực tối ưu rơi đâu. Kiểm soát ở đây chọn chống chế độ lỗi đó. Đặc tả đóng băng trước khi viết hiện thực, không đàm phán giữa chúng. Mức tối ưu và bộ cấp phát khớp, không tinh chỉnh mỗi bên. Và phân kỳ bắt buộc được ghi, không giải quyết theo hướng tác giả thấy dễ. Không điều nào giảm rủi ro còn lại về không. Nó làm rủi ro đó nhìn thấy, đó là lý do log idiom bắt buộc là sản phẩm chứ không phải phụ lục.

Tính hợp lệ cấu trúc: chỉ số có đo đúng câu hỏi? RQ1 được trả lời bằng phần trăm cao và max thay vì trung bình, đó là cách chuẩn đo độ dự đoán trong tài liệu này [6]. Hai rủi ro còn. Danh sách chỉ số thu hẹp thành bốn trong quá trình làm, vì jitter giữa thao tác và p99.99 được gắn công cụ nhưng sau chiến dịch báo cáo (Mục 6.4.1). Và p99.9 trên khối lượng tổng hợp không cùng đại lượng với deadline thủ tục bỏ lỡ trong mạng sống.

Khoảng cách cấu trúc lớn hơn trong RQ1 được tìm chứ không dự đoán. Độ trễ đầu-cuối quét báo cáo là 0.250% thao tác và phần còn lại bàn giao giữa luồng (Mục 6.4.2), nên nó đo chi phí giao của thiết kế trung thực và chi phí ngôn ngữ hầu như không. RQ1 hỏi cái nào phụ thuộc độc giả. Nhà cung cấp định cỡ nút muốn cái đầu, so sánh ngôn ngữ muốn cái thứ hai. Nghiên cứu báo cả hai và giữ chúng tách, nhưng câu hỏi ban đầu không tách chúng. Hai biện pháp được áp dụng, và không lượng thống kê nào thay thế được cả hai. Thao tác đo tách khỏi bàn giao, và quét được đẩy vào chế độ thiết kế ngừng hấp thụ khác biệt. Còn lại là microbenchmark một thao tác hẹp hơn đối tượng quét nó bổ sung.

Ba điều còn lại nêu trong Bảng 7.3 và giới hạn ở đây. Danh mục RQ2 là mẫu tác giả chọn. Bảy lớp biên giảm thiểu điều đó, vì buộc nó chứa ca kết luận kỳ vọng không bao phủ, và hai trong số đó trả kết quả ngược. Mười bốn chương trình vẫn là mười bốn chương trình. RQ3 bắt chi phí tài nguyên nhưng không nguồn gốc, vì không thu bộ đếm phần cứng hay hồ sơ cấp phát, nên nghiên cứu có thể nói phạt bộ nhớ

Bảng 7.3: Mối đe doạ tính hợp lệ theo loại suy luận mỗi cái làm nguy hiểm, kiểm soát áp dụng, và điều kiểm soát để lại. Chỉ mục cột cuối là rủi ro sống. Cột giữa là điều đã làm cho mỗi mối. Hàng không có kiểm soát nào, bộ đếm phần cứng thiếu sau RQ3, nói vậy thay vì làm mềm.

Mối đe doạ	Kiểm soát áp dụng	Phần còn lại
Nội tại: ngôn ngữ có thực sự là khác biệt duy nhất?		
Thiên lệch hiện thực giữa hai phiên bản	Đặc tả đóng băng, bộ cấp phát và mức tối ưu chung, hàm băm ghim, log idiom bắt buộc, kiểm thử tương đương cấp transcript	Bất đối xứng kỹ năng một tác giả. Tương đương giới hạn hành vi, không hiệu năng
Cấu trúc: chỉ số có đo đúng câu hỏi?		
Quét đo thiết kế, không ngôn ngữ	Thao tác đo tách khỏi bàn giao và cả hai được báo (§6.4.2)	Một thao tác trên một cấu trúc dữ liệu hẹp hơn quét nó bổ sung
Kết quả quá tải được quy kết nhưng không định vị	Thu hẹp hai mục log idiom bắt buộc (§6.4.4). Microbenchmark primitive tranh chấp (§6.4.5) loại hai cái đó một mình làm giải thích đầy đủ	Đường bàn giao đầy đủ dưới tải vẫn gồm điều phối variant và yêu cầu lớn hơn phía C++
Danh mục lỗi là mẫu tác giả chọn	Bảy lớp từ ngoài bảo đảm, kỳ vọng ghi trước	Mười bốn chương trình vẫn là mười bốn chương trình
RQ3 bắt chi phí tài nguyên nhưng không nguồn gốc	Không có trên máy này	Kết quả bộ nhớ thiết lập nhưng chưa giải thích
RQ4 đo đường nối, không an toàn toàn thể	Phân biệt được nêu mọi nơi trích kết quả	Một không gian địa chỉ: thuộc tính biên dịch phía Rust không giới hạn gì lúc chạy
Kết luận: khác biệt có thật hay nhiễu?		
Không xác định giữa các lần chạy	20 lần lặp xen kẽ, khoảng bootstrap, kiểm định hoán vị ghép cặp chính xác, hiệu chỉnh Holm, sàn được trích	Ghép cặp chỉ triệt tiêu điều khối chia sẻ, nên đuôi xa dưới bão hoà vẫn chưa giải quyết
p đã hiệu chỉnh không tái lập	Kiểm soát bộ cấp phát chạy mọi so sánh hai lần. Tái lập được theo p	Nó đã tốn một kết quả trước và sẽ lại. Ở đây hàng đuôi là hàng nó giữ lại
Lần chạy nhiễm đánh bại thống kê dựa trên trung bình	Quy tắc đọc §4.4, cố định trước, cộng số lần lặp ghép cặp đồng ý	Peak RSS dưới `jemalloc` đọc từ khoảng so p riêng
Ngoại tại: kết quả đi xa bao nhiêu?		
Một phân hệ, một khối lượng, một máy	RQ2 dựa compiler, RQ4 dựa compiler, ABI C và đồ thị phụ thuộc	Kết quả hiệu năng không đi xa, và quá tải đi xa nhất. Giây build RQ4 mô tả một máy, tỷ lệ thì không
Máy đo ảo hoá	Xen kẽ, ghép cặp và 20 lần lặp. Giới hạn khách ghi trong dữ liệu (§6.1)	Tần số không quan sát được và CPU steal khác không, không gỡ từ trong khách

xuất hiện chứ không vì sao. Cơ chế có khả năng nhất đã được cô lập nơi khác [72, 73]. RQ4 đo kích thước đường nối, `unsafe` nó tập trung, và chi phí mỗi lần gọi. Đó là đại lượng đúng cho câu hỏi như đặt, và chúng không đo hệ thống ghép an toàn đến đâu. Host và thành phần chia không gian địa chỉ, nên thuộc tính biên dịch phía Rust không giới hạn gì phía C++ có thể làm hỏng lúc chạy [47, 48].

Tính hợp lệ kết luận: khác biệt có thật hay nhiễu? Xử lý thống kê nêu ở Mục 4.4 và kết quả trong Bảng 6.2. Số lần lặp chọn để thiết kế không phải ràng buộc chính: kiểm định chính xác trên 20 cặp sàn ở 0.0000019, và ở 0.000027 sau hiệu chỉnh 14 so sánh, nên null ở đây phản ánh dữ liệu chứ không phải số học. Một lượt sớm bảy lần lặp không có tính chất đó, và sàn đã hiệu chỉnh trên 0,05 làm mọi null trong bảng không diễn giải được.

Ba hạn chế còn. Thứ nhất, phép đo lấy trong máy ảo không lộ tần số hay điều khiển turbo và ghi CPU steal khác không (Mục 6.1). Đuôi xa do đó chứa đóng góp từ hypervisor không tách khỏi phần mềm được kiểm thử. Đó là lý do đuôi xa dưới bão hoà báo chưa giải quyết chứ không phải null. Nó tương tác với ghép cặp theo cách đáng đặt tên. Ghép cặp gỡ nhiễu *chung* cho một lần lặp, đó là lý do thống kê cấp lần chạy như peak RSS, CPU và thông lượng bền vững đi qua sạch. Nó không gỡ nhiễu rơi một lần chạy mà không đối tác, và đó chính chế độ phần trăm đuôi sống. Đó cũng là lý do so sánh quá tải làm trên thông lượng chứ không trên độ trễ. Trung bình cấp lần chạy là thống kê ghép cặp có thể cứu trên host như thế này.

Thứ hai, p đã hiệu chỉnh dưới 0,05 cần thiết ở đây và rõ ràng chưa đủ, và điều ngược cũng đúng. Trong chiến dịch trước, trước khi hai build `jemalloc` giữ một cấu hình, sử dụng CPU vượt hiệu chỉnh Holm dưới một bộ cấp phát và đảo dấu dưới bộ kia. Không ngưỡng nào bắt được điều đó. Chỉ thiết kế chạy mọi so sánh hai lần mới thấy. Trong chiến dịch hiện tại kiểm soát không giữ gì ngưỡng sẽ thăng, dù hàng đuôi vẫn không vượt. Chiều ngược cũng có ca trong peak resident set size dưới `jemalloc` mô tả ở Mục 6.4.6. Ở đó khoảng xa không và 18 trong 20 lần lặp đồng ý đối p gần đơn vị, mà vài outlier thấp đủ tạo. Hàng đó đọc từ khoảng, theo quy tắc cố định Mục 4.4 chứ không chọn sau khi bất đồng xuất hiện. Hai phân cực đằng sau là khuyết điểm đo chứ không phải bộ cấp phát nào, vì ảnh hưởng cả hai ngôn ngữ cùng số lượng và xuất hiện mọi mức tải. Chẩn đoán để lại là công việc dụng cụ.

Thứ ba, phân kỳ quá tải được đo và tái lập, và cơ chế giờ được kiểm chứ không chỉ lập luận. Mục 6.4.5 cô lập hàng đợi có giới hạn và primitive hoàn thành dưới cùng mẫu 7 producer. Chúng không tái tạo thứ tự toàn dịch vụ ở mức ý nghĩa ($p = 0.226$). Điều đó loại hai cấu trúc đó một mình làm giải thích đầy đủ, dù chúng là nơi log idiom bắt buộc dự đoán tranh chấp. Nguyên nhân không không biết. Nó có thể gồm công việc thêm trên đường bàn giao C++, trong `Request` gắn thẻ lớn hơn, điều phối variant và thao tác bản đồ microbenchmark loại bỏ.

Tính hợp lệ ngoại tại: kết quả đi xa bao nhiêu? Nghiên cứu bao một phân hệ, một khối lượng, một cấu trúc đồng thời và một máy, và mỗi cái giới hạn tầm với

kết luận khác nhau. Kết quả an toàn và di chuyển (RQ2, RQ4) đi xa nhất, vì chúng là thuộc tính compiler và ABI C chứ không phải phân hệ này. Mọi codebase C++ viết các lớp lỗi này nhận cùng kết quả biên dịch, và mọi giao diện hẹp kiểu vô hướng nhận đường nối tương tự rẻ. Hai trục áp dụng RQ4 mang theo cùng cách, vì kích thước nhị phân và hình dạng đồ thị phụ thuộc theo toolchain. Trục thứ ba không đi xa như tuyệt đối, vì giây build thuộc máy đo chúng, và điều mang theo là tỷ lệ giữa hai toolchain đo luân phiên. *Hình dạng* thời gian build cũng đặc thù, theo cách đáng đặt tên. Thư viện C++ hai file và một crate Rust đều nhỏ, và kết quả tăng dần đặc biệt sẽ dịch chuyển trên codebase cấu trúc header khác.

Kết quả hiệu năng đi xa ít nhất. UECM được chọn cố ý nhỏ và có thể phân vùng, và thiết kế không chia sẻ loại tranh chấp khóa khỏi đường nóng. Phân hệ với trạng thái mutable chia sẻ thật, đồ thị gọi sâu hay áp lực cấp phát nặng do đó có thể kiểm tra chi phí trừu tượng và cấp phát hai ngôn ngữ khá khác. Khối lượng tổng hợp và được biện minh đặc tả, không phát lại từ traffic sản xuất, nên hỗn hợp thao tác hợp lý nhưng chưa xác thực. Không gì ở đây nói PHY hay xử lý L2, mặt người dùng nặng dữ liệu, hay mối quan tâm vận hành (nâng cấp, quan sát, đa tenant) mà triển khai sản xuất sẽ nêu.

Tuyên bố biện minh do đó hẹp và mang điều kiện. Đối với dịch vụ mặt điều khiển có trạng thái, đồng thời, có thể phân vùng loại này, và khi chúng đáp ứng tải đề nghị, lựa chọn ngôn ngữ không phải quyết định hiệu năng. Khi chúng không đáp ứng, lựa chọn ngôn ngữ là một. Và kết quả quá tải là phát hiện ít chuyển nhất trong nghiên cứu chứ không phải nhiều nhất, vì nó phụ thuộc giới hạn hàng đợi, một yêu cầu outstanding mỗi producer, và hai primitive thư viện chuẩn cụ thể. Mở rộng tuyên bố nào vượt lớp này nghĩa là chạy lại khung, đó là lý do khung chứ không phải số được đưa ra như đóng góp tái sử dụng.

Giới hạn cuối về tầm với là thời gian chứ không cấu trúc, và nó áp dụng cho bản thân so sánh chứ không một kết quả. Một trong hai đối tượng so sánh đang di chuyển. Phía C++ ghim C++23 như đặc tả và giao, và phiên bản compiler ghi Mục 6.1, trong khi ủy ban chuẩn đang theo đuổi profile an toàn bắt buộc (Mục 2.3). Điều này không điều kiện gì đo ở đây, vì không công việc đó chuẩn hoá hay triển khai rộng, và profile đồng thời chạm lớp biên RQ2 đến sau tập ban đầu vẫn chưa được đặc tả [38]. Nó có nghĩa kết quả RQ2 nên đọc như tuyên bố về hai ngôn ngữ như hiện tại, không phải vĩnh viễn. Profile vòng đời triển khai và bắt buộc sẽ thu hẹp nửa an toàn bộ nhớ danh mục. Profile đồng thời triển khai sẽ liên quan nửa đặt cố ý ngoài bảo đảm Rust. Một lần nữa khung là thứ sống sót. Danh mục được thiết kế để chạy lại với compiler bắt buộc profile, và câu trả lời khi đó được đo thay vì dự đoán.

Trạng thái tái lập: phát hiện nào đứng một mình. Mục 4.1 ghi không nghiên cứu trước đo so sánh này. Mệnh đề đối lập thuộc đây, giờ kết quả tồn tại: cái nào tài liệu rộng đồng ý. Tái lập nội bộ được thiết kế vào chiến dịch, trong 20 lần lặp xen kẽ và trong mọi so sánh chạy hai lần dưới bộ cấp phát khác. Tái lập ngoài là câu hỏi riêng, và câu trả lời không đều. Ba phát hiện có công ty đã xuất bản. Tương

đương trung vị dưới bão hoà là kết quả được ủng hộ nhất, với cặp khớp độc lập trong ba bối cảnh đặt Rust an toàn trong vài phần trăm đối tác C hoặc C++ [71, 102, 105]. Rằng bàn giao giữa luồng chứ không phải thao tác chi phối chi phí đầu-cuối là trường hợp của mẫu báo cáo cho cơ sở dữ liệu không chia sẻ trưởng thành [65]. Hầu hết lớp biên RQ2 là mối nguy tài liệu đã ghi. Ba phát hiện đứng một mình. Không nghiên cứu giao thức Phụ lục B tìm báo cáo đảo phụ thuộc tải giữa hai ngôn ngữ, và cái báo cùng hình dạng là luận văn thạc sĩ quy cho runtime thiết kế này loại trừ [55]. Không cái nào đo thời gian build hai toolchain, chưa kể tách cây phụ thuộc khỏi mã nguồn riêng. Giới hạn mỗi thao tác trên đường nối cũng không có đối tác công bố. Peak resident set size nằm giữa hai nhóm, với dấu tìm ở đây được hai luận văn lân cận báo cáo và đảo bởi thứ ba [52, 53, 77]. Chưa tái lập không có nghĩa không được ủng hộ. Điều nó đặt là mỗi kết quả mang được bao nhiêu trọng lượng một mình, và phân kỳ quá tải, hiệu ứng đơn lớn nhất đo ở đây, là phát hiện ít chứng cứ ngoài nhất.

7.7 Suy ngẫm đạo đức và chuyên môn

Tiến hành nghiên cứu không nêu câu hỏi bảo vệ dữ liệu. Nó không xử lý dữ liệu cá nhân, thuê bao hay nhà mạng bí mật. Mọi traffic là tổng hợp và do bộ điều khiển xác định tạo, ngữ cảnh mang định danh giả và trường bảo mật cố định giả, và phân hệ được mô hình theo mã sản xuất chứ không sao chép từ nó. Không mối quan tâm riêng tư, đồng ý hay lưu giữ dữ liệu phát sinh, nên artefact có thể phát hành công khai để tái tạo. *Cách dùng* nghiên cứu là chuyện khác và đáng nêu rõ, vì khuyến nghị chọn ngôn ngữ trong miền này không phải đầu ra kỹ thuật trung lập.

Hai trách nhiệm theo sau. Thứ nhất liên quan phóng đại tuyên bố. Nhà cung cấp có thể đọc hợp lý kết quả RQ2 như giấy phép mô tả thành phần di chuyển là “an toàn”, và ca biên là lý do cách đọc đó sai. Mặt điều khiển deadlock là sự cố, cập nhật bộ đếm mất im lặng có thể làm sai quyết định mobility, và không gì chứng minh ở đây ngăn cả hai. Báo cáo biên nổi bật như bảo đảm là nghĩa vụ chuyên môn chứ không phải lựa chọn phong cách, chính xác vì đối tượng công việc này đang ra quyết định ảnh hưởng mạng người ta phụ thuộc.

Thứ hai liên quan trọng lượng đặt lên bằng chứng host không hỗ trợ đầy đủ. Số độ trễ đuôi đo trong máy ảo với CPU steal khác không không phải cơ sở vững cho cam kết sản xuất. Nói vậy trong văn bản chỉ có tác dụng nếu điều kiện đi cùng số khi chúng được trích nơi khác. Đó là phần lý do bảng tóm tắt ghi trạng thái bằng chứng mỗi câu hỏi nghiên cứu chứ không chỉ phát hiện. Ngoài kết quả trực tiếp, khuyến nghị di chuyển cũng mang hệ quả tổ chức rơi lên người chứ không hệ thống: đào tạo lại, tuyển dụng và giai đoạn giảm vận tốc. Cách khung trung thực cho phát hiện RQ4 là chi phí chuyển từ thời gian chạy sang công sức con người, không bị xoá.

8 Kết luận và Công việc tương lai

Luận văn này so sánh C++ hiện đại và Rust trên một dịch vụ Quản lý Ngữ cảnh UE mô hình hoá theo một thành phần gNB-CU-CP thực của Nokia, theo bốn chiều: độ dự đoán được của độ trễ đuôi, ngăn chặn lỗi tại thời điểm biên dịch, chi phí mô hình an toàn, và chi phí áp dụng tăng dần. Xây dựng trên một đặc tả cố định duy nhất và chạy qua cùng các đường mã, hai bản hiện thực cho phép tách hiệu ứng ngôn ngữ khỏi khác biệt về thiết kế và kỹ năng.

Ba kết luận theo sau từ bằng chứng. Thứ nhất, về an toàn, khác biệt là khác biệt về *loại*, và nó có một cạnh xác định. Rust an toàn từ chối cả bảy lớp lỗi an toàn bộ nhớ và tranh chấp dữ liệu được tiêm vào tại thời điểm biên dịch, và một trong bảy lớp đó là lớp mà không bộ dò lỗi C++ nào báo cáo. Các mối nguy mà C++ chỉ có thể hy vọng bắt được trong một bài kiểm thử có gắn công cụ trở thành không thể viết. Cạnh đó dừng ở đó. Cả bảy lớp biên đều biên dịch và hành xử sai trong Rust an toàn đúng như trong C++, và trên hai lớp trong số đó công cụ C++ là bên báo cáo. Vì vậy bảo đảm được mô tả chính xác nhất như sau. Rust loại bỏ tranh chấp dữ liệu, không phải điều kiện đua, và lỗi aliasing, không phải lỗi thứ tự khóa, rò rỉ, số học quấn vòng hay ngăn chứa lỗi, và nó không loại bỏ lớp nào trong số đó ở vùng được đánh dấu `unsafe`. Ở nơi các lỗi biên được bắt, chúng được bắt tại thời điểm kiểm thử, trong hai trong ba trường hợp chỉ sau khi được viết lại theo các primitive của bộ kiểm tra mô hình, đó là một chi phí thực và khác với chi phí của bộ dò lỗi.

Thứ hai, về di chuyển chi phí chủ yếu do thiết kế API và đường cong học tập, không phải bởi bất kỳ thuế thời gian chạy nào. Một lõi Rust không sửa đổi có thể được điều khiển từ C++ qua một đường nối mỏng, cô lập `unsafe`, có chi phí trên một thao tác thực, đo so với cùng thao tác gọi bản địa, được giới hạn ở 10.91%. Các chi phí mà kỹ sư build thường nêu trước không đổi thứ tự đó. Phần lớn chi phí build thêm của Rust là biên dịch phụ thuộc chứ không phải công việc của chính trình biên dịch, khoảng cách kích thước nhị phân là chính sách liên kết và đảo chiều khi chính sách được khớp, và thành phần được so sánh không giải quyết mã bên thứ ba nào. Nơi C++ dẫn trước là vòng lặp sửa-biên dịch, và nó chỉ dẫn trước với các sửa đổi nằm trong một đơn vị dịch.

Thứ ba, về hiệu năng bằng chứng chia đôi hai lần trước khi chia theo tài nguyên. Nó chia trước theo *cái gì đang được đo*, rồi theo *liệu dịch vụ còn chịu tải được*. Làm đúng cả hai phép chia là chỉnh sửa quan trọng nhất của nghiên cứu đối với chính nó, và mỗi phép được phát hiện chứ không phải dự đoán trước. Một thao tác đầu-cuối trong thiết kế này là 0.250% công việc ngôn ngữ thực hiện, phần còn lại là bàn giao giữa luồng mà hệ điều hành thực hiện, nên một kết quả null đầu-cuối không thể giới hạn hiệu ứng ngôn ngữ. Đo riêng thao tác thì hai ngôn ngữ không tương đương. Rust nhanh hơn, theo 1.6×, vì lựa chọn bố cục bảng băm của thư viện chuẩn chứ không vì điều gì liên quan đến an toàn.

Phép chia thứ hai là theo chế độ, và nó quan trọng hơn. Dưới bão hoà hai ngôn ngữ tương đương ở trung vị cho mọi mục đích triển khai quan tâm, và đó là kết quả thực về các triển khai dạng này. Tương đương là thực tiễn chứ không phải thống kê, vì kiểm định ghép cặp vẫn phân giải 1.6 μs nghiêng về Rust dưới cả hai bộ cấp phát. Đuôi xa của chúng vẫn không phân giải được trên máy ảo hoá, đó là thừa nhận về dụng cụ đo. Qua bão hoà chúng không tương đương. Bản build C++ duy trì 1.8× thông lượng của bản build Rust, trong 20/20 lần lặp xen kẽ dưới cả hai bộ cấp phát, trong khi tiêu thụ 1.9× CPU trên mỗi thao tác hoàn thành, và các primitive cô lập không tái tạo thứ tự đó ở mức ý nghĩa (Mục 6.4.5). Về bộ nhớ bản build Rust mang thêm vài MiB dấu chân cố định dưới cả hai bộ cấp phát, nhỏ hơn một bậc độ lớn so với chi phí của chính lựa chọn bộ cấp phát.

Gộp lại, chúng ủng hộ một tuyên bố hẹp, hẹp hơn tuyên bố luận văn này dự kiến đưa ra. Đối với các dịch vụ mặt điều khiển có trạng thái, đồng thời, có thể phân vùng dạng này, và *ở tải danh nghĩa*, lựa chọn giữa C++ hiện đại và Rust không phải quyết định hiệu năng. Đó là quyết định về lớp lỗi nào tổ chức muốn làm không thể viết, và liệu tổ chức có trả chi phí kỹ thuật và học tập để đạt được hay không. Dưới quá tải nó lại thành quyết định hiệu năng, và hiện nay ủng hộ C++. Lý do nằm ở hạ tầng đồng thời chứ không phải bảo đảm của ngôn ngữ nào, và dịch vụ bắt buộc phải suy giảm êm không thể coi chúng là phụ. Điều luận văn này bổ sung do đó là một dụng cụ đo chứ không phải phán quyết: một đặc tả cố định, một cặp hiện thực đã kiểm thử tương đương, một khung đo có thể tái lập vượt qua điểm gối bão hoà, một phân rã nói phần nào trong số đo đó ngôn ngữ phải chịu trách nhiệm, và một danh mục lỗi ánh xạ cả phần bảo đảm bao phủ lẫn nơi nó dừng.

Một số hướng mở rộng công việc, mỗi hướng là biện pháp trực tiếp cho một hạn chế đã nêu ở trên. Ưu tiên tức thời là chiến dịch bare-metal, mục tiêu giờ đã hẹp: điểm gối bão hoà và so sánh quá tải đã được giải quyết, nên bare-metal sẽ mang lại đuôi xa dưới bão hoà. Bộ đếm phần cứng và hồ sơ cấp phát thuộc cùng chuyến đo và là khoảng trống dụng cụ đo rõ nhất trong nghiên cứu. Chúng sẽ biến kết quả bộ nhớ của Mục 6.4.6 từ quan sát thành giải thích, và xác nhận hoặc bác bỏ lời giải thích về cục bộ bộ nhớ đệm mà Mục 6.4.2 đưa ra cho khoảng cách ở mức thao tác. Ngoài ra:

- Phát lại các trace sản xuất đã ẩn danh hoá thay cho khối lượng công việc tổng hợp, để hỗn hợp thao tác được xác thực chứ không chỉ được biện minh bằng đặc tả.
- Mở rộng so sánh sang thêm các phân hệ mặt điều khiển, hoặc sang đường L2/PHY nặng dữ liệu, nơi trạng thái dùng chung có thể thay đổi và áp lực cấp phát sẽ kiểm tra hai ngôn ngữ khác với thiết kế không chia sẻ.
- Mở rộng danh mục lỗi dọc theo biên được ánh xạ ở đây. Lạm dụng `unsafe`, và các lớp biên dưới bộ kiểm tra mô hình toàn chương trình thay vì mô hình thu nhỏ thủ công, là các bước tiếp theo hiển nhiên. Một bộ kiểm tra bao phủ chương trình Rust cùng phụ thuộc nước ngoài của nó [51] sẽ chạm đường nối

của Mục 6.4.8, mà nghiên cứu này đo chi phí nhưng không khám phá các xen kẽ của nó.

- Đánh giá runtime async của Rust so với mô hình một luồng mỗi shard dùng ở đây, dưới các kiểm soát nghiên cứu này áp đặt. Điều đó sẽ tách sự phân kỳ đuôi mà Arvidsson và Kaller quy cho `async/await` không stack [55] khỏi bản thân ngôn ngữ, và sẽ xác định bao nhiêu phần khoảng cách Olsson và Stenström đo giữa Rust bất đồng bộ và DPDK [54] thuộc runtime chứ không phải framework.
- Đo chi phí cô lập trong tiến trình hai heap tại đường nối, vì lõi không `unsafe` cần được bảo vệ cũng như được xác minh [47, 48].
- Theo dõi mật độ lỗi và công sức review khi Rust được áp dụng tăng dần trong mã nguồn RAN sản xuất.

Statement on the Use of AI Tools

Tool. Anthropic's Claude Opus 5, used through the Cursor development environment, is the only generative AI system used in preparing this thesis, and it was used throughout, from June to November 2026. It is named neither as an author, which University of Helsinki policy forbids, nor as a source, which is the author's own choice.

What the assistance covered. It was extensive. The model drafted and revised the running prose and proposed the order in which the arguments are made. It wrote and refactored both implementations of the UE Context Management service, the load generator, the fault-injection catalogue, the interoperability prototype, the analysis code behind the bootstrap intervals and the paired permutation tests, and the figure-generating, build and LaTeX sources. It also suggested candidate literature. The abstract is the one part it must not touch: University of Helsinki policy never permits a language model in a maturity test, and in this programme the maturity test is the abstract. In this draft it has not yet been rewritten unaided, and this statement will say so until it has been.

What the model did not settle. The research problem, the four research questions, the three hypotheses, and the control on which the study's value rests are the author's. That control is one frozen language-neutral specification and one concurrency model, so language remains the only systematic difference between the two implementations. The author also chose to carry defect classes from outside Rust's compile-time guarantee, which is what keeps RQ2 falsifiable. Every measurement was run by the author on the hosts documented here, and the quantities the thesis quotes reach the page as macros regenerated from the raw logs rather than as figures typed into a paragraph. The readings are the author's too, including the two restraints the evidence imposes: an unresolved far tail is not equivalence, and a sweep dominated by cross-thread handoff cannot bound an effect of the language.

How the output was checked. Model output was treated as a draft to be falsified. Every bibliography entry was resolved against Crossref, DataCite, or the issuing body and traced to the page or clause carrying the claim made of it. That audit found four passages that overstated or misplaced their source, one a quotation absent from the work it was attributed to. All four were corrected, and the audit is kept beside the artefacts. Code was checked by running it, against a conformance suite whose transcripts are diffed line by line and a fault catalogue that fixes each expected outcome before the defect is injected.

What the author took from it. The model is dependable at mechanical work, at transposing an idiom between two languages or keeping a build honest. It is undependable in one predictable way: it will supply a plausible citation for a claim and a confident sentence where the data supports a hedge. Its fluency is not evidence. The effort saved in drafting is spent again in checking, and the defensible parts of

this thesis are the parts checked against a source, a specification, or a measurement.

Responsibility. The author is responsible for the whole of this thesis. Where it is wrong, the error is the author's.

Tài liệu tham khảo

[1] 3GPP. *NG-RAN; Architecture description*. Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip.

[2] 3GPP. *NR; NR and NG-RAN Overall description; Stage 2*. Technical Specification (TS) 38.300. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.300/38300-h50.zip.

[3] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia. "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges". In: *IEEE Communications Surveys & Tutorials* 25.2 (2023), pp. 1376–1411. DOI: 10.1109/COMST.2023.3239220. URL: https://doi.org/10.1109/COMST.2023.3239220.

[4] 3GPP. *NR; Radio Resource Control (RRC); Protocol specification*. Technical Specification (TS) 38.331. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), July 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.331/38331-h50.zip.

[5] 3GPP. *Study on scenarios and requirements for next generation access technologies*. Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip.

[6] J. Dean and L. A. Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[7] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. "Attack of the Killer Microseconds". In: *Communications of the ACM* 60.4 (2017), pp. 48–54. DOI: 10.1145/3015146. URL: https://doi.org/10.1145/3015146.

[8] M. Miller. *Trends, challenge, and shifts in software vulnerability mitigation*. BlueHat IL, Microsoft Security Response Center (MSRC). Approximately 70% of Microsoft CVEs are memory-safety issues. 2019. URL: https://github.com/microsoft/MSRC-Security-Research/tree/master/presentations/2019_02_BlueHatIL.

[9] J. Vander Stoep. *Memory Safe Languages in Android 13*. Google Security Blog. Dec. 2022. URL: https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html.

[10] S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6.

[11] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 66:1–66:34. DOI: 10.1145/3158154. URL: https://doi.org/10.1145/3158154.

[12] ITU-R. *Framework and overall objectives of the future development of IMT for 2030 and beyond*. Recommendation ITU-R M.2160-0. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline. International Telecommunication Union, Radiocommunication Sector, Nov. 2023. URL: https://www.itu.int/rec/R-REC-M.2160/en.

[13] M. Polese, N. Mohamadi, S. D'Oro, L. Bonati, and T. Melodia. "Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G". In: *IEEE Communications Magazine* 64.7 (2026), pp. 90–95. DOI: 10.1109/MCOM.001.2500438. URL: https://doi.org/10.1109/MCOM.001.2500438.

[14] AI-RAN Alliance. *AI-RAN Platform and Infrastructure Orchestrator*. AI-RAN Alliance, Working Group 2 (AI-and-RAN). Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes that the two kinds of workload may have different service-level agreements. Con-

tributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks and Northeastern University. Accessed August 2026. Oct. 2025. URL: https://ai-ran.org/documents/AI-RAN-Platform-Infrastructure-Orchestrator.pdf.

[15] J. Vander Stoep. *Rust in Android: Move Fast and Fix Things*. Google Security Blog. Nov. 2025. URL: https://blog.google/security/rust-in-android-move-fast-fix-things/.

[16] A. Rebert and C. Kern. *Secure by Design: Google's Perspective on Memory Safety*. Google Security Engineering, Technical Report. 2024. URL: https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/.

[17] R. N. M. Watson, J. Baldwin, D. Chisnall, T. Chen, J. Clarke, B. Davis, N. W. Filardo, B. Gutstein, G. Jenkinson, B. Laurie, A. Mazzinghi, S. W. Moore, P. G. Neumann, H. Okhravi, A. Richardson, A. Rebert, P. Sewell, L. Tratt, M. Vijayaraghavan, H. Vincent, and K. Witaszczyk. "It Is Time to Standardize Principles and Practices for Software Memory Safety". In: *Communications of the ACM* 68.2 (2025), pp. 40–45. DOI: 10.1145/3708553. URL: https://doi.org/10.1145/3708553.

[18] Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously*. Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[19] Office of the National Cyber Director. *Back to the Building Blocks: A Path Toward Secure and Measurable Software*. Technical Report. Washington, DC: The White House, Office of the National Cyber Director (ONCD), Feb. 2024. URL: https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf.

[20] European Parliament and Council of the European Union. *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union, OJ L, 2024/2847. Nov. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[21] A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. "Firecracker: Lightweight Virtualization for Serverless Applications". In: *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2020, pp. 419–434. URL: https://www.usenix.org/conference/nsdi20/presentation/agache.

[22] Dell'Oro Group. *RAN Mostly Stable in 3Q 2025*. Dell'Oro Group, RAN Quarterly Report summary. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25 RAN market, up from 95% in 2024. Accessed August 2026. Nov. 2025. URL: https://www.delloro.com/news/ran-mostly-stable-in-3q-2025/.

[23] Dell'Oro Group. *RAN Market Stabilized in 2025*. Dell'Oro Group, RAN Quarterly Report summary. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index reached a ten-year high. Accessed August 2026. Feb. 2026. URL: https://www.delloro.com/news/ran-market-stabilized-in-2025/.

[24] J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work; ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road.

[25] Cybersecurity and Infrastructure Security Agency. *Product Security Bad Practices*. Joint Guidance. Version 2.0. Bad practice 1 covers the development of new product lines in memory-unsafe languages and the memory-safety roadmap expectation, from which only products with an announced end-of-support date before 1 January 2030 are exempt. CISA and FBI, Jan. 2025. URL: https://www.cisa.gov/resources-tools/resources/product-security-bad-practices.

[26] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk. "System Programming in Rust: Beyond Safety". In: *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*. 2017, pp. 156–161. DOI: 10.1145/3102980.3103006. URL: https://doi.org/10.1145/3102980.3103006.

[27] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. "Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2020, pp. 763–779. DOI: 10.1145/3385412.3386036. URL: https://doi.org/10.1145/3385412.3386036.

[28] 3GPP. *Service requirements for the 5G system; Stage 1*. Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip.

[29] 3GPP. *NG-RAN; E1 Application Protocol (E1AP)*. Technical Specification (TS) 38.463. V17.0.0, Release 17. 3rd Generation Partnership Project (3GPP), Apr. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.463/38463-h00.zip.

[30] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble. "Tales of the Tail: Hardware, OS, and Application-level Sources of Tail Latency". In: *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*. 2014, 9:1–9:14. DOI: 10.1145/2670979.2670988. URL: https://doi.org/10.1145/2670979.2670988.

[31] B. Schroeder, A. Wierman, and M. Harchol-Balter. "Open Versus Closed: A Cautionary Tale". In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale.

[32] A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033.

[33] L. Szekeres, M. Payer, T. Wei, and D. Song. "SoK: Eternal War in Memory". In: *Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P)*. 2013, pp. 48–62. DOI: 10.1109/SP.2013.13. URL: https://doi.org/10.1109/SP.2013.13.

[34] S. Bensalem and K. Havelund. "Dynamic Deadlock Analysis of Multi-threaded Programs". In: *Hardware and Software, Verification and Testing (Haifa Verification Conference)*. Vol. 3875. Lecture Notes in Computer Science. Springer, 2005, pp. 208–223. DOI: 10.1007/11678779_15. URL: https://doi.org/10.1007/11678779_15.

[35] The Sanitizers Project. *ThreadSanitizer deadlock detector*. Project documentation. Accessed August 2026. 2026. URL: https://github.com/google/sanitizers/wiki/ThreadSanitizerDeadlockDetector.

[36] H. Sutter. *Core Safety Profiles for C++26*. ISO/IEC JTC1/SC22/WG21 document P3081R2. Feb. 2025. URL: https://isocpp.org/files/papers/P3081R2.pdf.

[37] S. Baxter and C. Mazakas. *Safe C++*. ISO/IEC JTC1/SC22/WG21 document P3390R0. Sept. 2024. URL: https://safecpp.org/P3390R0.html.

[38] D. Vandevoorde, J. Garland, P. E. McKenney, R. Orr, B. Stroustrup, and M. Wong. *Profiles and Safety: A Call to Action*. ISO/IEC JTC1/SC22/WG21 document P3970R0. Jan. 2026. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p3970r0.pdf.

[39] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "Safe Systems Programming in Rust". In: *Communications of the ACM* 64.4 (2021), pp. 144–152. DOI: 10.1145/3418295. URL: https://doi.org/10.1145/3418295.

[40] R. Jung, H.-H. Dang, J. Kang, and D. Dreyer. "Stacked Borrows: An Aliasing Model for Rust". In: *Proceedings of the ACM on Programming Languages* 4.POPL (2020), 41:1–41:32. DOI: 10.1145/3371109. URL: https://doi.org/10.1145/3371109.

[41] V. Astrauskas, C. Matheja, F. Poli, P. Müller, and A. J. Summers. "How Do Programmers Use Unsafe Rust?" In: *Proceedings of the ACM on Programming Languages* 4.OOPSLA (2020), 136:1–136:27. DOI: 10.1145/3428204. URL: https://doi.org/10.1145/3428204.

[42] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. "Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs". In: *ACM Transactions on Software Engi-*

neering and Methodology 31.1 (2022), 3:1–3:25. DOI: 10.1145/3466642. URL: https://doi.org/10.1145/3466642.

[43] A. N. Evans, B. Campbell, and M. L. Soffa. "Is Rust Used Safely by Software Developers?" In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 246–257. DOI: 10.1145/3377811.3380413. URL: https://doi.org/10.1145/3377811.3380413.

[44] W. Bugden and A. Alahmar. "Rust: The Programming Language for Safety and Performance". In: *arXiv preprint arXiv:2206.05503* (2022). DOI: 10.48550/arXiv.2206.05503. URL: https://doi.org/10.48550/arXiv.2206.05503.

[45] P. Enberg, A. Rao, and S. Tarkoma. "The Impact of Thread-per-Core Architecture on Application Tail Latency". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874.

[46] M. Škoudlil, M. Sojka, and Z. Hanzálek. "A First Look at ROS 2 Applications Written in Asynchronous Rust". In: *37th Euromicro Conference on Real-Time Systems (ECRTS 2025)*. Vol. 335. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 1:1–1:21. DOI: 10.4230/LIPIcs.ECRTS.2025.1. URL: https://doi.org/10.4230/LIPIcs.ECRTS.2025.1.

[47] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. "Keeping Safe Rust Safe with Galeed". In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: 10.1145/3485832.3485903. URL: https://doi.org/10.1145/3485832.3485903.

[48] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz. "PKRU-Safe: Automatically Locking Down the Heap Between Safe and Unsafe Languages". In: *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*. 2022, pp. 132–148. DOI: 10.1145/3492321.3519582. URL: https://doi.org/10.1145/3492321.3519582.

[49] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33.

[50] X. Cai, J. Liu, C. Liu, L. Bao, Y. Yu, and L. Jiang. "Fortifying the Seams Between C/C++ and Rust: Characterizing Bugs in Interop Tools". In: *ACM Transactions on Software Engineering and Methodology* (2026). DOI: 10.1145/3795532. URL: https://doi.org/10.1145/3795532.

[51] O. Pearce, J. Lange, and D. O'Keeffe. "RustMC: Automated Verification of Real-World Concurrent Rust". In: *Formal Techniques for Distributed Objects, Components, and Systems (FORTE)*. Lecture Notes in Computer Science. Springer, 2026, pp. 235–255. DOI: 10.1007/978-3-032-28187-6_14. URL: https://doi.org/10.1007/978-3-032-28187-6_14.

[52] R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307.

[53] V. Olsson. "Rust Programming Language as an Alternative to C for RAN Management Applications". Carried out at Ericsson AB. Master's thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822.

[54] J. Olsson and I. Stenström. "Rust Async for Thread-per-Core Packet Processing". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029.

[55] A. Arvidsson and M. Kaller. "Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746.

[56] C. Emberg and A. Wadsten. "Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages". Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552.

[57] R. Gjomemo, P. H. Phung, E. Ballou, K. S. Namjoshi, V. N. Venkatakrishnan, and L. Zuck. "Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers". In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are affiliated with Nokia. 2016, pp. 323–334. DOI: 10.1109/QRS.2016.44. URL: https://doi.org/10.1109/QRS.2016.44.

[58] S. Michaelides, J. Lapawa, D. Eguiguren Chavez, and M. Henze. "Evaluation of Security-Induced Latency on 5G RAN Interfaces and User Plane Communication". In: *Proceedings of the 19th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec '26)*. 2026, pp. 176–181. DOI: 10.1145/3765613.3797453. URL: https://doi.org/10.1145/3765613.3797453.

[59] R. Natella, D. Cotroneo, and H. S. Madeira. "Assessing Dependability with Software Fault Injection: A Survey". In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425.

[60] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7. DOI: 10.1007/978-1-4899-4541-9. URL: https://doi.org/10.1007/978-1-4899-4541-9.

[61] C. Demichelis and P. Chimento. *IP Packet Delay Variation Metric for IP Performance Metrics (IPPM)*. Request for Comments (RFC) 3393. Standards Track; defines packet delay variation as the difference between consecutive one-way delays. Internet Engineering Task Force (IETF), Nov. 2002. DOI: 10.17487/RFC3393. URL: https://www.rfc-editor.org/rfc/rfc3393.

[62] S. Holm. "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733.

[63] G. Tene. *HdrHistogram: A High Dynamic Range Histogram*. Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram.

[64] T. Yin. *Lizard: An extensible cyclomatic complexity analyzer for many programming languages*. Version 1.23.0. Accessed August 2026. 2026. URL: https://github.com/terryyin/lizard.

[65] X. Zhou, V. Leis, X. Yu, and M. Stonebraker. "OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck". In: *15th Annual Conference on Innovative Data Systems Research (CIDR 2025)*. 2025. URL: https://www.vldb.org/cidrdb/papers/2025/p17-zhou.pdf.

[66] The LLVM Project. *clang-tidy check `bugprone-use-after-move`*. Extra Clang Tools documentation. Accessed August 2026. 2026. URL: https://clang.llvm.org/extra/clang-tidy/checks/bugprone/use-after-move.html.

[67] J. Hughes and L. Tratt. "Garbage Collection for Rust: The Finalizer Frontier". In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2 (2025), pp. 3588–3614. DOI: 10.1145/3763179. URL: https://doi.org/10.1145/3763179.

[68] M. Cui, H. Xu, H. Tian, and Y. Zhou. "rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust". In: *IEEE Transactions on Software Engineering* 50.9 (2024), pp. 2472–2484. DOI: 10.1109/TSE.2024.3443624. URL: https://doi.org/10.1109/TSE.2024.3443624.

[69] W. Dietz, P. Li, J. Regehr, and V. Adve. "Understanding Integer Overflow in C/C++". In: *ACM Transactions on Software Engineering and Methodology* 25.1 (2015), 2:1–2:29. DOI: 10.1145/2743019. URL: https://doi.org/10.1145/2743019.

[70] J. Bornholt, R. Joshi, V. Astrauskas, B. Cully, B. Kragl, S. Markle, K. Sauri, D. Schleit, G. Slatton, S. Tasiran, J. Van Geffen, and A. Warfield. "Using Lightweight Formal Methods

to Validate a Key-Value Storage Node in Amazon S3". In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 836–850. DOI: 10.1145/3477132.3483540. URL: https://doi.org/10.1145/3477132.3483540.

[71] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. "The Case for Writing Network Drivers in High-Level Programming Languages". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892.

[72] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. "Towards Understanding the Runtime Performance of Rust". In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2022, 140:1–140:6. DOI: 10.1145/3551349.3559494. URL: https://doi.org/10.1145/3551349.3559494.

[73] S. Harris, S. Cooksey, M. Vollmer, and M. Batty. "Rust for Morello: Always-On Memory Safety, Even in Unsafe Code (Experience Paper)". In: *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP)*. Vol. 263. Leibniz International Proceedings in Informatics (LIPIcs). 2023, 39:1–39:27. DOI: 10.4230/LIPIcs.ECOOP.2023.39. URL: https://doi.org/10.4230/LIPIcs.ECOOP.2023.39.

[74] Y. Zhang, K. Zhang, G. Liu, Y. Cai, and S. Qin. "Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs". In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint arXiv:2401.01114. DOI: 10.1007/s10515-026-00614-z. URL: https://doi.org/10.1007/s10515-026-00614-z.

[75] R. Tang, M. Wang, X. Sun, L. Huang, Y. Huang, and X. Ma. "Converos: Practical Model Checking for Verifying Rust OS Kernel Concurrency". In: *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC)*. USENIX Association, 2025, pp. 1143–1159. URL: https://www.usenix.org/conference/atc25/presentation/tang.

[76] S. Lu, S. Park, E. Seo, and Y. Zhou. "Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics". In: *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2008, pp. 329–339. DOI: 10.1145/1346281.1346323. URL: https://doi.org/10.1145/1346281.1346323.

[77] A. Sundvisson and S. Hofer. "User-Space Device Drivers on Embedded Linux: A Comparative Case Study of C and Rust". Bachelor's thesis. Blekinge Institute of Technology, Department of Software Engineering, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-29775.

[78] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[79] W. Crichton and S. Krishnamurthi. "Profiling Programming Language Learning". In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 29–54. DOI: 10.1145/3649812. URL: https://doi.org/10.1145/3649812.

[80] H. Li, L. Guo, Y. Yang, S. Wang, and M. Xu. "Rust Meets Linux: Lessons from an Evolving Experiment". In: *ACM Transactions on Computer Systems* 44.2 (2026). Extended version of the authors' USENIX ATC '24 study of Rust-for-Linux. DOI: 10.1145/3789260. URL: https://doi.org/10.1145/3789260.

[81] Y. Fan, C. Wang, Z. Dong, T. Liu, and H. Wang. "An Empirical Analysis of Rust Integration in Android Open Source Project". In: *Companion Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering (FSE Companion)*. 2026, pp. 545–556. DOI: 10.1145/3803437.3805227. URL: https://doi.org/10.1145/3803437.3805227.

[82] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. "Energy Efficiency Across Programming Languages: How Do Energy, Time, and Memory Relate?" In: *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*. 2017, pp. 256–267. DOI: 10.1145/3136014.3136031. URL: https://doi.org/10.1145/3136014.3136031.

[83] A. Levy, B. Campbell, B. Ghena, P. Pannuto, P. Dutta, and P. Levis. "The Case for Writing a Kernel in Rust". In: *Proceedings of the 8th Asia-Pacific Workshop on Systems (APSys)*. 2017, pp. 1–7. DOI: 10.1145/3124680.3124717. URL: https://doi.org/10.1145/3124680.3124717.

[84] C. Mo, L. Wang, S. Li, K. Hu, and B. Jiang. "Rust-Shyper: A Reliable Embedded Hypervisor Supporting VM Migration and Hypervisor Live-Update". In: *Journal of Systems Architecture* 142 (2023), p. 102948. DOI: 10.1016/j.sysarc.2023.102948. URL: https://doi.org/10.1016/j.sysarc.2023.102948.

[85] Y. Xiao, Y. Chen, X. Zhao, J. Duan, X. Zhang, W. Li, and X. Tao. "rpkt: A Generic, Safe, and Efficient Userspace Packet Processing Library in Rust". In: *Network and Parallel Computing (NPC 2024)*. Lecture Notes in Computer Science. 2025, pp. 454–465. DOI: 10.1007/978-981-96-2864-3_36. URL: https://doi.org/10.1007/978-981-96-2864-3_36.

[86] C. Cui and H. Xu. "Unleashing the Efficiency of Rust: An Empirical Study of Performance Bugs in Rust Projects". In: *Proceedings of the 2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*. 2025, pp. 371–381. DOI: 10.1109/ISSRE66568.2025.00045. URL: https://doi.org/10.1109/ISSRE66568.2025.00045.

[87] M. Naylor, A. Joannou, A. T. Markettos, P. Metzger, S. W. Moore, and T. M. Jones. "CHERI-SIMT: Implementing Capability Memory Protection in GPUs". In: *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2026, pp. 50–66. DOI: 10.1145/3760250.3762234. URL: https://doi.org/10.1145/3760250.3762234.

[88] Y. Bae, Y. Kim, A. Askar, J. Lim, and T. Kim. "Rudra: Finding Memory Safety Bugs in Rust at the Ecosystem Scale". In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 84–99. DOI: 10.1145/3477132.3483570. URL: https://doi.org/10.1145/3477132.3483570.

[89] X. Zheng, Z. Wan, Y. Zhang, R. Chang, and D. Lo. "A Closer Look at the Security Risks in the Rust Ecosystem". In: *ACM Transactions on Software Engineering and Methodology* 33.2 (2024), 34:1–34:30. DOI: 10.1145/3624738. URL: https://doi.org/10.1145/3624738.

[90] B. Qin, Y. Chen, H. Liu, H. Zhang, Q. Wen, L. Song, and Y. Zhang. "Understanding and Detecting Real-World Safety Issues in Rust". In: *IEEE Transactions on Software Engineering* 50.6 (2024), pp. 1306–1324. DOI: 10.1109/TSE.2024.3380393. URL: https://doi.org/10.1109/TSE.2024.3380393.

[91] L. Schuermann, A. Thomas, and A. Levy. "Encapsulated Functions: Fortifying Rust's FFI in Embedded Systems". In: *Proceedings of the 1st Workshop on Kernel Isolation, Safety and Verification (KISV)*. 2023, pp. 41–48. DOI: 10.1145/3625275.3625397. URL: https://doi.org/10.1145/3625275.3625397.

[92] M. Emre, P. Boyland, A. Parekh, R. Schroeder, K. Dewey, and B. Hardekopf. "Aliasing Limits on Translating C to Safe Rust". In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA1 (2023), pp. 551–579. DOI: 10.1145/3586046. URL: https://doi.org/10.1145/3586046.

[93] A. Sharma, S. Sharma, S. R. Tanksalkar, S. Torres-Arias, and A. Machiry. "Rust for Embedded Systems: Current State and Open Problems". In: *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2024, pp. 2296–2310. DOI: 10.1145/3658644.3690275. URL: https://doi.org/10.1145/3658644.3690275.

[94] I. McCormack, T. Dougan, S. Estep, H. Hibshi, J. Aldrich, and J. Sunshine. "A Mixed Methods Study on the Implications of Unsafe Rust for Interoperation, Encapsulation, and Tooling". In: *ACM Transactions on Software Engineering and Methodology* (2026). DOI: 10.1145/3798277. URL: https://doi.org/10.1145/3798277.

[95] J. Hong and S. Ryu. "Automatically Translating C to Rust". In: *Communications of the ACM* 68.11 (2025), pp. 58–65. DOI: 10.1145/3737696. URL: https://doi.org/10.1145/3737696.

[96] J. Huang, B. Mozafari, G. Schoenebeck, and T. F. Wenisch. "A Top-Down Approach to Achieving Performance Predictability in Database Systems". In: *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*. 2017, pp. 745–758. DOI: 10.1145/3035918.3064016. URL: https://doi.org/10.1145/3035918.3064016.

[97] Y.-S. Liu, S. Qi, P.-Y. Lin, H.-S. Tsai, K. K. Ramakrishnan, and J.-C. Chen. "L^{2}5GC+: An Improved, 3GPP-Compliant 5G Core for Low-Latency Control Plane Operations". In: *Proceedings of the 2023 IEEE 12th International Conference on Cloud Networking (CloudNet)*. 2023, pp. 203–211. DOI: 10.1109/CloudNet59005.2023.10490024. URL: https://doi.org/10.1109/CloudNet59005.2023.10490024.

[98] C. Park and J. Nam. "CtrlBench: Towards Accurate API-Level Benchmarking in 5G Core Control Plane". In: *Proceedings of the 2025 16th International Conference on Information and Communication Technology Convergence (ICTC)*. 2025, pp. 1832–1837. DOI: 10.1109/ICTC66702.2025.11388531. URL: https://doi.org/10.1109/ICTC66702.2025.11388531.

[99] L. Ardito, L. Barbato, R. Coppola, and M. Valsesia. "Evaluation of Rust Code Verbosity, Understandability and Complexity". In: *PeerJ Computer Science* 7 (2021), e406. DOI: 10.7717/peerj-cs.406. URL: https://doi.org/10.7717/peerj-cs.406.

[100] Z. A. Qazi, M. Walls, A. Panda, V. Sekar, S. Ratnasamy, and S. Shenker. "A High Performance Packet Core for Next Generation Cellular Networks". In: *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*. 2017, pp. 348–361. DOI: 10.1145/3098822.3098848. URL: https://doi.org/10.1145/3098822.3098848.

[101] N. Larkin. *Alsoran: A 5G gNodeB Centralized Unit Written in Rust*. Open-source software repository. Proof of concept; development discontinued. Accessed August 2026. URL: https://github.com/nplrkn/alsoran.

[102] Z. Li, T. Huang, V. Narayanan, and A. Burtsev. "Understanding the Overheads of Hardware and Language-Based IPC Mechanisms". In: *Proceedings of the 11th Workshop on Programming Languages and Operating Systems (PLOS)*. 2021, pp. 53–61. DOI: 10.1145/3477113.3487275. URL: https://doi.org/10.1145/3477113.3487275.

[103] V. Narayanan, T. Huang, D. Detweiler, D. Appel, Z. Li, G. Zellweger, and A. Burtsev. "RedLeaf: Isolation and Communication in a Safe Operating System". In: *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2020, pp. 21–39. URL: https://www.usenix.org/conference/osdi20/presentation/narayanan-vikram.

[104] F. Suchert and J. Castrillon. "STAMP-Rust: Language and Performance Comparison to C on Transactional Benchmarks". In: *Benchmarking, Measuring, and Optimizing (Bench 2022)*. Vol. 13852. Lecture Notes in Computer Science. Springer, 2023, pp. 160–175. DOI: 10.1007/978-3-031-31180-2_10. URL: https://doi.org/10.1007/978-3-031-31180-2_10.

[105] A. Gonzalez, D. Mvondo, and Y.-D. Bromberg. "Takeaways of Implementing a Native Rust UDP Tunneling Network Driver in the Linux Kernel". In: *Proceedings of the 12th Workshop on Programming Languages and Operating Systems (PLOS)*. 2023, pp. 18–25. DOI: 10.1145/3623759.3624547. URL: https://doi.org/10.1145/3623759.3624547.

[106] K. Zhuang. "FlowPrioritize: A Model-Driven Priority Scheduling Library for Asynchronous Rust Runtimes". In: *SoftwareX* 35 (2026), p. 102771. DOI: 10.1016/j.softx.2026.102771. URL: https://doi.org/10.1016/j.softx.2026.102771.

[107] O. Andersson. "Programming Language Standards and Their Impact on Memory Safety: C++, Rust and Python". Report LIU-IDA/LITH-EX-G–26/057–SE. Bachelor's thesis. Linköping University, Department of Computer and Information Science, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-226479.

[108] S. Okawa and S. Yamaguchi. "A Performance Study on Rust and C Programs". In: *Proceedings of the 12th International Symposium on Computing and Networking Workshops (CANDARW)*. 2024, pp. 355–359. DOI: 10.1109/CANDARW64572.2024.00066. URL: https://doi.org/10.1109/CANDARW64572.2024.00066.

[109] S. Pirelli and G. Candea. "Safe Low-Level Code Without Overhead is Practical". In: *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2023, pp. 2173–2184. DOI: 10.1109/ICSE48619.2023.00183. URL: https://doi.org/10.1109/ICSE48619.2023.00183.

Phụ lục A Viết tắt và thuật ngữ chính

Các viết tắt và thuật ngữ dùng xuyên suốt luận văn, cho độc giả ngoài lĩnh vực viễn thông.

Bảng A.1: Viết tắt và thuật ngữ chính dùng xuyên suốt luận văn.

Thuật ngữ	Ý nghĩa
RAN	Mạng truy nhập vô tuyến: phần mạng di động nối thiết bị với lõi
NG-RAN	RAN thế hệ tiếp theo, kiến trúc mạng truy nhập vô tuyến 5G
IMT-2030	Khung ITU cho thế hệ sau 5G, thường gọi là 6G
AI-RAN	Kiến trúc trong đó AI và khối lượng RAN chia sẻ một hạ tầng tăng tốc
gNB	Trạm gốc 5G ("next-generation NodeB")
gNB-CU / DU	Đơn vị trung tâm / Đơn vị phân tán: hai nửa tách rời của gNB
gNB-CU-CP	Phần mặt điều khiển của đơn vị trung tâm: lớp hệ thống được nghiên cứu ở đây
UE	Thiết bị người dùng: thiết bị đầu cuối (điện thoại, modem) gắn vào mạng
UECM	Quản lý ngữ cảnh UE: phân hệ quản lý trạng thái theo UE được hiện thực lại ở đây
RRC	Điều khiển tài nguyên vô tuyến: giao thức báo hiệu mặt điều khiển giữa UE và gNB
DRB	Kênh mang dữ liệu vô tuyến theo UE: kênh dữ liệu mặt người dùng thiết lập trong phiên
Mặt điều khiển	Đường báo hiệu và quản lý, trái với đường dữ liệu mặt người dùng
Độ trễ đuôi	Độ trễ phần trăm cao (p99, p99.9), chỉ số thời gian hướng trường hợp xấu nhất
Jitter	Mức độ độ trễ dao động giữa các thao tác, trái với giá trị điển hình
Bão hoà	Tải đề nghị vượt quá khả năng dịch vụ theo kịp và yêu cầu bắt đầu xếp hàng
Shard	Một trong các lát trạng thái UE độc lập, mỗi lát thuộc đúng một luồng worker
FFI	Giao diện hàm nước ngoài: gọi Rust từ C++ qua ABI C
HDR histogram	Biểu đồ tần suất dải động cao, dùng để ghi độ trễ chính xác

Phụ lục B Giao thức tìm kiếm tài liệu

Mục 4.1 báo cáo hai phát hiện mà tìm kiếm này tạo ra. Giao thức được ghi ở đây để khoảng trống mà nghiên cứu tuyên bố có thể được kiểm tra chứ không phải tin theo lời nói.

Tài liệu ứng viên được thu thập từ ACM Digital Library, IEEE Xplore, USENIX và Google Scholar, dùng các tổ hợp thuật ngữ *Rust*, *C++*, *memory safety*, *tail latency*, *systems programming*, *performance comparison*, và *RAN / 5G control plane*. Tìm kiếm giới hạn ở hội nghị có phản biện, trừ khi báo cáo ngành hoặc chính phủ chính là đối tượng nghiên cứu, như các quy định an toàn bộ nhớ và thống kê lỗ hổng của nhà cung cấp. Tập kết quả sau đó được mở rộng bằng truy vết trích dẫn ngược và xuôi từ các bài an toàn Rust cốt lõi, và lọc giữ công trình hoặc đo ngôn ngữ hệ thống bằng thực nghiệm, hoặc hình thức hoá bảo đảm của Rust, hoặc báo cáo kinh nghiệm áp dụng. Tài liệu miền (đặc tả 3GPP và khảo sát O-RAN) được thêm riêng để cố định bối cảnh kiến trúc.

Chỉ các nguồn trên sẽ bỏ sót công trình tiền nhiệm gần nhất, đó là lý do hai nguồn nữa được thêm. Các luận văn do nhà cung cấp lưu trữ gần nghiên cứu này nhất (Mục 7.5) không xuất hiện trong cơ sở dữ liệu trích dẫn nào: không có trong Crossref, OpenAlex, DBLP hay Semantic Scholar. Truy vết xuôi hay ngược không chạm tới chúng, vì không có mục được lập chỉ mục trích dẫn chúng và danh sách tài liệu tham khảo của chúng không được nộp. Chúng được tìm bằng cách tìm trực tiếp trên kho lưu trữ thể chế, DiVA cho các trường Bắc Âu và Finna cho các trường Phần Lan. Cùng hai tìm kiếm đó thiết lập sự vắng mặt ở phía kia. Finna, liên kết các kho luận văn đại học Phần Lan, không trả luận văn nào về Rust trong bối cảnh truy nhập vô tuyến hay viễn thông. Tìm theo tác giả thuộc Nokia trong các ấn phẩm được lập chỉ mục trên OpenAlex cũng không trả kết quả. Các thực thể của hãng giữ khoảng ba vạn công trình được lập chỉ mục, vài trăm trong số đó về mạng truy nhập vô tuyến, và một công trình nhắc Rust, trong một bản thảo trước in không liên quan về giám sát thực thi tin cậy.

Một ghi chú thực tiễn thuộc cùng giao thức, vì nó làm méo tìm kiếm ngây thơ. Truy vấn văn bản tự do cho *Rust* trong các chỉ mục chung xếp bệnh học thực vật và hóa học ăn mòn trên ngôn ngữ lập trình, nên cần truy vấn giới hạn trường hoặc lĩnh vực.

Phụ lục C Thiết kế thống kê: thành viên họ và khả năng phân giải

Mục 4.4 cố định so sánh nào vào họ đã hiệu chỉnh và thiết kế nhìn sâu bao xa vào một khác biệt. Ba điểm đằng sau các cam kết đó được ghi ở đây thay vì trong văn bản chính.

Vì sao một so sánh không phân giải được không miễn phí. Kiểm thử thông lượng đạt được dưới bão hoà sẽ đo lại điểm vận hành chứ không phải mã. Nó cũng tốn các so sánh có thể phân giải điều gì, vì Holm chia ngân sách trên cả họ và một kiểm định không tách gì vẫn chiếm một phần. Qua bão hoà lập luận tương tự chạy ngược. Điều mỗi biến thể vẫn duy trì ở đó không bị ghim vào gì, nên loại trừ nó sẽ bỏ đi đại lượng quan sát duy nhất chế độ đó cung cấp.

Vì sao không có phần trăm nào được kiểm định khi quá tải. Dưới lịch gửi cố định dự định, một producer không theo kịp sẽ tụt lại tích luỹ, nên phần trăm đo ở đó báo cáo tồn đọng tích luỹ trước cửa sổ cắt nó. Gắn giá trị p vào cận dưới sẽ kiểm định bộ điều khiển chứ không phải phần mềm.

Các sàn mà thiết kế có thể chạm. Một kiểm định chính xác trên n cặp không thể trả p hai phía dưới $2/2^n$, và sau hiệu chỉnh một họ m so sánh giá trị điều chỉnh nhỏ nhất có thể là m lần số đó. Đó là nguồn gốc của 0.0000019 và 0.000027 ở Mục 4.4. Một chiến dịch có sàn đã hiệu chỉnh nằm trên ngưỡng đang áp dụng không thể chứng minh ý nghĩa dù hiệu ứng lớn đến đâu, nên báo "không có ý nghĩa" từ chiến dịch như vậy sẽ nói nhiều về số lần lặp hơn về ngôn ngữ.

Phụ lục D Cấu hình máy chủ và chiến dịch

Bảng D.1 liệt kê máy đo của Mục 6.1 so với giao thức bare-metal mà số đuôi cấp testbed yêu cầu. Bảng D.2 gom cấu hình dịch vụ, khối lượng công việc và chiến dịch, các giá trị suy ra của chúng Mục 6.2 trích dẫn.

Bảng D.1: Máy chủ mà kết quả được báo cáo đo trên, so với giao thức mà số đuôi cấp testbed yêu cầu (`bench/TESTBED.md`). Các hàng hai cột trùng là thuộc tính của mã nguồn và trình biên dịch. Các hàng khác nhau là điều một chiến dịch bare-metal vẫn mang lại. Chỉ khác biệt cuối giờ còn tốn một kết quả, vì điểm gối bão hoà và so sánh tài nguyên đã được giải quyết trên máy này: phần còn lại là đuôi xa dưới bão hoà.

Khía cạnh	Dùng cho kết quả báo cáo ở đây	Yêu cầu cho số đuôi cấp testbed
Máy chủ	Máy chủ hai socket chuyên dụng, 2× Xeon E5-2680 v4, 16 vCPU, một nút NUMA	Cùng lớp máy, không hypervisor
Ảo hoá	Khách KVM: tần số và turbo do host điều khiển và không quan sát được	Không. Governor và turbo đặt và xác minh
CPU steal	Khác không, cỡ 10 ms thời gian vCPU mỗi giây	Không
Kernel	Rocky Linux 8.10, kernel bản phân phối	Giống, với `isolcpus` và `nohz_full` trên lõi benchmark
Toolchain	`g++ 15.1.1`, `rustc 1.97`	`g++ 15.1.1`, `rustc 1.97`
Ghim CPU	Lõi 2–15 qua `taskset`, 7 shard, 7 producer	Giống, trên lõi cũng bị gỡ khỏi bộ lập lịch
Đa luồng đồng thời	Vắng trong topology khách thấy. vCPU có chia anh em luồng hay không do host quyết	Vắng hoặc tắt, và xác minh trên máy
Khởi động nóng	Phần đầu mỗi lần chạy bỏ	Phần đầu mỗi lần chạy bỏ
Lần lặp và xen kẽ	20 lần lặp, bốn biến thể xen kẽ mỗi lần	20 lần lặp, bốn biến thể xen kẽ mỗi lần
Ghi môi trường	`bench/capture_env.sh`, lưu cùng mỗi kết quả	`bench/capture_env.sh`, lưu cùng mỗi kết quả
Bộ đếm phần cứng	Không có trong khách	Có: khoảng trống dụng cụ đo rõ nhất
Đủ cho	Tương đương, tiêm lỗi, cấu trúc FFI, điểm gối bão hoà và so sánh quá tải, và so sánh tài nguyên	Thêm đuôi xa dưới bão hoà, và cơ chế đằng sau kết quả tài nguyên và thao tác

Bảng D.2: Cấu hình dịch vụ được kiểm thử, khối lượng công việc điều khiển nó, và chiến dịch đo. Tham số dịch vụ cố định bởi đặc tả đóng băng, một ngoại lệ: số shard đặt khớp lõi máy cung cấp (Mục 6.1). Tất cả, kể cả ngoại lệ đó, giống nhau ở hai ngôn ngữ. Số chiến dịch sinh từ nhật ký đo cùng các bảng báo cáo chúng. Tải đại diện không chọn bằng mắt mà suy ra, là tải quét bận nhất mà cả bốn biến thể build vẫn giữ p99 dưới 5 ms.

Tham số	Giá trị	Ghi chú
Dịch vụ được kiểm thử, giống nhau ở hai ngôn ngữ		
Shard và luồng worker	7	một luồng sở hữu một shard, mỗi loại một trên lõi được ghim
Giới hạn hàng đợi mỗi shard	1 024 yêu cầu	có giới hạn và chặn, nên quá tải quan sát được
Dung lượng bản đồ ban đầu	4 096 mục	dự trữ để trạng thái ổn định không rehash
Bearer mỗi ngữ cảnh	tối đa 8	
Payload báo hiệu	64 byte	
Trần ngữ cảnh mỗi shard	200 000	
Chọn shard	SplitMix64, rồi modulo	ghim: khuếch tán định danh tuần tự
Hash trong shard	FNV-1a, 64-bit	ghim hai bên và kiểm bit-bit
Khối lượng công việc		
Khối lượng	signal, mixed	mixed đi qua toàn vòng đời RAN
UE hoạt động	5 000	
Mức tải đề nghị	14	tới 784 kops/s
Tải đại diện	56 kops/s	suy ra, xem chú thích
Bão hoà đo được	335 kops/s	theo tải đề nghị tới 336 kops/s
Mô hình tải	vòng hở, lịch dự định cố định	độ trễ đo từ thời điểm gửi dự định
Chiến dịch		
Biến thể build	4	hai ngôn ngữ × hai bộ cấp phát
Lần lặp mỗi điểm quét	20	biến thể xen kẽ trong mỗi lần lặp
Lần chạy đo	1120	
Ghi độ trễ	HDR histogram, $[1\,\text{ns}, 3.6\,\text{s}]$, 3 chữ số có nghĩa	cùng thuật toán và cấu hình hai bên
Thống kê tóm tắt	trung vị với CI bootstrap 95%	khoảng báo đầy đủ, không phải nửa độ rộng
So sánh ngôn ngữ	kiểm định hoán vị ghép cặp chính xác	14 kiểm định, hiệu chỉnh Holm
Sàn p có thể đạt	0.0000019 / 0.000027	chưa hiệu chỉnh / sau hiệu chỉnh

Phụ lục E Ví dụ tiêm lỗi

Là minh hoạ cho nghiên cứu tiêm lỗi (Chương 6, RQ2), lỗi use-after-free được trình bày ở cả hai ngôn ngữ. Trong C++ bí danh vào bản đồ ngữ cảnh sống sót sau `erase` giải phóng nó, nên chương trình sau biên dịch sạch và chỉ được AddressSanitizer đánh dấu lúc chạy:

```
std::unordered_map<unsigned long, UeContext> ctxs;
ctxs[1001] = UeContext{1001, 0};
UeContext* p = &ctxs.at(1001);  // alias into the container
ctxs.erase(1001);               // frees the node that p refers to
p->messages_handled += 1;       // use-after-free
```

Chương trình Rust an toàn tương ứng không biên dịch: bộ kiểm tra mượn cấm sửa bản đồ (`remove`) trong khi tham chiếu `p` vào nó vẫn sống, nên lỗi không thể viết ngay từ đầu (lỗi `E0499`):

```
let mut ctxs: HashMap<u64, UeContext> = HashMap::new();
ctxs.insert(1001, UeContext { messages_handled: 0 });
let p = ctxs.get_mut(&1001).unwrap(); // mutable borrow starts
ctxs.remove(&1001);                   // borrow while p live -> rejected
p.messages_handled += 1;              // p used after the remove
```